Integração de Code Review Automatizado

com Large Language Models no Azure DevOps

Relatório de Estágio Curricular

Licenciatura em Engenharia Informática

Estudante	Bruno Teixeira
Número	1230741
Orientador ISEP	Prof. Nuno Morgado
Supervisor Externo	David Mota — DevScope
Instituição	ISEP — Instituto Superior de Engenharia do Porto
Entidade Acolhedora	DevScope
Ano Letivo	2025 / 2026
Data de Entrega	7 de junho de 2026

Declaração de Originalidade

O presente relatório foi elaborado exclusivamente pelo estudante Bruno Teixeira no âmbito do estágio curricular realizado na empresa DevScope, no ano letivo 2025/2026.

O autor declara que todo o trabalho aqui descrito é original e que não recorreu a qualquer fonte sem a devida identificação. As fontes consultadas encontram-se devidamente referenciadas ao longo do documento, em conformidade com as normas académicas em vigor no ISEP.

O estágio decorreu sob orientação do Prof. Nuno Morgado (ISEP) e do Eng.º David Mota (DevScope), tendo o autor plena consciência de que a utilização de material alheio sem citação constitui plágio, sujeito às penalidades previstas no regulamento académico.

Porto, junho de 2026

Bruno Teixeira
Estudante

Agradecimentos

Quatro meses de estágio envolvem muito mais do que código e documentação. Há decisões arquiteturais que tomam forma em conversas informais, bloqueios que se desbloqueiam com a ajuda certa no momento certo — e este relatório não existiria sem isso.

Ao Professor Nuno Morgado agradeço, antes de mais, a disponibilidade para discutir as decisões arquiteturais mais críticas do projeto, muitas vezes sem soluções claras à partida. O rigor científico que transmitiu ao longo de todo o processo foi determinante para a qualidade do que aqui se apresenta.

Ao Engenheiro David Mota, da DevScope, devo muito da forma como encaro hoje a engenharia de software. A integração num ambiente de trabalho exigente, o apoio na configuração da infraestrutura Azure nas fases mais críticas, e — sobretudo — a autonomia concedida para explorar abordagens não convencionais foram decisivas para que o projeto chegasse onde chegou.

À equipa da DevScope, pelo ambiente genuíno de partilha técnica. A cultura de inovação que encontrei lá transformou o que poderia ter sido apenas um estágio num processo real de aprendizagem.

Por fim, à minha família e amigos: o suporte incondicional ao longo de todo o percurso académico nunca foi dado como adquirido.

Resumo

A revisão de código é das práticas mais subestimadas da engenharia de software — e das mais críticas. Quando o volume de Pull Requests sobe, o problema deixa de ser só a demora: a equipa acaba bloqueada à espera de feedback que tarda. Este relatório descreve o desenvolvimento de uma Prova de Conceito para automatizar essa revisão diretamente no Azure DevOps, com modelos de linguagem que correm dentro do perímetro de segurança da organização.

O sistema usa uma arquitetura multiagente com roteamento dinâmico: um agente foca-se em vulnerabilidades técnicas, outro cruza o código com os critérios de aceitação dos Work Items. Para medir o desempenho com rigor, foram construídos seis conjuntos de teste com 102 defeitos injetados e catalogados — o que permitiu comparar três arquiteturas distintas nas mesmas condições. A opção por defeitos conhecidos em vez de código de produção foi deliberada: sem ground truth verificável, não é possível calcular taxas de falsos negativos de forma matemática.

Os resultados superaram o que estava previsto. A arquitetura multiagente detetou 88% dos defeitos injetados e apanhou todas as 26 vulnerabilidades críticas, sem excepção. A latência média ficou abaixo dos 90 segundos e o custo por Pull Request abaixo dos cinco cêntimos. A avaliação revelou também um ponto cego partilhado pelos modelos testados: funções com assinaturas aparentemente conformes que escondem corpos não conformes passam despercebidas. É esta limitação que orienta o trabalho futuro em análise de fluxo de dados.

No geral, a abordagem é tecnicamente sólida e economicamente viável para uso real em empresa — e faz algo que as ferramentas comerciais disponíveis não conseguem: manter o código-fonte dentro do perímetro organizacional.

Palavras-chave (Tema): Code Review Automatizado, Inteligência Artificial, Large Language Models, Azure DevOps, Arquitetura Multiagente, Clean Architecture.

Palavras-chave (Tecnologias): Docker, REST API, Webhooks, Python, Azure AI Foundry

Abstract

Code review is one of the most underestimated practices in software engineering — and one of the most important. As pull request volume scales up, the cost is no longer just time: the whole delivery pipeline stalls waiting for feedback. This report covers the development of a proof of concept that brings automated review directly into Azure DevOps, running large language models inside the organisation's own security boundary.

The architecture is multi-agent with dynamic model routing: one agent handles technical vulnerabilities, another checks code against the acceptance criteria defined in Work Items. To evaluate this rigorously, six test sets were built with 102 hand-crafted, catalogued defects — giving a controlled baseline to compare three architectural configurations head-to-head. Using injected defects rather than production code was a deliberate choice: without a known ground truth, false negative rates cannot be calculated with any mathematical precision.

The results came in above expectations. The multi-agent setup caught 88% of injected defects overall and identified every one of the 26 critical-severity vulnerabilities, with average latency under 90 seconds and cost below five cents per pull request. The evaluation also surfaced a structural blind spot shared by both models: functions whose signatures look compliant but whose bodies are not — a pattern that no static file-level analysis can catch, and the main motivation for future work on data flow tracing.

Taken together, the approach is both technically sound and economically realistic for enterprise use — and it does something the available commercial alternatives do not: keep the source code inside the organisation at all times.

Keywords (Theme): Automated Code Review, Artificial Intelligence, Large Language Models, Azure DevOps, Multi-Agent Architecture, Clean Architecture.

Keywords (Technologies): Docker, REST API, Webhooks, Python, Azure AI Foundry

Índice

Declaração de Originalidade i
Agradecimentos ii
Resumo iii
Abstract iv
Índice v
Índice de Figuras viii
Índice de Tabelas ix
Glossário e Acrónimos x
1 Introdução 1
1.1 Enquadramento e Contexto 1
1.2 Descrição do Problema 1
1.3 Objetivos 2
1.4 Abordagem 2
1.5 Contributos 3
1.6 Planeamento 4
1.7 Estrutura do relatório 5
2 Estado da Arte 5
2.1 Revisão de Código: Fundamentos e Desafios 5
2.1.1 O Processo de Code Review no Desenvolvimento Moderno 5
2.1.2 Limitações do Processo Manual e a Oportunidade de Automação 6
2.2 Trabalhos Relacionados 6
2.2.1 Soluções Comerciais Existentes e o Dilema da Privacidade 6
2.2.2 Análise Comparativa e Lacunas das Soluções Existentes 7
2.2.3 Investigação Académica em Code Review Automatizado 9
2.3 Large Language Models (LLMs) na Engenharia de Software 11
2.3.1 Capacidades e Raciocínio (Chain-of-Thought) 11
2.3.2 Modelos Open-Source Relevantes 12
2.4 Síntese e Posicionamento da Proposta 14
3 Análise e Desenho da Solução 14
3.1 Domínio do Problema 14
3.1.1 Entidades e Conceitos Centrais 16
3.1.2 Modelo de Domínio e Relações entre Entidades 17
3.2 Requisitos Funcionais e Não Funcionais 18
3.3 Desenho 21
3.3.1 Evolução do Âmbito e Transição Arquitetural 21

3.3.2 Arquitetura da Solução (Ports & Adapters) 22
3.3.3 Justificação da Arquitetura Multiagente 25
3.3.4 Fluxo de Execução Orquestrado 26
3.3.5 Seleção e Justificação do Modelo Base 29
4 Implementação 30
4.1 Descrição da Implementação 30
4.1.1 Ambiente e Stack Tecnológico 30
4.1.2 Protótipo e Integração com o Azure DevOps 32
4.1.3 Migração para a Cloud e Modelos 35
4.1.4 Clean Architecture e Governança 36
4.1.5 Parsing e Segurança 41
4.1.6 Otimização e Multiagente 42
4.1.7 Escalabilidade e Observabilidade 46
4.2 Testes 48
4.2.1 Protocolo Experimental 48
4.2.2 Testes de Componentes 50
4.3 Avaliação da Solução 57
4.3.1 Análise de Custos Operacionais 58
4.3.2 Síntese Comparativa e Limitações 60
4.3.3 Ameaças à Validade 62
5 Conclusões e Trabalho Futuro 63
5.1 Objetivos Concretizados 63
5.2 Limitações e Trabalho Futuro 67
5.3 Apreciação Final 68
Referências 70
Anexos 73
Anexo A — Ata de Início de Estágio 73
Anexo B — Exemplos de Output do Sistema 74
B.1 Screenshot: Comentário de Code Review publicado no ADO 74
B.2 Screenshot: Comentário de Requirement Review publicado no ADO 75
B.3 Screenshot: Terminal ngrok (túnel HTTPS ativo) 76
B.4 Screenshot: Logs Docker em estado idle 77
B.5 Screenshot: Logs Docker em processamento de PR 77
B.6 Screenshot: Relatório de Cobertura de Testes Unitários 78
Anexo C — Resultados Completos da Avaliação 79
Tabela C.1 — Matriz de confusão global por arquitetura 79
Tabela C.2 — Métricas operacionais por execução 80
Tabela C.3 — Identificação dos Pull Requests de teste 82

Anexo D — Diagramas Arquiteturais e de Processo (Renderizados a partir de PlantUML) 83
D.1 Modelo de Domínio DDD (Entidades) 83
D.2 Camadas de Domínio e Aplicação 84
D.3 Sequência Completa de Orquestração 85
D.4 C4 System Context 86
D.5 C4 Container Diagram 88
D.6 C4 Component Diagram 89
D.7 Vista de Deployment (Docker Compose) 90
D.8 Fluxo de Atividade do Sistema 91
D.9 Diagrama de Classes Granular 92
D.10 Padrões de Design (Clean Architecture) 93
D.11 Sequência de Deduplicação Distribuída (Redis) 94
D.12 Mapeamento Lógico Hexagonal 95

Índice de Figuras

Figura 1 - Evolução das ferramentas de code review: da análise estática manual à automação multiagente baseada em IA (2013–2025) (Elaboração própria) 6
Figura 2 - O Dilema da Privacidade: Cloud Pública Genérica vs. Perímetro Empresarial Governado (Azure AI Foundry) (Elaboração própria) 7
Figura 3 - Fluxo de raciocínio do agente de IA baseado em Chain-of-Thought. (Adaptado de Wei et al.[3]) 12
Figura 4 - Diagrama de Visão Geral (Elaboração própria) 15
Figura 5 - Diagrama do Modelo de Domínio (Domain Core) do ScopeReview AI (Elaboração própria) 18
Figura 6 - Diagrama Use Case (Elaboração própria) 20
Figura 7 - Arquitetura Ports & Adapters, ilustrando o isolamento do Core de Domínio (orquestrador e agentes) face às integrações externas (Elaboração própria) 24
Figura 8 - Fenómeno Lost in the Middle, ilustrando a distribuição de atenção dos LLMs e o risco inerente de negligência de critérios críticos em documentação e contextos de código extensos (Adaptado de Liu et al. [36]) 27
Figura 9 - Princípio conceptual de Orquestração Multiagente baseado em Model Routing (Elaboração própria, inspirado em Moslem & Kelleher [37]) 30
Figura 10 - Diagrama de sequência do processamento assíncrono de Webhooks (Elaboração própria) 34
Figura 11 - Diagrama de sequência da orquestração multiagente e partilha de contexto (Elaboração própria) 39
Figura 12 - Fluxo de execução do Motor de Normalização de Veredictos (Elaboração própria) 40
Figura 13 - Arquitetura de Orquestração Multi-LLM e fluxo de dados em tempo de execução (Elaboração própria) 45
Figura 14 - Evolução da Taxa de Deteção por Severidade e Arquitetura (Teste Massive) (Elaboração própria) 52
Figura 15 - Pipeline de Higienização de Contexto e redução sequencial de tokens (Elaboração própria) 56
Figura 16 - Rácio Qualidade/Custo: % Deteção de Bugs Críticos vs. Custo por PR (€) (Elaboração própria) 60
Figura 17 - Posicionamento competitivo do ScopeReview AI face às alternativas comerciais (Elaboração própria) 65

Índice de Tabelas

Tabela 1 - Planeamento de Estágio 4
Tabela 2 - Comparação de Ferramentas de Code Review com IA (Elaboração própria, dados recolhidos das fontes [5], [6], [12], [13]) 8
Tabela 3 - Análise comparativa de trabalhos relacionados e posicionamento da solução (Elaboração própria) 9
Tabela 4 - Panorama de Modelos de Linguagem para Análise de Código (Elaboração própria, dados recolhidos das fontes [26], [27], [28], [29], [30], [31]) 13
Tabela 5 - Identificação e descrição das entidades centrais do Domain Core (Elaboração própria) 16
Tabela 6 - Requisitos Funcionais (Elaboração própria) 19
Tabela 7 - Requisitos Não Funcionais (Elaboração própria) 20
Tabela 8 - Stack tecnológico do projeto ScopeReview AI (Elaboração própria) 31
Tabela 9 - Separação de responsabilidades na arquitetura Ports & Adapters implementada (Elaboração própria) 37
Tabela 10 - Estrutura do Prompt com injeção do Knowledge Ledger e restrições explícitas (Elaboração própria) 47
Tabela 11 - Características dos seis conjuntos de teste do benchmark final (Elaboração própria) 49
Tabela 12 - Deteção de vulnerabilidades nos seis testes por arquitetura e severidade (Elaboração própria) 51
Tabela 13 - Evolução da taxa de deteção entre testes Medium e Massive (Elaboração própria) 52
Tabela 14 - Falsos negativos na validação de requisitos por arquitetura (Elaboração própria) 53
Tabela 15 - Complementaridade entre análise estática e revisão semântica por LLM (Elaboração própria) 54
Tabela 16 - Impacto dos mecanismos de higienização (Elaboração própria) 56
Tabela 17 - Poupança estimada por PR com higienização (Elaboração própria) 57
Tabela 18 - Custo estimado por PR por arquitetura e teste (Elaboração própria) 58
Tabela 19 - Projeção de custo operacional mensal (Elaboração própria) 59
Tabela 20 - Síntese do cumprimento dos objetivos propostos (Elaboração própria) 64
Tabela 21 - Verificação das Hipóteses e Requisito Arquitetural (Elaboração própria) 66

Glossário e Acrónimos

AC — Acceptance Criteria — Critérios definidos nos Work Items que o código tem de satisfazer para o Pull Request ser aceite

API — Application Programming Interface — Contrato técnico que define como dois sistemas de software se comunicam entre si

AST — Abstract Syntax Tree — Árvore de Sintaxe Abstrata. Representação hierárquica da estrutura de um programa, usada neste projeto para compressão de contexto e triagem de ficheiros

Azure AI Foundry — Plataforma Microsoft para IA generativa em contexto empresarial. Permite invocar LLMs via API sem gerir GPUs diretamente, e garante que os dados ficam dentro do perímetro de segurança da organização

Azure DevOps — Plataforma Microsoft que centraliza repositórios Git, Pull Requests, pipelines CI/CD e gestão de Work Items num único ecossistema

CI/CD — Continuous Integration / Continuous Delivery — Conjunto de práticas que automatizam a integração e entrega de código, reduzindo o intervalo entre uma alteração e a sua entrada em produção

CoT — Chain-of-Thought — Técnica de prompting em que o modelo explicita passos de raciocínio intermédios antes de chegar a uma conclusão. Estudada por Wei et al. como forma de melhorar a qualidade das respostas em tarefas complexas

DDD — Domain-Driven Design — Metodologia de design de software que centra a modelação na lógica de negócio, usando uma linguagem partilhada entre programadores e especialistas de domínio

DevScope — Empresa de consultoria e engenharia de software especializada no ecossistema Microsoft, Cloud Computing e metodologias DevOps.

Docker — Plataforma de contentorização usada neste projeto para garantir que o ambiente de desenvolvimento e o de produção se comportam de forma idêntica

F1-Score (F1) — Métrica harmónica que equilibra a Precisão e a Revocação num único indicador, penalizando sistemas que otimizam apenas uma das dimensões. Fórmula: $2 \times \frac{Precisão \times Revocação}{Precisão + Revocação}$

Git Diff — Representação textual das diferenças entre versões de ficheiros num repositório Git

HAR — Human Acceptance Rate — Proporção de comentários automáticos que os programadores efetivamente aceitam e resolvem. Principal indicador de eficácia real em sistemas de code review assistido por IA em produção

HMAC — Hash-based Message Authentication Code — Mecanismo criptográfico que garante que uma mensagem não foi alterada e veio de quem diz ter vindo. Usado neste projeto para validar a autenticidade dos webhooks recebidos

ISEP — Instituto Superior de Engenharia do Porto

LLM — Large Language Model — Modelo de linguagem de grande escala, treinado em volumes massivos de texto e código. Neste projeto, os LLMs atuam como extratores semânticos — não como árbitros de decisão

MaaS — Model-as-a-Service — Modelo de consumo em que os LLMs são acedidos via API, sem que a organização precise de gerir servidores ou GPUs

MoE — Mixture-of-Experts — Arquitetura em que o modelo ativa apenas uma fração dos seus parâmetros por inferência

MoSCoW — Técnica de priorização de requisitos em quatro categorias: Must have (obrigatório), Should have (desejável), Could have (opcional) e Won't have (fora de âmbito)

NFR — Non-Functional Requirements — Requisitos Não Funcionais. Características de qualidade do sistema analisadas pelos agentes

PAT — Personal Access Token — Credencial de acesso programático à REST API do Azure DevOps, configurada com âmbito de permissões mínimas necessárias

PoC — Proof of Concept — Implementação focada em validar se uma abordagem funciona, antes de investir numa solução de produção completa

PR — Pull Request — mecanismo de proposição e revisão de alterações de código numa plataforma de gestão de repositórios

Precision — Métrica de classificação binária que quantifica a proporção de verdadeiros positivos face ao total de alertas emitidos pelo sistema, medindo o nível de "ruído" ou falsos positivos. Fórmula: $\frac{VP}{VP+FP}$

Prompt Engineering — Arte de formular instruções para LLMs de modo a obter respostas mais precisas, consistentes e acionáveis

Recall — Métrica de classificação binária que quantifica a proporção de defeitos reais que foram efetivamente identificados pelo sistema. Fórmula: $\frac{VP}{VP+FN}$

REST — Representational State Transfer — Estilo arquitetural para APIs web baseadas em HTTP

RF — Functional Requirements — Requisitos Funcionais. Declarações das funcionalidades, serviços lógicos e comportamentos que o sistema deve obrigatoriamente executar

RGPD — Regulamento Geral sobre a Proteção de Dados — Regulamento UE 2016/679. Define as regras de proteção de dados pessoais na União Europeia e é um dos principais motivadores do requisito de soberania de dados deste projeto

SLM — Small Language Model — Modelo de linguagem de menor dimensão (tipicamente menos de 10 mil milhões de parâmetros), utilizado na análise de mercado deste projeto.

SMA — Multi-Agent Systems — Arquiteturas computacionais descentralizadas onde múltiplos agentes autónomos interagem num ambiente compartilhado.

ScopeReview AI — Nome dado à PoC desenvolvida neste estágio: um sistema multiagente que revisa código e requisitos diretamente no Azure DevOps, mantendo todo o processamento dentro do perímetro da organização

Webhook — Notificação HTTP enviada automaticamente por uma plataforma quando ocorre um evento relevante, neste caso, a criação ou atualização de um Pull Request no Azure DevOps

1 Introdução

Este relatório resulta do estágio curricular que realizei na DevScope, entre fevereiro e junho de 2026, no âmbito da Licenciatura em Engenharia Informática do Instituto Superior de Engenharia do Porto (ISEP). O trabalho centrou-se num problema concreto que a empresa enfrenta no dia-a-dia: a ausência de uma solução de revisão de código automatizada que funcione nativamente no Azure DevOps sem comprometer a privacidade do código-fonte. A resposta desenvolvida passa pela integração de Large Language Models (LLMs) diretamente no fluxo de revisão da plataforma.

1.1 Enquadramento e Contexto

A revisão de código é uma das práticas mais consolidadas na engenharia de software moderna. Não é apenas um mecanismo de deteção de defeitos, é também uma forma de partilhar conhecimento técnico e garantir conformidade com padrões de codificação [1]. Há um número que costuma surpreender quem não trabalha diretamente com isto: segundo estudos referidos na literatura, os programadores gastam entre 20% a 25% do seu tempo de desenvolvimento em atividades de Code Review [2]. Cerca de quinto do tempo de trabalho, em revisão. Quando o volume de Pull Requests aumenta, esse esforço escala linearmente, e o processo que devia garantir qualidade torna-se, ele próprio, um problema.

Na DevScope, o ecossistema de trabalho é exclusivamente Azure DevOps. Isso cria uma assimetria difícil de ignorar: ao contrário de plataformas como GitHub ou GitLab, o Azure DevOps não dispõe de mecanismos nativos de revisão assistida por IA. As alternativas disponíveis no mercado resolvem o problema à superfície, mas ao custo de transferir o código-fonte para infraestruturas externas não auditadas, algo incompatível com as políticas internas de conformidade e confidencialidade.

1.2 Descrição do Problema

O problema central é a falta de uma solução que opere nativamente no Azure DevOps, respeite a privacidade do código-fonte e tenha um custo operacional realista em simultâneo. As ferramentas comerciais analisadas no Capítulo 2 resolvem parte da equação técnica, mas partilham uma limitação estrutural comum, processam o código em servidores externos.

Num contexto europeu, isso não é apenas inconveniente. O código-fonte de uma empresa contém frequentemente lógica de negócio proprietária, schemas de bases de dados, ou mesmo dados pessoais em mockups de testes. Enviá-lo para infraestruturas de terceiros colide diretamente com o

Regulamento Geral sobre a Proteção de Dados (RGPD) e com diretivas de proteção de segredos comerciais, um risco que o episódio de fuga de código na Samsung, em 2023, tornou difícil de ignorar para qualquer organização que leve a conformidade a sério. Desenvolver uma alternativa proprietária exige, portanto, uma arquitetura que mantenha os dados dentro do perímetro corporativo sem sacrificar a capacidade analítica dos LLMs, e que seja economicamente viável à escala industrial.

1.3 Objetivos

O trabalho materializa-se numa Prova de Conceito (PoC) com um objetivo claro: integrar modelos de linguagem no fluxo de revisão do Azure DevOps, mantendo todo o processamento dentro do perímetro de segurança da organização. Para tornar a validação rigorosa, a execução foi estruturada em quatro objetivos, cada um com critérios de sucesso concretos e mensuráveis:

- Análise de código local: Extrair e processar as diferenças de código (git diff) via REST API, garantindo 100% do processamento dentro do perímetro seguro do Azure AI Foundry, sem exfiltração de dados para terceiros;
- Auditoria Técnica Bidimensional: Identificar vulnerabilidades de segurança, violações de diretrizes arquiteturais e oportunidades de refatoração, providenciando justificação técnica para os achados;
    - Critério de Sucesso: Atingir uma Revocação (Recall) superior a 80%, garantindo a interceção mandatória de 100% das anomalias de severidade crítica.
- Validação de Conformidade Funcional: Cruzar as alterações propostas com os critérios de aceitação (Acceptance Criteria) associados aos Work Items, gerando resumos técnicos que evidenciem o impacto no negócio;
- Sustentabilidade Operacional: Implementar mecanismos de compressão semântica e deduplicação que garantam a viabilidade económica da integração contínua.
    - Critério de Sucesso: Manter o custo operacional médio inferior a €0,05 por Pull Request.

1.4 Abordagem

Do ponto de vista técnico, o desenvolvimento seguiu um ciclo iterativo e empírico, assente no paradigma de Agentes Autónomos de IA [1]. A lógica central é relativamente direta: um serviço backend interceta notificações via Webhooks, processa o código com recurso a Chain-of-Thought (CoT), técnica que Wei et al. [3] demonstraram ser eficaz para elicitar raciocínio lógico em LLMs, e publica feedback estruturado diretamente nas linhas afetadas do Pull Request. Uma decisão de design que se revelou

determinante foi tornar a arquitetura agnóstica em relação ao modelo subjacente: isso permitiu substituir e rotear dinamicamente entre LLMs sem tocar na lógica de integração, o que mais tarde se mostrou essencial quando os modelos disponíveis na plataforma mudaram de forma inesperada.

Para garantir o rigor metodológico, o desenvolvimento orientou-se por um requisito arquitetural e duas hipóteses testáveis, validadas empiricamente no Capítulo 4:

- **R1 — Soberania de Dados (Requisito Arquitetural):** A execução dos modelos dentro do perímetro do Azure AI Foundry assegura, por desenho, que o código-fonte não é exposto a infraestruturas externas;
- **H1 — Redução do Lead Time (Hipótese Empírica 1):** A automação da triagem inicial reduz o tempo de feedback técnico, mantendo a latência média de execução abaixo de 90 segundos e mitigando o estrangulamento causado pela indisponibilidade de revisores;
- **H2 — Consistência Analítica (Hipótese Empírica 2):** O sistema uniformiza a verificação de diretrizes e vulnerabilidades em todas as submissões, atingindo um Recall superior a 80%, compensando a fadiga humana em revisões extensas.

1.5 Contributos

Invocar um LLM para analisar código não é, por si só, uma novidade. O que distingue esta PoC é a combinação específica de três eixos funcionais: integração nativa e segura no ecossistema Microsoft; orquestração dinâmica de modelos (Model Routing); e validação bidimensional simultânea (qualidade técnica e conformidade funcional). Nenhuma das ferramentas analisadas na literatura resolve os três em simultâneo — e é essa interseção que oferece à organização algo que o mercado ainda não disponibiliza: uma alternativa proprietária com soberania de dados, custo previsível e análise consistente, independente do número de revisores disponíveis.

A mais-valia desta PoC materializa-se em três contributos originais face ao estado da arte:

- Arquitetura Multiagente com Model Routing: Superação do paradigma monolítico através da especialização de input e roteamento dinâmico entre modelos de raciocínio (slow thinking) e arquiteturas Mixture-of-Experts.
- Governança Determinística sobre IA Probabilística: Separação formal entre o LLM (extrator semântico) e o motor de domínio (autoridade de decisão), mitigando alucinações de aprovação.
- Validação Bidimensional (Técnica + Funcional): Integração nativa de critérios de aceitação (Work Items) no pipeline de auditoria, endereçando a lacuna de ferramentas que se limitam à análise sintática.

1.6 Planeamento

A execução do estágio estruturou-se em cinco fases sequenciais, totalizando aproximadamente quatro meses de trabalho. O planeamento detalhado encontra-se na Tabela 1, com marcos de entrega definidos para cada etapa.

Tabela 1 - Planeamento de Estágio

Fase	Nome	Período	Atividades
F1	Ramp-up	23 fev – 09 mar	Integração na empresa e primeiro contacto com as tecnologias envolvidas.
F2	Estado da Arte e Análise	09 mar – 13 abr	Estudo de AI Agents, análise de ferramentas concorrentes, levantamento de requisitos e desenho da arquitetura.
F3	Implementação da PoC	14 abr – 10 mai	Desenvolvimento do Webhook Receiver, integração com o LLM e geração de comentários automáticos nos PRs.
F4	Testes e Validação	11 mai – 31 mai	Análise comparativa e recolha de métricas de avaliação com Pull Requests de validação.
F5	Relatório Final	Até 07 jun	Redação dos capítulos de Implementação, Avaliação e Conclusões.

Desde o início ficou claro que um projeto com esta dependência de infraestruturas externas precisava de uma análise de riscos contínua, não um documento estático feito no arranque. O risco mais evidente era a volatilidade do catálogo de modelos do Azure AI Foundry, agravada pelos constrangimentos naturais de uma subscrição académica. Esse risco materializou-se mesmo, durante a fase F3: o modelo que estava planeado para os testes de integração foi descontinuado da plataforma sem aviso prévio. A adoção de uma arquitetura agnóstica (Ports & Adapters, detalhada no Capítulo 3) permitiu migrar entre famílias de modelos, Phi-4, Llama 3.3, o4-mini, sem refatorar o núcleo de domínio ou a lógica de negócio. O que começou como um risco operacional acabou por ser, na prática, uma validação empírica da resiliência do desenho arquitetural.

1.7 Estrutura do relatório

O relatório divide-se em cinco capítulos. O Capítulo 2 cobre o estado da arte com as ferramentas comerciais existentes, capacidades dos modelos de linguagem e trabalhos académicos relevantes. O Capítulo 3 descreve a arquitetura da solução proposta, incluindo o modelo de domínio e os requisitos funcionais e não funcionais. No Capítulo 4 está documentada a implementação técnica da PoC: as iterações de desenvolvimento, a configuração do ecossistema multiagente e a metodologia usada para validar as hipóteses. O Capítulo 5 fecha o relatório com uma análise do grau de cumprimento dos objetivos, as limitações identificadas e as direções para trabalho futuro.

2 Estado da Arte

Este capítulo cobre o terreno que precede e motiva o projeto. Começa pelo processo de revisão de código: o que é, como falha e onde a automação pode ajudar. Posteriormente passa pelas ferramentas comerciais existentes e pelos modelos de linguagem relevantes, e termina com o posicionamento da proposta face ao que já existe

2.1 Revisão de Código: Fundamentos e Desafios

2.1.1 O Processo de Code Review no Desenvolvimento Moderno

Code Review é o processo em que os pares examinam o código-fonte antes de este ser integrado, com o objetivo de encontrar defeitos, verificar conformidade com padrões e partilhar conhecimento. Bacchelli e Bird [2] mostraram, num estudo de referência, que os programadores esperam muito mais do que bug hunting: querem melhorar a legibilidade, validar decisões de design, aprender com os colegas. Uma revisão eficaz é pedagógica, não apenas corretiva, e essa distinção tem consequências diretas quando se pensa em automatizar o processo.

Na prática, a escala quebra este ideal. Com ciclos CI/CD curtos e filas de PRs a crescer, a fadiga cognitiva instala-se e a profundidade da análise cai. Bacchelli e Bird [2] documentaram que a maioria dos comentários visa compreensão e melhoria, não só correção. Isto torna impossível garantir quando o revisor tem dez PRs à espera. Automatizar a primeira passagem não substitui o revisor: liberta-o para as decisões que realmente exigem julgamento humano. A Figura 1 ilustra como esta evolução aconteceu ao longo da última década.

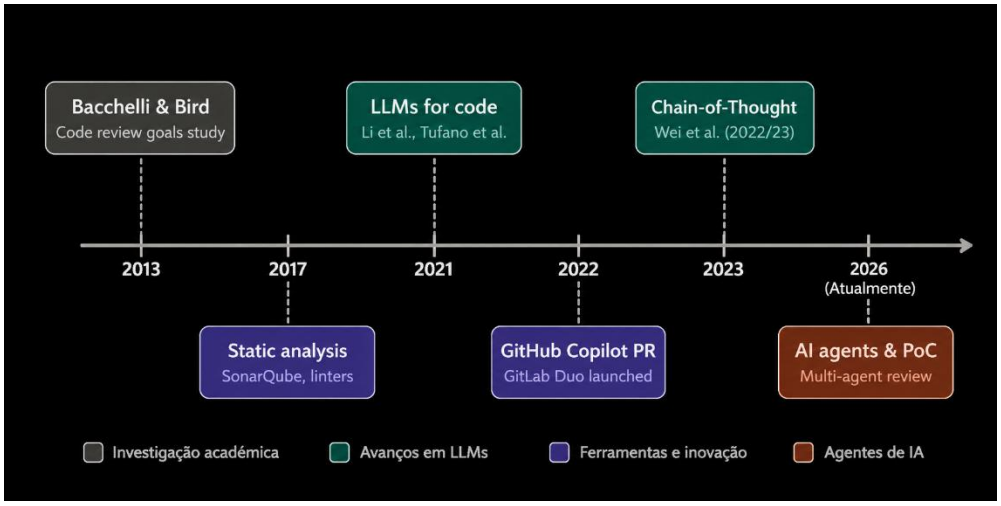


Figura 1 - Evolução das ferramentas de code review: da análise estática manual à automação multiagente baseada em IA (2013–2025) (Elaboração própria)

2.1.2 Limitações do Processo Manual e a Oportunidade de Automação

O problema escala mal. Numa equipa com cadência alta de PRs, os engenheiros seniores acabam a gastar uma fração crescente do seu tempo em revisão, o que é simultaneamente caro e frágil: quando estão sobrecarregados, as revisões ficam superficiais e defeitos críticos passam. Delegar as tarefas mecânicas (verificação de conformidade, deteção de padrões conhecidos, sumário técnico do PR) a ferramentas automatizadas é a saída natural. No caso da DevScope, a ausência de mecanismos nativos no Azure DevOps torna isso mais urgente: as extensões disponíveis no mercado resolvem parte do problema, mas à custa de enviar o código para fora.

A ideia de automatizar partes da revisão não é nova. Balachandran [4] mostrou em 2013 que recomendar revisores automaticamente e combinar isso com análise estática já reduzia o esforço sem degradar a qualidade. O que mudou com os LLMs é o alcance: já não se trata apenas de detetar padrões por regras, mas de compreender semântica, cruzar código com requisitos de negócio e raciocinar sobre o que devia estar presente, mas não está. É esse salto qualitativo que este projeto explora.

2.2 Trabalhos Relacionados

À medida que os projetos cresceram em complexidade, a revisão manual foi ficando cada vez menos sustentável. O mercado respondeu com ferramentas baseadas em IA e a evolução foi rápida. Mas, como se verá, as soluções disponíveis deixam uma lacuna específica por preencher: a privacidade do código-fonte.

2.2.1 Soluções Comerciais Existentes e o Dilema da Privacidade

Soluções como GitHub Copilot, GitLab Duo e CodeRabbit [5], [6], [7] lideram o mercado de Code Review assistido por IA, oferecendo funcionalidades avançadas de sumarização e sugestão em tempo real.

O problema é direto: o código sai da empresa. As políticas de retenção existem no papel, mas são difíceis de auditar externamente, e Al-Maamari [8] documenta práticas que não passam num escrutínio RGPD. A fuga de código da Samsung em 2023 [9] tornou este risco difícil de ignorar para qualquer organização com código proprietário sensível. O Azure AI Foundry resolve isto por arquitetura: o código fica no perímetro da organização e não alimenta modelos externos [10]. É essa garantia que justifica o requisito RNF01 e, por extensão, a decisão de não usar as ferramentas comerciais disponíveis.

A Figura 2 ilustra o contraste: em baixo, o fluxo das soluções comerciais, onde o código passa inevitavelmente por infraestruturas de terceiros. Em cima, o Azure AI Foundry a manter tudo dentro do perímetro organizacional, sem exfiltração [11].

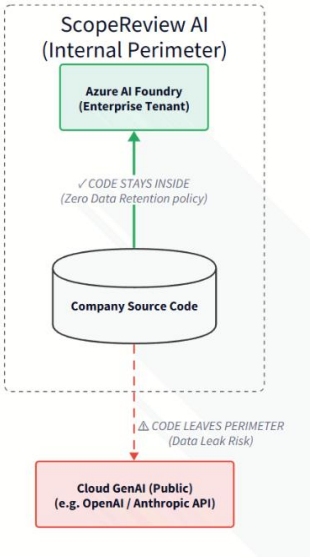


Figura 2 - O Dilema da Privacidade: Cloud Pública Genérica vs. Perímetro Empresarial Governado (Azure AI Foundry) (Elaboração própria)

2.2.2 Análise Comparativa e Lacunas das Soluções Existentes

O mercado cresce depressa, mas com um ponto cego específico. Para perceber onde, foram definidos quatro critérios que traduzem as necessidades concretas da DevScope: suporte nativo ao Azure DevOps, custo operacional realista, garantia de que o código não sai da infraestrutura controlada, e acesso a modelos open-source. A Tabela 2 mostra o resultado.

Tabela 2 - Comparação de Ferramentas de Code Review com IA (Elaboração própria, dados recolhidos das fontes [5], [6], [12], [13])

Ferramenta	Suporte ADO	Custo Licença	Código Permanece Local	Modelos Open-Source
GitHub Copilot PR	Não	Pago (Enterprise)	Não	Não
GitLab Duo	Não	Pago (Ultimate)	Não	Não
CodiumAI PR-Agent	Sim	Freemium / Pago	Parcial (self-host)	Parcial[1]
CodeRabbit	Sim	Freemium / Pago	Não[2]	Não
SonarQube + IA	Sim	Freemium / Pago	Sim (self-host)	Não
ScopeReview AI (PoC)	Sim	Gratuito[3]	Sim (perímetro Azure)	Sim

O primeiro corte é óbvio: GitHub Copilot e GitLab Duo não funcionam no Azure DevOps, pois estão presas aos seus próprios ecossistemas [5], [6].

Das restantes, nenhuma cumpre os quatro critérios ao mesmo tempo. As mais próximas, CodiumAI e CodeRabbit, tropeçam nos mínimos de licenciamento: o CodeRabbit Enterprise exige 500 lugares [7], [13] o que está fora de qualquer equipa de dimensão normal. A solução desenvolvida neste projeto não tem mínimos: a privacidade é garantida por arquitetura e o custo acompanha o consumo real. É esse posicionamento que a Secção 2.3 detalha.

[1] A integração é viável, mas carece de um comportamento nativo ou out-of-the-box no ecossistema de pull requests do Azure DevOps.

[2] CodeRabbit oferece self-hosting exclusivamente no plano Enterprise (mínimo 500 licenças).

[3] "Gratuito" refere-se ao modelo de custo da PoC em ambiente de desenvolvimento; em produção, aplicam-se custos de consumo de tokens no Azure AI Foundry (ver Secção 4.3.1).

2.2.3 Investigação Académica em Code Review Automatizado

A revisão de literatura foi feita nas bases Google Scholar, Semantic Scholar e ACM Digital Library, com os termos automated code review, LLM code review, pull request analysis e multi-agent software engineering, cobrindo 2013–2026. A seleção privilegiou estudos empíricos com dados quantitativos em contexto real. O filtro prático foi simples: só entraram trabalhos que toquem pelo menos uma de quatro dimensões: integração com plataformas DevOps, arquiteturas multiagente, gestão de contexto em LLMs, ou avaliação experimental de code review automático. A Tabela 3 resume os mais relevantes, com a limitação central de cada um e a forma como este projeto a aborda.

Tabela 3 - Análise comparativa de trabalhos relacionados e posicionamento da solução (Elaboração própria)

Autor(es)	Métodos	Limitação identificada	Como este projeto supera
Cihan et al. [14]	GPT-4o/Gemini, 492 blocos	68,5% acurácia; agente único monolítico	Arquitetura multiagente com especialização de input e Model Routing
Cihan et al. [15]	LLM em 238 programadores	Aumento do tempo de fecho de PRs (falsos positivos)	Motor determinístico que remove decisão probabilística
Goldman et al. [16]	Análise de comentários aceites	Programadores ignoram sugestões verbosas	Output pedagógico com CoT + código alternativo concreto
Içöz [17]	LLMs com raciocínio simbólico para code review	Foca-se no raciocínio simbólico genérico, carecendo de mecanismos nativos e seguros para integração contínua (CI/CD) em ambientes corporativos reais.	A proposta desenha-se de raiz sobre a infraestrutura da DevScope, integrando-se nativamente e de forma segura nas pipelines do Azure DevOps.
Haider & Zimmermann [18]	Revisão de código gerado por IA	Lacunas de segurança em código agentic	Deteção sistemática dessas lacunas como caso de uso futuro

Autor(es)	Métodos	Limitação identificada	Como este projeto supera
Hong et al. [19]	Framework multiagente focado na geração de software do zero (SDLC)	Carece de auditoria assíncrona de diffs em CI/CD	Especialização exclusivamente em auditoria (código e requisitos) com integração nativa via Webhooks, sem gerar ou reescrever código autonomamente
Tufano et al. [20]	Fine-tuning de CodeBERT para code review automatizado	Modelo fine-tuned estático, sem adaptação dinâmica a diferentes tipos de PR nem validação de requisitos funcionais	Roteamento dinâmico de modelos (Model Routing) e validação bidimensional técnica + funcional
Lu et al. [21]	CodeBERT — pré-treino para linguagem de programação	Avaliação em tarefas isoladas (clone detection, search), sem integração em pipelines CI/CD	Integração nativa Azure DevOps via Webhooks com execução governada no perímetro organizacional
Pornprasit & Tantithamthavorn [22]	JIT-Fine — revisão just-in-time ao nível do commit	Opera ao nível do commit, não do PR; não valida conformidade com critérios de aceitação	Auditoria ao nível do Pull Request com cruzamento explícito contra Work Items e Acceptance Criteria
Li et al. [23]	Automating Code Review Activities por pré-treino em larga escala	Foco em deteção de defeitos técnicos; ausência de validação de requisitos funcionais	Agente de Requirements Review dedicado com motor AC-Aware Context Pruning

Da Tabela 3 saem dois padrões claros. A maioria dos sistemas avaliados usa um único agente para tudo, o paradigma monolítico que este projeto abandona. E não existe, na literatura consultada,

nenhuma integração nativa no Azure DevOps com execução controlada de modelos. Mais relevante ainda: nenhum dos trabalhos mede o trade-off custo-precisão em arquiteturas multiagente heterogéneas, nem quantifica o impacto da compressão semântica na taxa de deteção. É aí que este projeto entra, e o Capítulo 4 apresenta a avaliação comparativa.

Ficou também em aberto uma questão metodológica: como se mede o desempenho de um sistema deste tipo? Em produção, usa-se a Human Acceptance Rate (HAR) [14], [16]. Esta métrica mede a proporção de sugestões aceites pelos programadores. É intuitiva, mas inaplicável numa PoC que não pode expor código real. A alternativa, usada nos estudos empíricos controlados, é injetar defeitos conhecidos e medir Recall e Precisão. É este caminho que o projeto segue, sendo o único que permite validar as hipóteses do Capítulo 1 com rigor matemático.

2.3 Large Language Models (LLMs) na Engenharia de Software

Os LLMs fizeram um percurso rápido da academia para a indústria. A capacidade de compreender linguagem natural estendeu-se ao código de forma quase orgânica, e o impacto vai muito além do autocompleção.

2.3.1 Capacidades e Raciocínio (Chain-of-Thought)

Ao contrário da análise estática, que opera sobre regras fixas e não percebe contexto, os LLMs lêem intenção. Conseguem identificar bugs que dependem de semântica, sugerir refatorizações com justificação, e gerar documentação com um nível de contextualização que as ferramentas anteriores não tinham [14], [16].

O que separa os modelos mais capazes não é só a compreensão, mas também a capacidade de raciocinar. A técnica Chain-of-Thought (CoT), estudada por Wei et al. [3], força o modelo a explicitar passos intermédios antes de chegar a uma conclusão, em vez de saltar diretamente para o veredicto. No code review, o efeito é concreto: o comentário gerado deixa de ser uma flag vaga e passa a ter justificação e evidência. Goldman et al. [16] mostram que isso aumenta a probabilidade de o programador resolver o finding. Neste projeto, o CoT serve esse mesmo propósito, maximizar a HAR substituindo afirmações por raciocínio verificável.

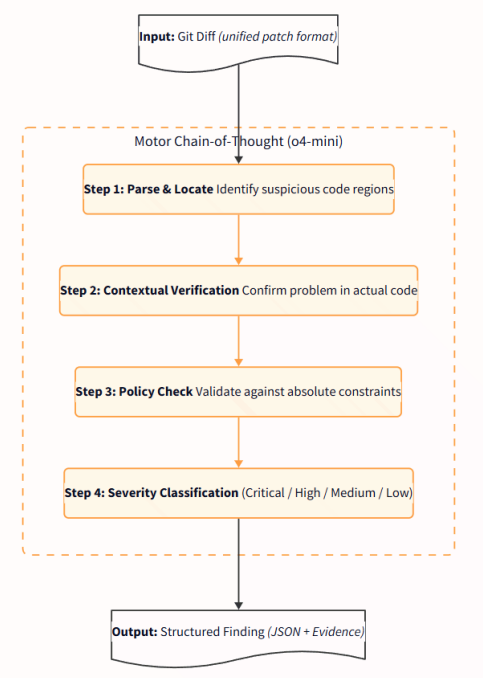


Figura 3 - Fluxo de raciocínio do agente de IA baseado em Chain-of-Thought. (Adaptado de Wei et al.[3])

A limitação central dos LLMs é a inconsistência. O mesmo código pode receber veredictos diferentes em execuções distintas [24]. Num pipeline de CI/CD, isso é inaceitável. Não se pode bloquear um PR com base numa decisão que muda consoante o humor do modelo. A solução adotada é separar papéis: o LLM extrai dados estruturados, o Python decide. As regras de domínio determinísticas (baseadas em DDD [25]) aplicam-se sobre o output probabilístico do modelo, garantindo que a decisão final é auditável.

2.3.2 Modelos Open-Source Relevantes

O ecossistema open-source cresceu muito nos últimos anos, com modelos que variam enormemente em capacidade e custo. Para este projeto, a distinção relevante é entre SLMs (menos de 10B parâmetros, adequados a tarefas simples) e LLMs de larga escala (70B+), capazes de raciocínio semântico em contextos extensos. Com o paradigma MaaS, ambos os segmentos ficaram acessíveis via API sem gestão de infraestrutura. O que muda é a lógica de seleção técnica, não a barreira de acesso.

O estado atual resulta de três gerações sobrepostas. A primeira estabeleceu os SLMs como viáveis para inferência local. A segunda trouxe modelos densos de grande escala, o Llama-3.3-70B e o Qwen2.5-72B são os exemplos mais relevantes, com capacidade analítica substancialmente superior [26], [27]. A terceira, ainda em curso, aposta em arquiteturas MoE e Reasoning Models: o DeepSeek-V3.2 e o o4-mini processam contextos longos com custos e latências que modelos densos equivalentes não conseguem igualar [28], [29]. A Tabela 4 compara os mais relevantes para este projeto.

Tabela 4 - Panorama de Modelos de Linguagem para Análise de Código (Elaboração própria, dados recolhidos das fontes [26], [27], [28], [29], [30], [31])

Modelo	Parâmetros	Tipo	Janela Contexto	Custo input /1M tokens	Custo output /1M tokens	HumanEval	Licença
o4-mini[4]	N/D	LLM	128K	~$1,10	~$4.40	~93,0%	OpenAI
DeepSeek-V3.2	671B (37B ativos)	LLM	128K	~$0.58	~$1.68	~91.0%	MIT
Qwen2.5-72B-Instruct	72B	LLM	128K	~$0,40	~$1.20	~90.0%	Apache 2.0
Llama 3.3 70B	70B	LLM	128k	~$0,71	~$0.71	~88,0%	Meta Llama 3
Qwen3-8B	8B	SLM	128k	~$0,15	~$0.60	91,2%	Apache 2.0
Llama 3 8B	8B	SLM	128K	~$0,06	~$0.06	72.6%	Meta Llama 3
Phi-4 Mini	3.8B	SLM	128K	~$0,075	~$0.75	~72.0%	MIT

O Qwen3-8B merece uma nota: 91,2% no HumanEval a custo muito baixo é impressionante. Mas a literatura é clara sobre os modelos na casa dos 8B: degradam-se rapidamente quando o contexto mistura código completo com múltiplos critérios de aceitação. Para a validação bidimensional deste projeto, essa limitação é crítica. A opção foi por arquiteturas acima de 70B parâmetros ou MoE, que mantêm atenção uniforme em contextos longos.

A lógica é esta: Reasoning Models fazem sentido quando o contexto é pequeno e o raciocínio tem de ser profundo (diffs). Arquiteturas MoE fazem sentido quando o volume é grande e o custo

[4] Inclui os tokens de raciocínio interno (thinking tokens), que são faturados à mesma taxa dos tokens de output visíveis.

importa (requisitos). Como essa matriz se comporta na prática, e como resiste à volatilidade dos catálogos de infraestrutura, está documentado no Capítulo 4

2.4 Síntese e Posicionamento da Proposta

O estado da arte divide-se em dois campos que não se tocam: as soluções comerciais resolvem o problema técnico, mas comprometem a privacidade. As propostas académicas têm rigor, mas não chegam a integrar-se em pipelines reais. O ScopeReview AI situa-se nessa lacuna:

- **Soberania:** Execução isolada no Azure AI Foundry (vs. Cloud pública genérica).
- **Arquitetura:** Multiagente com Model Routing (vs. Monolítica).
- **Validação:** Bidimensional - Técnica + Requisitos (vs. Apenas Sintática).
- **Governança:** Determinística em Python (vs. Probabilística do LLM).

O ScopeReview AI não é uma ferramenta existente reconfigurada. O model routing especializado (Secção 2.3.2) combinado com os mecanismos de controlo de atenção em contextos longos (Secção 3.3.4) resulta num sistema construído de raiz para as restrições reais do Azure DevOps, onde a capacidade analítica dos LLMs é aproveitada sem abdicar da soberania sobre o código.

3 Análise e Desenho da Solução

Este capítulo descreve o domínio do problema, os requisitos identificados para a solução e a arquitetura proposta. A abordagem seguida parte do geral para o particular, começando pela identificação dos conceitos de domínio e culminando na especificação técnica dos componentes.

3.1 Domínio do Problema

Três sistemas de natureza distinta convergem neste projeto: o Azure DevOps, que gere o ciclo de vida do código; o motor de orquestração desenvolvido, responsável pela análise automatizada; e os serviços de inferência do Azure AI Foundry, que providenciam o raciocínio semântico. A complexidade emerge das dependências entre estes componentes, particularmente da exigência de que o código permaneça no perímetro controlado enquanto o feedback é publicado em tempo útil, sem bloquear o fluxo dos programadores.

Historicamente, as expetativas do *Code Review* manual focam-se tanto na deteção de defeitos como na partilha de conhecimento e conformidade arquitetural [2]. No entanto, ao transportar este paradigma para a automação assistida por IA num contexto corporativo, o domínio expande-se para

contemplar uma dupla dimensão: a validação técnica da qualidade do código (*Code Review*) e a validação funcional da conformidade com o negócio (*Requirement Review*) [32].

Esta bidimensionalidade constitui a principal contribuição original do presente projeto face ao estado da arte revisto na Secção 2.4: enquanto a literatura existente trata a análise técnica de código e a validação de conformidade funcional como problemas independentes [1], o presente sistema trata-os como dimensões complementares de um único processo de auditoria, partilhando contexto através do Knowledge Ledger definido na Secção 3.1.1.

Desta forma, o problema a resolver não se limita à identificação de anomalias sintáticas ou vulnerabilidades suportadas por pré-treino de LLMs, mas exige também a capacidade de cruzar as alterações propostas com os requisitos de negócio originais. O sistema atua assim como um validador bidimensional [1], [33].

A Figura 4 apresenta uma visão geral do domínio, ilustrando a interação entre o programador, o Azure DevOps, o sistema de orquestração desenvolvido e os modelos de IA no Azure AI Foundry. Uma representação formal do mesmo contexto em notação C4 consta na Figura D.4 (Anexo D)

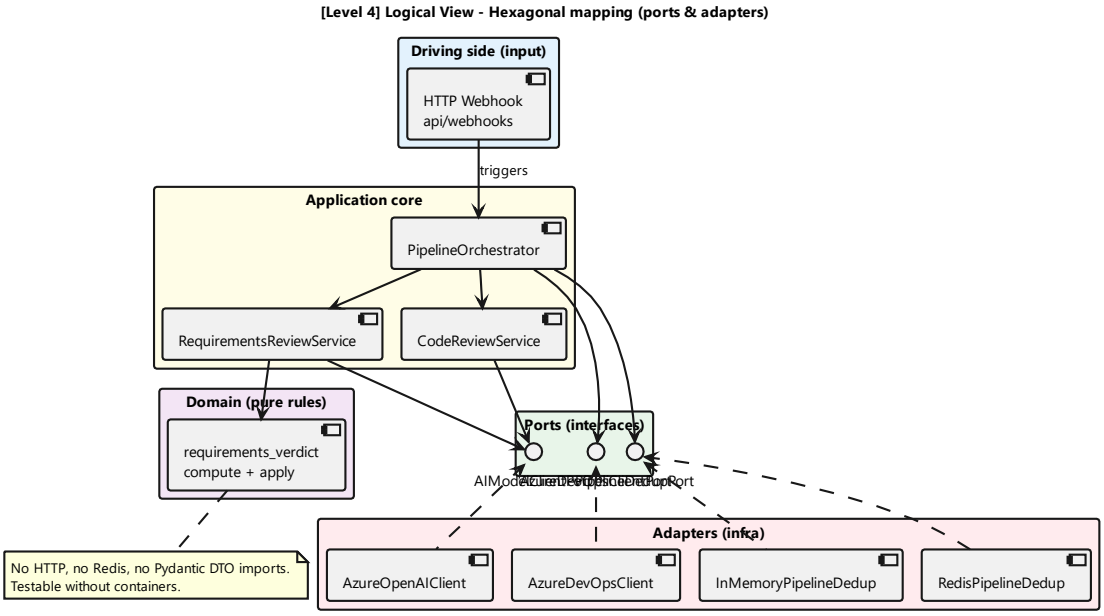


Figura 4 - Diagrama de Visão Geral (Elaboração própria)

O mapeamento topológico exposto no diagrama evidencia a separação estrita e hermética entre o perímetro externo de infraestrutura (Azure DevOps e as instâncias de inferência do Azure AI Foundry) e o motor de orquestração interna. Esta delimitação garante que o código-fonte proprietário nunca seja persistido ou exposto a entidades terceiras não autorizadas, constituindo uma salvaguarda estrutural direta do requisito de soberania de dados (R1).

3.1.1 Entidades e Conceitos Centrais

A modelação de domínio seguiu a abordagem proposta por Evans [25]. A partir do evento de criação de um Pull Request, identificaram-se os agregados, as entidades dependentes e os serviços de domínio. Esta metodologia assegura que o modelo espelha a lógica de negócio, não a estrutura técnica da implementação.

Neste nível de desenho, a arquitetura estabelece a Linguagem Ubíqua do sistema através de um modelo concetual puro. Este modelo é desenhado para ser totalmente agnóstico em relação à infraestrutura subjacente, blindando o núcleo de negócio contra constrangimentos ou detalhes de plataformas externas (como fornecedores de repositórios ou motores de Inteligência Artificial). Embora na concretização arquitetural estes conceitos puros venham a necessitar de ser transportados e representados através de objetos de transferência de dados (Data Transfer Objects), moldados aos contratos das plataformas externas, a definição funcional e semântica das entidades no Domínio permanece isolada e imutável.

Com o intuito de mapear estes conceitos estruturais, a Tabela 5 sistematiza as entidades centrais do Core Domain e as respetivas responsabilidades operacionais.

Tabela 5 - Identificação e descrição das entidades centrais do Domain Core (Elaboração própria)

Entidade	Tipo DDD	Descrição
AuditRequest	Aggregate Root	Entidade central que representa o pedido de auditoria a uma submissão. Atua como o agregador do ciclo de vida da análise e das regras de governança.
CodePatch	Value Object	Representação imutável e isolada das alterações de código (o delta semântico), abstraída do formato específico do sistema de controlo de versões externo.
AuditTrigger	Value Object	O evento de domínio que inicia o ciclo de análise, abstraindo a origem técnica (ex: notificação reativa via Webhook).
BusinessRequirement	Entity	A especificação de negócio associada ao patch, contendo as regras e objetivos que o código deve satisfazer.

Entidade	Tipo DDD	Descrição
Acceptance Criteria	Value Object	Condições formais extraídas dos requisitos, utilizadas pelo motor de validação para aferir a conformidade funcional.
AI Agent (Orquestrador)	Domain Service	Serviço de domínio que coordena o ciclo de vida da auditoria, gerindo o roteamento de agentes e a propagação de contexto.
KnowledgeLedger	Domain Service	Registo estruturado que atua como "verdade única" entre agentes, eliminando a redundância cognitiva ao partilhar factos pré-validados.
AuditReport	Entity	O artefacto de saída do domínio, contendo os veredictos determinísticos e o feedback estruturado, pronto para ser mapeado para o sistema externo.
ContextPruner	Domain Service	Motor lógico que mitiga o Context Window Bloat, filtrando o código para reter a estrutura e o léxico essenciais à validação.
TriageGate	Domain Service	Componente de triagem que descarta liminarmente ativos não-analisáveis para prevenir a degradação de atenção dos modelos.

3.1.2 Modelo de Domínio e Relações entre Entidades

A Figura 5 apresenta o Modelo de Domínio simplificado do sistema, ilustrando as relações estruturais entre as entidades descritas. O que o diagrama torna imediatamente visível é a separação lógica entre as entidades do repositório e o motor de avaliação. O diagrama de classes granular do Domain Core, com a representação completa dos Aggregate Roots e Value Objects, consta na Figura D.1 (Anexo D).

Observa-se que o AuditRequest atua como o agregador central, estando associado aos BusinessRequirements, que por sua vez definem as AcceptanceConditions do negócio. O Agente de IA (Orquestrador) funciona como o serviço de domínio que interceta e processa esta informação,

iniciada pelo AuditTrigger. Ao analisar o CodePatch e validar os requisitos em simultâneo, o Agente emite um AuditReport (o veredito final determinístico) e gera o feedback estruturado.

A desacoplagem entre Azure DevOps e o motor de IA, materializada nos adaptadores da Secção 3.3.2, isola a lógica de orquestração de alterações externas. Esta propriedade foi validada na prática: a migração sucessiva entre três modelos de IA (Phi-4-mini, Llama 3.3 70B e a configuração multiagente final) não exigiu alterações à lógica de domínio, confirmando a resiliência do isolamento adotado. Esta modelação reflete os princípios de sistemas orientados a eventos e de Domain-Driven Design [25].

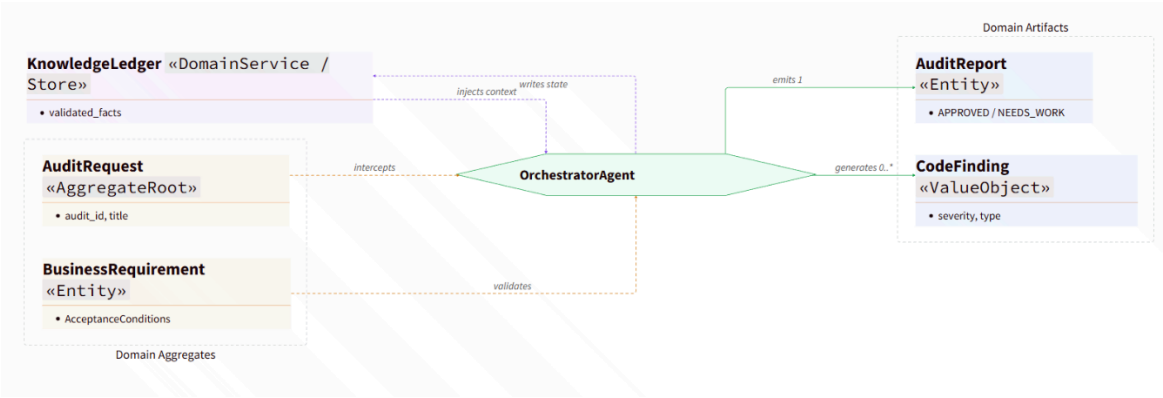


Figura 5 - Diagrama do Modelo de Domínio (Domain Core) do ScopeReview AI (Elaboração própria)

O Domain Core foi desenhado para ser indiferente ao que acontece fora dele. Os motores de triagem, os registos do ledger e as regras de normalização estão isolados dos contratos de integração das APIs externas, o que significa que flutuações operacionais ou alterações nos endpoints de terceiros não chegam a tocar a lógica de governança central. Na prática, isso traduz-se em resiliência: o sistema não quebra quando a infraestrutura muda.

Face à natureza stateless do pipeline e aos requisitos de latência (RNF04), a solução não recorre a um modelo de dados relacional persistente. O estado transacional é gerido em memória através do Knowledge Ledger e de estruturas imutáveis em Python, enquanto a coordenação distribuída e a deduplicação de eventos são asseguradas por instâncias efémeras no Redis [34]. Esta decisão arquitetural elimina a sobrecarga de I/O de base de dados, alinhando-se diretamente com os constrangimentos de eficiência e soberania de dados (RNF01, RNF02).

3.2 Requisitos Funcionais e Não Funcionais

O levantamento de requisitos teve três fontes: a análise das limitações das ferramentas existentes (Secção 2.2.2), os objetivos definidos para o estágio (Secção 1.2) e as necessidades operacionais identificadas em conjunto com o supervisor externo na DevScope. A priorização seguiu o modelo MoSCoW, os requisitos críticos são os cuja ausência tornaria impossível atingir os objetivos H1 e H2 da Secção 1.4.

Os requisitos dividem-se em dois grupos: funcionais (Tabela 6), que definem o comportamento esperado na validação de código, e não funcionais (Tabela 7), que fixam as restrições operacionais. Os dois conjuntos não são independentes, existe uma rastreabilidade direta entre eles. Por exemplo, RF03 (validação de requisitos funcionais) é diretamente constrangido por RNF02 (eficiência de custos), o que motivou a adoção de técnicas como a poda de contexto e os limites de geração detalhados nas secções seguintes [35].

Tabela 6 - Requisitos Funcionais (Elaboração própria)

ID	Descrição	Prioridade
RF01	Deteção de Bugs e Qualidade: O sistema deve analisar automaticamente o código alterado para identificar erros lógicos, redundâncias e violações de boas práticas de programação.	Crítica
RF02	Identificação de Vulnerabilidades de Segurança: O sistema deve detetar padrões de segurança inseguros e a presença de segredos (passwords, chaves API) expostos no código.	Alta
RF03	Validação de Requisitos Funcionais: O sistema deve verificar se as alterações no código cumprem os critérios de aceitação definidos nos Work Items vinculados ao Pull Request.	Crítica
RF04	Publicação de Feedback Contextual: As sugestões de melhoria devem ser publicadas diretamente como comentários nas linhas de código correspondentes no Azure DevOps.	Crítica
RF05	Triagem Automática (Veredito): O sistema deve emitir um veredito global (Aprovado ou Necessita Atenção) para acelerar a decisão do revisor humano.	Alta
RF06	Análise Estática Determinística: O sistema deve executar uma camada de análise estática baseada em regras antes da invocação de LLMs, detetando padrões conhecidos de forma determinística (imports não utilizados, credenciais expostas, funções vulneráveis em C/C++, tokens JWT expostos, configurações CORS permissivas). Esta camada serve dois propósitos: reduzir o consumo de tokens do LLM e garantir cobertura de padrões que os modelos de linguagem tendem a tratar de forma inconsistente.	Alta

O principal compromisso de design da solução situa-se na tensão entre RNF02 (eficiência de custos) e RNF04 (latência máxima de 90s), dois requisitos que puxam em direções opostas. É essa tensão que justifica a transição para uma orquestração multiagente, detalhada no capítulo seguinte. Os casos de uso que enquadram funcionalmente o sistema estão representados na Figura 6.

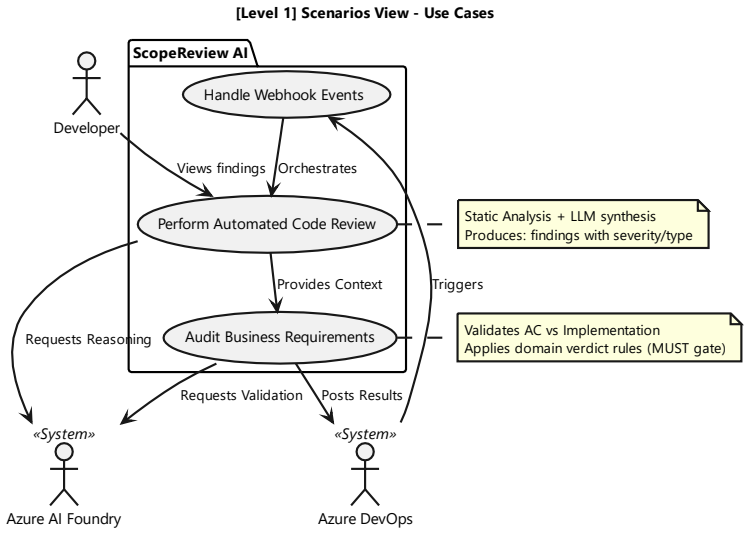


Figura 6 - Diagrama Use Case (Elaboração própria)

A Figura 6 ilustra os casos de uso do sistema, envolvendo três atores: o Programador (submete o PR), o Azure DevOps (despoleta o webhook e exibe o feedback) e o Sistema ScopeReview AI (orquestra a extração, análise e publicação). O limite do sistema garante que a aprovação final (merge) permanece sob alçada humana (Human-in-the-Loop).

A Tabela 7 delimita as restrições de performance, segurança e viabilidade económica que governam a solução em ambiente industrial. As métricas aqui definidas condicionaram diretamente as decisões de desenho arquitetural apresentadas no capítulo seguinte.

Tabela 7 - Requisitos Não Funcionais (Elaboração própria)

ID	Descrição	Prioridade
RNF01	Soberania e Privacidade de Dados: Todo o processamento de código deve ocorrer exclusivamente dentro do perímetro de segurança da Azure da organização, garantindo que o código não é usado para treino de modelos externos.	Crítica

ID	Descrição	Prioridade
RNF02	Eficiência de Custos: O sistema deve otimizar o uso de modelos de linguagem (LLMs) através de técnicas de deduplicação (evitar re-análise do mesmo código), limites de tokens, context Pruning dinâmico, constrangimento de geração através de schemas JSON assimétricos e propagação de contexto inter-agentes.	Crítica
RNF03	Proteção de Recursos (Rate Limiting): A solução deve incluir mecanismos de controlo de tráfego para evitar o consumo excessivo da quota da API de IA e proteger o serviço de sobrecarga.	Média
RNF04	Baixa Latência de Resposta: O ciclo completo de auditoria (desde o evento de trigger até à emissão do AuditReport) deve concluir-se num tempo médio inferior a 90 segundos.	Alta

O limiar de 90 segundos para o RNF04 não foi escolhido arbitrariamente. Deriva de dois constrangimentos concretos: o timeout síncrono de 30 segundos impostos pelos Service Hooks do Azure DevOps (contornado via assincronia arquitetural) e a necessidade de garantir que o feedback automático chega antes de o revisor humano iniciar a sua análise. O problema de engenharia é simples de enunciar, mas difícil de resolver: modelos com maior capacidade de raciocínio produzem análises mais precisas, mas demoram mais. É precisamente essa tensão entre RNF02 e RNF04 que a arquitetura multiagente com Model Routing procura resolver.

3.3 Desenho

3.3.1 Evolução do Âmbito e Transição Arquitetural

O Azure AI Foundry (anteriormente Azure AI Studio) é a plataforma unificada da Microsoft para desenvolvimento e gestão de soluções de IA generativa em contexto empresarial. Para efeitos deste projeto, o aspeto mais relevante é o modelo de deployment: a plataforma oferece duas opções. O deployment dedicado reserva capacidade de inferência exclusiva para a organização, com latência previsível e custo fixo, adequado para volumes elevados e consistentes. O deployment serverless fatura apenas o consumo efetivo de tokens, sem infraestrutura reservada, e é o que se aplica aqui: uma prova de conceito com utilização variável adapta-se melhor ao modelo pay-per-token.

Do ponto de vista da privacidade, a escolha do Foundry não é neutra. A documentação da Microsoft garante que os dados processados em ambientes empresariais são isolados por organização e não alimentam o treino de modelos globais, o código-fonte submetido fica dentro do perímetro de segurança da Microsoft. A isso juntam-se políticas de Zero-Trust e mecanismos nativos de Content Safety [35], que cobrem parte dos vetores de risco identificados para implementações de LLMs em contexto corporativo [33].

A transição de uma abordagem de agente monolítico para uma arquitetura multiagente adaptativa foi motivada pela identificação precoce de três riscos operacionais e metodológicos críticos:

- **Risco Estrutural:** A deteção de regressões (orientada ao diff) e a validação de conformidade funcional (orientada ao estado atual) exigem inputs, estratégias de raciocínio e critérios de sucesso distintos. Forçar um único agente a executar ambas degrada as duas dimensões (validado empiricamente em 4.2.2).
- **Risco Cognitivo:** A saturação da janela de contexto com código-fonte acessório induz o fenómeno Lost in the Middle, degradando severamente a atenção e a acuidade do modelo na interpretação de critérios de aceitação funcionais (validado empiricamente em 4.2.2)[36].
- **Risco Económico:** A execução indiscriminada de inferências analíticas extensas em modelos de raciocínio profundo gera um consumo inflacionado de tokens, tornando o custo operacional por submissão insustentável à escala industrial (validado empiricamente em 4.3.1).

A conjugação destes três riscos motivou a transição para uma arquitetura multiagente com Model Routing dinâmico, onde cada agente recebe exclusivamente o input para o qual o seu modelo foi selecionado. Esta decisão é suportada pela literatura sobre Model Routing [37] e Sistemas Multiagente [33], e a sua validação empírica consta da Secção 4.2.

3.3.2 Arquitetura da Solução (Ports & Adapters)

A escolha do padrão Ports & Adapters (Arquitetura Hexagonal) [38] respondeu a dois problemas concretos identificados na análise de requisitos, não a uma preferência de design. O primeiro: o catálogo de modelos do Azure AI Foundry muda. Novos modelos entram, outros saem ou são descontinuados. O segundo: testar os componentes de domínio de forma isolada, sem depender de serviços externos pagos, era um requisito de desenvolvimento desde o início.

Com o núcleo de domínio isolado dos detalhes de infraestrutura, substituir um modelo de IA ou adaptar o sistema a alterações na REST API do Azure DevOps resume-se a implementar novos adaptadores, uma vez que a lógica central de avaliação não é tocada. Isto foi validado na prática durante a fase F3, quando a migração forçada entre modelos não exigiu qualquer alteração ao orquestrador.

A integração com o Azure DevOps é viabilizada pela combinação de dois mecanismos complementares que permitem, respetivamente, a notificação de eventos e a interação programática com os recursos da plataforma [10].

Os Webhooks do Azure DevOps constituem o ponto de entrada reativo da solução. Através da configuração de um *Service Hook*, a plataforma notifica automaticamente um serviço externo sempre que ocorre um evento relevante, neste caso, a criação ou atualização de um Pull Request, correspondente ao evento *git.pullrequest.created* ou *git.pullrequest.updated*. Esta notificação é enviada sob a forma de um payload JSON via HTTP POST para o endpoint registado no agente. Um aspeto crítico desta integração é o contrato de tempo de resposta imposto pelo ADO: o serviço externo deve responder com um código HTTP de sucesso num prazo de aproximadamente 30 segundos, findo o qual a plataforma considera o Webhook falhado e aciona mecanismos de retry automático. Esta restrição tem implicações diretas no design arquitetural do agente, que não pode processar a inferência de IA de forma síncrona dentro deste prazo.

A REST API do Azure DevOps oferece um conjunto abrangente de endpoints que permitem ao agente interagir programaticamente com todos os recursos da plataforma [10]. Através desta API, é possível extrair as iterações de um Pull Request e os metadados dos ficheiros alterados, descarregar o conteúdo real dos ficheiros através do endpoint */items* com especificação do commit SHA via *versionDescriptor*, e publicar comentários estruturados diretamente nas threads de revisão, simulando a intervenção de um revisor humano. A autenticação é realizada via Personal Access Token (PAT), codificado em Base64 e transmitido no header *Authorization*.

Os blocos lógicos principais são:

- **Event Gateway & Deduplication:** Ponto de entrada assíncrono. Responde imediatamente às notificações para evitar timeouts da plataforma e utiliza um mecanismo de bloqueio distribuído para garantir que o mesmo evento não é processado concorrentemente.
- **Context Extractor (Adapter):** Módulo responsável por abstrair a comunicação com o provedor de repositórios (neste caso, o Azure DevOps) para extrair os diffs, os ficheiros head completos e os dados dos Work Items.
- **Orchestrator Engine:** O motor central que gere o ciclo de vida. É responsável pelo Model Routing dinâmico e pela propagação de contexto (Context Propagation) entre os diferentes agentes [33].
- **Code Review Agent:** Motor focado em bugs e segurança, desenhado para atuar sobre blocos reduzidos de código (o Patch), utilizando Chain-of-Thought [3].

- **Requirement Review Agent:** Motor focado em mapear código extenso contra as regras de negócio. Utiliza modelos otimizados para leitura de grandes volumes de texto (como a arquitetura Mixture-of-Experts) [39].
- **Verdict Normalization Engine (Domain Policy):** Inspirado nos princípios de Domain-Driven Design (DDD), especificamente no conceito de Domain Service, esta é uma camada de governação local. Funciona como um "disjuntor" que interceta as saídas JSON da IA e aplica regras determinísticas da organização, mitigando ativamente o risco de alucinações de aprovação [25].
- **Semantic Pruning Engine & Triage Gate (Domain Service):** Um serviço de domínio crucial que interceta os ficheiros brutos extraídos do repositório. Aplica uma técnica desenvolvida de AC-Aware Pruning, cruzando o vocabulário dos Work Items com a estrutura do código para eliminar funções utilitárias e boilerplate, reduzindo o volume do payload enviado aos agentes sem comprometer a semântica de negócio [35].

A rastreabilidade entre requisitos e componentes arquiteturais é directa: RF01 e RF02 são servidos pelo Code Review Agent; RF03 pelo Requirement Review Agent; RF04 pelo Publication Adapter; RF05 e RF06 pelo Verdict Normalization Engine e pelo Triage Gate, respectivamente.

Esta separação de responsabilidades, ilustrada na Figura 7, garante que cada requisito não funcional de segurança e latência é isolado em módulos testáveis independentemente da evolução do catálogo de modelos no Azure AI Foundry. O Context Pruner, definido no modelo de domínio, foi materializado através de um Semantic Pruning Engine customizado. Este motor não faz uma poda aleatória, mas sim uma poda 'AC-Aware' (consciente dos critérios de aceitação)

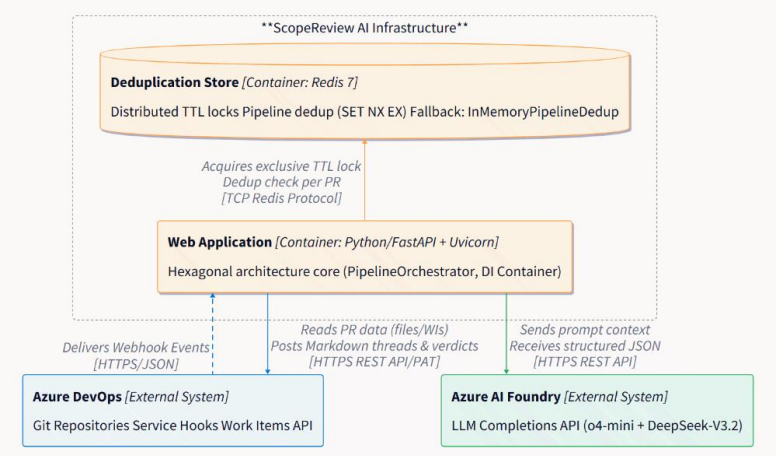


Figura 7 - Arquitetura Ports & Adapters, ilustrando o isolamento do Core de Domínio (orquestrador e agentes) face às integrações externas (Elaboração própria)

A vista de contentores C4 do sistema, os componentes internos do serviço FastAPI e o mapeamento lógico hexagonal completo encontram-se detalhados nas Figuras D.2, D.5 e D.12 (Anexo D), respetivamente.

A estratégia de Model Routing adotada baseia-se num design rationale de especialização tecnológica para otimizar o desempenho e o custo [37]. O modelo o4-mini é mobilizado para a análise de diffs curtos, tirando partido da sua natureza como Reasoning Model com suporte nativo a cadeias de raciocínio (Chain-of-Thought) para um escrutínio lógico profundo das alterações. Em contrapartida, para a revisão de requisitos que exige o processamento de contextos extensos, selecionou-se o DeepSeek-V3.2. A sua arquitetura Mixture-of-Experts (MoE) ativa apenas aproximadamente 37 mil milhões de parâmetros por inferência, garantindo elevada eficiência económica e precisão semântica na validação de grandes volumes de documentação e código completo.

3.3.3 Justificação da Arquitetura Multiagente

Isolar a lógica de domínio da infraestrutura resolve um problema de manutenibilidade. Mas a eficácia analítica do sistema depende de uma escolha diferente: o paradigma de agentes. A investigação em IA deslocou-se progressivamente de modelos isolados para o conceito de Agente Autónomo, uma entidade capaz de planear tarefas, interagir com o ambiente (compiladores, sistemas de controlo de versão) e corrigir os seus próprios erros com base no feedback recebido [1]. A distinção em relação ao uso pontual de um LLM é relevante: um modelo isolado responde a um prompt e termina; um agente mantém um ciclo contínuo de perceção, raciocínio e ação.

Em engenharia de software, este paradigma evoluiu para Sistemas Multiagente (SMA), onde "especialistas" virtuais distintos, cada um orientado a uma dimensão específica da análise, como segurança ou performance, colaboram numa revisão mais abrangente [31]. A literatura documenta que decompor responsabilidades entre agentes especializados, com mecanismos de comunicação estruturada, produz resultados consistentemente superiores às abordagens monolíticas assentes num único LLM generalista.

A propagação de contexto entre agentes evita conclusões contraditórias: ao injetar os achados do primeiro agente como input estruturado do segundo, estabelece-se uma cadeia de raciocínio distribuída. Esta abordagem garante coerência analítica, eliminando o risco de avaliações independentes produzirem veredictos opostos sobre o mesmo Pull Request [31]. A literatura identifica a ausência deste mecanismo como causa frequente de falhas de coordenação em sistemas multiagente.

Contudo, a independência dos agentes pode introduzir ineficiências quando existe sobreposição de domínios analíticos, resultando no fenómeno de redundância cognitiva ou double-thinking [40]. Neste cenário, múltiplos agentes podem reanalisar e reportar de forma duplicada os mesmos problemas não-funcionais (NFRs), gerando desperdício de tokens e relatórios contraditórios. Para mitigar esta

lacuna, a investigação aponta para a necessidade de memórias partilhadas estruturadas, ou Knowledge Ledgers, que permitam aos agentes registar e consultar factos já verificados, eliminando a duplicação de raciocínio e garantindo a coerência absoluta do veredito final.

A autonomia dos agentes tem um limite deliberado, enquadrado pelo conceito de Human-in-the-Loop [41]. O sistema opera de forma independente nas fases de extração, triagem e análise, mas não tem autoridade para editar código diretamente nem para aprovar o pipeline. A decisão final pertence sempre ao engenheiro humano. Não se trata de uma limitação técnica: é uma escolha de design, e intencional.

O estudo empírico da MSR'26 [18] acrescenta uma camada de urgência a este argumento: o código gerado por sistemas de IA introduz omissões lógicas subtis e padrões de vulnerabilidade mais difíceis de detetar do que os erros típicos de programadores humanos. Para o ScopeReview AI, isso é diretamente relevante, pois auditar código gerado por IA com um agente monolítico genérico seria insuficiente. A opção por modelos de raciocínio profundo (slow thinking) numa arquitetura multiagente responde precisamente a essa exigência.

A materialização técnica deste pipeline, incluindo a ordem de invocação dos agentes e a partilha de estado, é detalhada na secção seguinte.

3.3.4 Fluxo de Execução Orquestrado

Trabalhar com LLMs em contexto de engenharia de software tem um problema prático que a teoria muitas vezes subestima: a gestão da janela de contexto. Modelos recentes suportam 128K tokens ou mais, o que pode criar a ilusão de que basta enviar tudo. Na prática, a submissão exaustiva de ficheiros completos e documentação de requisitos gera o que a literatura chama de Context Window Bloat: os custos de inferência disparam, a latência aumenta, e o modelo começa a perder precisão analítica por saturação semântica.

A Figura 8 demonstra que os LLMs apresentam uma quebra acentuada de atenção e precisão quando a informação crucial se encontra diluída no meio de contextos muito extensos [36]. No domínio da revisão de requisitos funcionais, isto traduz-se num risco acrescido de Falsos Negativos, onde o modelo ignora critérios de aceitação críticos devido à sobrecarga cognitiva provocada por boilerplate, importações e código utilitário irrelevante.

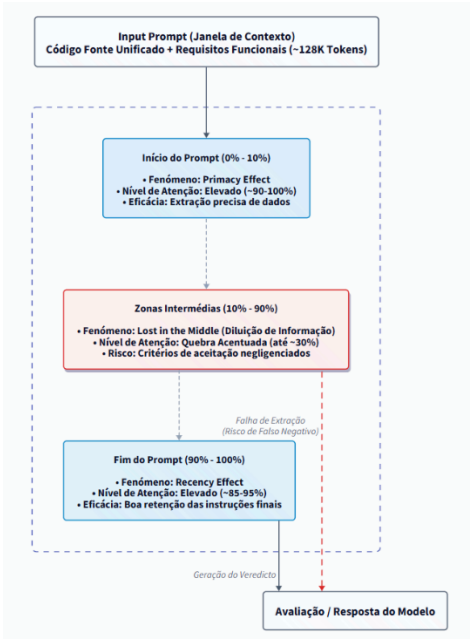


Figura 8 - Fenómeno Lost in the Middle, ilustrando a distribuição de atenção dos LLMs e o risco inerente de negligência de critérios críticos em documentação e contextos de código extensos (Adaptado de Liu et al. [36])

Para mitigar esta degradação, a investigação recente tem explorado técnicas de Context Pruning (poda de contexto) para compressão do input. Abordagens puramente estruturais recorrem a ferramentas de parsing para extrair a Abstract Syntax Tree (AST) do código, mantendo apenas assinaturas de funções e removendo o corpo lógico [23]. Contudo, embora eficazes na redução de tokens, abordagens baseadas exclusivamente em AST resultam frequentemente na perda de semântica de negócio (ex: queries SQL ou literais de domínio essenciais para a validação de requisitos). Em alternativa, estratégias de poda semântica propõem a extração dinâmica de vocabulário de negócio a partir dos requisitos documentados (ex: Acceptance Criteria), permitindo filtrar o código para reter a estrutura "muscular" juntamente com o léxico de domínio.

O input é apenas metade do problema. A geração de texto pelo LLM (Output Tokens) é o estrangulamento mais moroso e oneroso do pipeline. A solução adotada foi impor constrangimentos de geração estruturada via schemas JSON assimétricos: o modelo omite justificações para validações bem-sucedidas e só detalha evidências quando deteta anomalias. O resultado é uma redução significativa de latência sem perder acionabilidade nos findings relevantes.

O fluxo de execução abandona a linearidade em favor de um pipeline orquestrado com partilha de contexto:

- **Despoletar e Extrair:** O *Gateway* deteta a submissão, inicia o processamento assíncrono e delega ao *Context Extractor* a recolha do código e dos *issues* de negócio.

- **Triagem Semântica Inicial (Triage Gate):** Antes de qualquer invocação de IA, o motor de triagem avalia a relevância do payload extraído. Documentação, ativos binários e configurações estáticas são imediatamente descartados, poupando tempo de rede e esforço computacional.
- **Fase 1 (Análise Estática e Code Review):** O Orchestrator submete os ficheiros válidos a uma validação estática inicial. De seguida, o Code Review Agent foca o seu raciocínio profundo exclusivamente sobre o diff puro e não comprimido, garantindo que nenhum contexto técnico imediato é perdido na deteção de vulnerabilidades e falhas lógicas da alteração.
- **Context Propagation:** Os achados validados na Fase 1 são injetados de forma estruturada na base de conhecimento central (Knowledge Ledger). Em paralelo, aciona-se o Semantic Pruning Engine para preparar a fase seguinte: cruzando o vocabulário de negócio com a estrutura do código, este motor comprime apenas os ficheiros de código completo, mitigando o ruído antes da auditoria de negócio [33], [35].
- **Fase 2 (Requirement Review):** O Requirement Review Agent recebe o contexto propagado da Fase 1 e os ficheiros completos já comprimidos pela poda semântica, procedendo à verificação de conformidade transversal. Para maximizar a eficiência, este agente é governado por um padrão de geração JSON de densidade assimétrica: requisitos validados com sucesso produzem apenas estados compactos, enquanto falhas obrigam à injeção rica de evidências e sugestões corretivas, suprimindo o desperdício de tokens na geração do output.
- **Governação e Efetivação:** As avaliações de ambos os agentes passam pelo Verdict Normalization Engine. Após o cálculo determinístico garantir que não existem alucinações de aprovação, o resultado é convertido em Markdown estruturado e submetido diretamente às threads de revisão da plataforma. É neste ponto de fronteira que se materializa o paradigma Human-in-the-Loop (pós-facto) [14]. O sistema orquestrador atua de forma totalmente autónoma na extração e análise, mas não detém privilégios de edição direta no código-fonte nem de aprovação final de integração (merge). Ao depositar os findings na thread, o agente atua como um assistente consultivo, transferindo a autoridade para o revisor humano, que escrutina o relatório e toma a decisão final de bloquear ou aceitar as correções.

O fluxo de atividade completo do pipeline — desde a receção do webhook até à publicação do relatório, passando pelo Triage Gate e pelas duas fases de análise — encontra-se diagramado na Figura D.8 (Anexo D). A sequência de orquestração multiagente detalhada com partilha de contexto via Knowledge Ledger consta na Figura D.3 (Anexo D). A materialização técnica deste fluxo, incluindo a serialização de payloads, tratamento de falhas de rede e implementação assíncrona, é documentada no Capítulo 4 (Secções 4.1.2 e 4.1.6). A sua validação quantitativa consta da Secção 4.3.

A decisão última permanece sempre do lado do revisor humano, que recebe o relatório automático como input consultivo, não como veredicto vinculativo. Esta salvaguarda não é uma limitação da solução: é uma escolha de design deliberada, alinhada com as melhores práticas identificadas na literatura [41] e com os requisitos de governança da DevScope. Desta forma, garante-se que a resposta final entregue ao programador cumpre os critérios de clareza e acuidade definidos.

3.3.5 Seleção e Justificação do Modelo Base

A evolução recente dos LLMs deu origem a três paradigmas arquiteturais complementares, Reasoning Models, Mixture-of-Experts (MoE) e Model Routing, cuja aplicação conjunta otimiza simultaneamente a qualidade do raciocínio e a eficiência operacional (Secção 2.3.2). O sistema desenvolvido materializa estes paradigmas através de uma estratégia de roteamento dinâmico (Figura 9), onde o o4-mini (Reasoning Model) é mobilizado para análise de diffs curtos, tirando partido do slow thinking para escrutínio lógico profundo [29], enquanto o DeepSeek-V3.2 (arquitetura MoE com 671B parâmetros, ~37B ativos) processa contextos extensos de requisitos a custo competitivo [28], [39].

A seleção do modelo base seguiu quatro critérios: disponibilidade via MaaS no Azure AI Foundry (RNF01), escala suficiente para raciocínio semântico (>70B parâmetros), especialização em código (HumanEval >85%) e custo operacional sustentável. Cruzando estes critérios com os dados da Tabela 4, o Qwen2.5-72B-Instruct destaca-se: 72B parâmetros, ~90% no HumanEval, e desempenho competitivo com o Llama-3-405B-Instruct a um quinto dos parâmetros [26], [27].

A escolha de um modelo específico é, nesta arquitetura, secundária. O contributo central de design é a agnosticidade estrutural: o Core Domain interage exclusivamente com abstrações (Ports), o que significa que rotear dinamicamente ou migrar entre famílias de modelos não toca a lógica de negócio. Num mercado onde os catálogos de LLMs mudam ao ritmo que mudam, essa propriedade passa de preferência arquitetural a requisito de sobrevivência. A sua validação empírica consta no Capítulo 4.

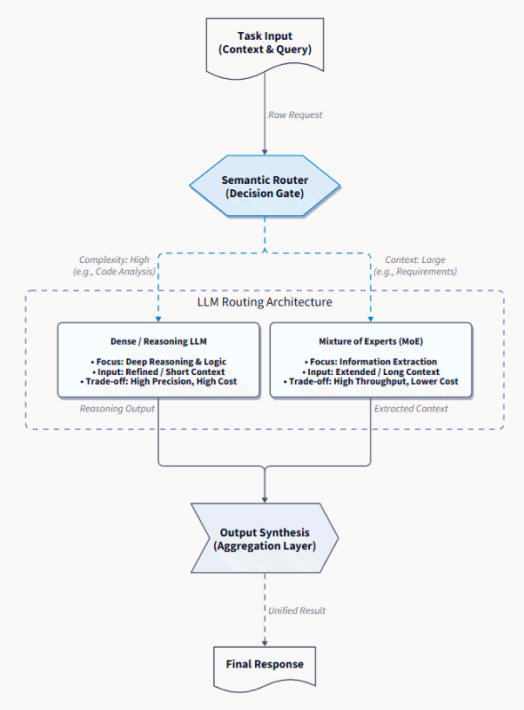


Figura 9 - Princípio conceptual de Orquestração Multiagente baseado em Model Routing (Elaboração própria, inspirado em Moslem & Kelleher [37])

Como ilustra a Figura 9, o roteamento dinâmico atribui a cada tipo de tarefa o modelo com o perfil custo-precisão mais adequado, princípio cuja materialização técnica consta no Capítulo 4.

4 Implementação

Este capítulo acompanha a construção do ScopeReview AI desde o primeiro protótipo até à arquitetura multiagente final. O percurso não foi linear. A solução foi descoberta por tentativa e erro, e cada obstáculo encontrado acabou por ditar uma decisão de design que ficou no sistema. Estão documentadas as tecnologias escolhidas, as opções arquiteturais que resultaram das restrições de infraestrutura e de rede, os detalhes de implementação dos componentes principais e a forma como os Large Language Models são orquestrados. O capítulo fecha com as métricas de desempenho e a avaliação dos resultados.

4.1 Descrição da Implementação

4.1.1 Ambiente e Stack Tecnológico

O desenvolvimento decorreu num ambiente Windows 11 com WSL2 (Ubuntu 22.04), recorrendo à containerização Docker para garantir reprodutibilidade entre os ambientes de desenvolvimento e de

deployment. A linguagem de implementação escolhida foi Python 3.12, pela sua maturidade no ecossistema de integração com APIs de inferência e pela expressividade das suas primitivas assíncronas nativas. O framework web adotado foi o FastAPI com servidor ASGI Uvicorn, cuja relevância arquitetural é detalhada na Secção 4.1.2 [42].

A Tabela 8 resume o stack utilizado nas várias fases do projeto. Cada escolha teve uma razão concreta: o Python 3.12 foi adotado pelas suas primitivas assíncronas nativas, que eram necessárias para o mecanismo de BackgroundTasks detalhado na Secção 4.1.2; o Redis entrou mais tarde, para substituir o lock em memória por uma solução de deduplicação que funciona com múltiplos contentores, algo que um simples dicionário em RAM não consegue garantir (Secção 4.1.7).

Tabela 8 - Stack tecnológico do projeto ScopeReview AI (Elaboração própria)

Componente	Tecnologia / Versão	Função no Sistema
Linguagem	Python 3.12.8	Lógica de orquestração e integração
Framework Web	FastAPI 0.115.6	Receção de webhooks e BackgroundTasks
Servidor ASGI	Uvicorn 0.32.1	Servidor de alta performance para o FastAPI
Containerização	Docker 27.x / Docker Compose 2.31	Isolamento e deployment reprodutível
Deduplicação distribuída	Redis 7.4 (Alpine)	Distributed lock para múltiplos contentores
Plataforma DevOps	Azure DevOps (ADO)	Repositório Git, Pull Requests, Work Items
Plataforma IA	Azure AI Foundry (MaaS)	Inferência de modelos LLM via API
Agente Code Review	o4-mini (OpenAI / Azure)	Raciocínio profundo sobre diffs de código

Agente Req. Review	DeepSeek-V3.2 (MoE)	Análise semântica de contextos extensos
Modelo Inicial	Phi-4-mini-instruct (3.8B)	Validação da infraestrutura a custo mínimo
Modelo Intermediário	Llama-3.3-70B-Instruct	Elevação da qualidade de raciocínio
Exposição local	Ngrok 3.39.1	Túnel HTTPS para receção de webhooks em dev cuja configuração em ambiente de desenvolvimento é visível na Figura B.3 (Anexo B)
Pipeline de qualidade local	verify_all.py (script customizado)	11 quality gates de integração contínua local

O serviço é exposto através do FastAPI, uma framework Python assíncrona baseada em ASGI, cuja adoção é justificada pelo mecanismo de BackgroundTasks que permite retornar a resposta HTTP imediata ao Webhook do Azure DevOps enquanto a inferência decorre em segundo plano, solução direta ao contrato de 30 segundos impostos pela plataforma. O Docker garante a portabilidade e reprodutibilidade do ambiente de execução, encapsulando a aplicação e dependências numa imagem de contentor isolada; a orquestração dos serviços complementares (incluindo o Redis para deduplicação distribuída) é gerida via Docker Compose. Os detalhes de configuração e as decisões de segurança de containerização são documentados na Secção 4.1.5. A configuração do túnel HTTPS de desenvolvimento, utilizado para expor o endpoint local à plataforma Azure DevOps, é apresentada na Figura B.3 (Anexo B).

4.1.2 Protótipo e Integração com o Azure DevOps

O projeto iniciou-se com um protótipo monolítico para validação da receção de webhooks do Azure DevOps. Durante esta fase inicial, identificou-se um obstáculo crítico na configuração do Service Hook: a API de iterações de Pull Request não devolvia o código-fonte dos ficheiros alterados, mas apenas os seus metadados e caminhos. Consequentemente, o modelo de IA recebia um contexto vazio, impossibilitando qualquer análise real.

A resolução deste problema exigiu a extração do identificador exato do commit (SHA) através do campo lastMergeSourceCommit, presente nos metadados gerais do Pull Request. Com o SHA disponível, tornou-se possível invocar o endpoint de conteúdo de ficheiros (/items) da REST API do Azure DevOps, descarregando o texto real. O código foi adicionalmente pré-processado com a injeção de números de linha antes do envio ao modelo, garantindo que os findings gerados referenciassem localizações concretas e verificáveis [10].

Este protótipo validou a viabilidade da integração bidirecional com o Azure DevOps, mas expôs imediatamente o segundo obstáculo crítico: a incompatibilidade entre o tempo de processamento do LLM e o contrato de timeout imposto pela plataforma, abordado na secção seguinte.

A plataforma Azure DevOps impõe um contrato temporal rígido: respostas HTTP de sucesso em aproximadamente 30 segundos, sob pena de reenvio automático do webhook. Como a inferência dos LLMs demorava entre 40 e 90 segundos, a plataforma interpretava sistematicamente a latência como falha, disparando processamento redundante do mesmo Pull Request. Podendo levar, em alguns casos, quatro a cinco instâncias concorrentes, com esgotamento de quota da API e comentários duplicados.

Para resolver o problema, foi necessário atacá-lo em três frentes em simultâneo. A primeira foi a mais imediata: refatorizar o endpoint de receção de webhooks para responder logo, usando o BackgroundTasks do FastAPI [42]. A partir daí, o sistema devolve um HTTP 202 Accepted em milissegundos, o prazo do Azure DevOps deixou de ser um problema, e a extração de código, a inferência e a publicação do relatório passaram a correr numa tarefa desacoplada, sem qualquer relação com o ciclo de vida do pedido HTTP original.

O Snippet abaixo ilustra a implementação deste endpoint assíncrono na API de webhooks (src/api/webhooks.py), evidenciando a técnica de injeção de dependências do FastAPI (Depends) para receber o orquestrador do pipeline de forma desacoplada. O processamento de inferência semântica é diferido de forma assíncrona recorrendo ao serviço BackgroundTasks da framework, permitindo que a rota devolva instantaneamente um código de estado 202 Accepted ao Azure DevOps e previna o cancelamento prematuro por timeout da plataforma.

```
@router.post("/orchestrate", dependencies=[Depends(check_webhook_secret), Depends(check_rate_limit)])
async def webhook_orchestrate(
    payload: ADOWebhookPayload,
    background_tasks: BackgroundTasks,
    orchestrator: PipelineOrchestrator = Depends(get_pipeline_orchestrator)
) -> JSONResponse:
    """Recebe eventos de Pull Request do ADO e inicia o processamento assíncrono."""
    pr_id = payload.resource.pullRequestId
    repo_id = payload.resource.repository.id
    project = payload.resource.repository.project.name

    # Delegação da tarefa pesada do pipeline ao executor em segundo plano
    background_tasks.add_task(
        orchestrator.process_pr_pipeline,
```

```
    pr_id,
    repo_id,
    project
)

# Resposta HTTP 202 imediata para evitar o timeout de 30 segundos do ADO
return JSONResponse(
    status_code=202,
    content={
        "status": "accepted",
        "message": f"Orchestrator Pipeline started for PR {pr_id}"
    }
)
```

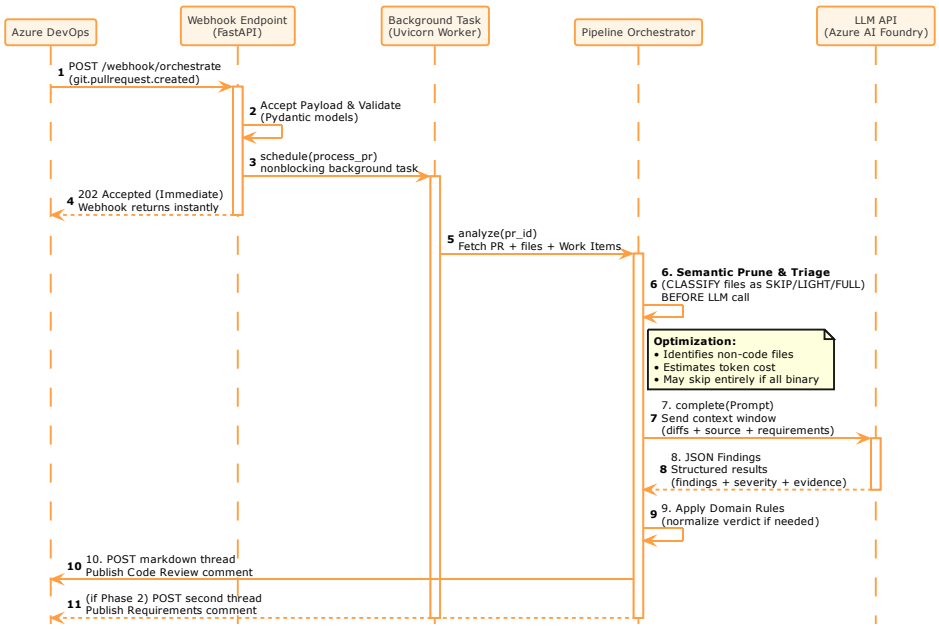


Figura 10 - Diagrama de sequência do processamento assíncrono de Webhooks (Elaboração própria)

Ainda assim, responder depressa não chegava. Se o webhook disparasse duas vezes para o mesmo PR, o que acontecia, ambas as tarefas corriam em paralelo. A solução foi verificar atomicamente, antes de qualquer inferência, se o PR já estava a ser processado; se sim, a segunda notificação é simplesmente descartada. Isto evita tanto o desperdício de quota de API como comentários duplicados no repositório.

O terceiro problema era mais subtil: falhas transitórias nas APIs externas, os habituais 429 e 503, podiam interromper a análise a meio. A primeira implementação usava recursão com backoff, o que funcionava, mas criava risco de stack overflow em instabilidades prolongadas. Passou-se para um ciclo iterativo com espera exponencial e um componente aleatório (jitter), que distribui as retentativas no tempo e evita que todas as instâncias batam na API ao mesmo segundo, o chamado Thundering Herd Problem [43].

4.1.3 Migração para a Cloud e Modelos

A inviabilidade da execução local, conjugada com os requisitos de privacidade estabelecidos (RNF01), motivou a migração para o Azure AI Foundry no paradigma Model-as-a-Service. Esta migração foi acompanhada de uma reestruturação profunda do código: o script monolítico inicial, que concentrava toda a lógica numa única unidade, foi decomposto em módulos com responsabilidades claramente separadas, seguindo os princípios de Clean Architecture e o padrão Ports & Adapters descrito no Capítulo 3 [38].

Esta reestruturação foi motivada por razões práticas imediatas: à medida que a integração com o Azure AI Foundry expôs novos requisitos de autenticação, tratamento de erros e configuração, tornou-se evidente que qualquer alteração ao script monolítico requeria a compreensão do contexto global, aumentando o risco de regressões. A separação em módulos com interfaces bem definidas permitiu iterar sobre cada componente de forma independente.

Os blocos lógicos resultantes da reestruturação correspondem diretamente à arquitetura Ports & Adapters descrita na Secção 3.3.2.

A migração para o Azure AI Foundry expôs imediatamente uma restrição operacional determinante: a subscrição Azure for Students utilizada no âmbito do estágio não permite a aceitação de termos do Azure Marketplace, o que bloqueou o deployment de modelos de fornecedores terceiros. A alternativa disponível no catálogo nativo da Microsoft foi o Phi-4-mini-instruct, um modelo de 3,8 mil milhões de parâmetros.

A indisponibilidade de modelos de fornecedores terceiros na subscrição académica tornou-se uma vantagem metodológica: validar a infraestrutura de integração com o Phi-4-mini, de inferência rápida e custo mínimo, permitiu isolar e testar autenticação, parsing e publicação de comentários antes de escalar para modelos mais complexos.

A integração com o Azure AI Foundry revelou duas incompatibilidades não documentadas de forma proeminente. Em primeiro lugar, os modelos nativos da Microsoft nesta plataforma utilizam um mecanismo de autenticação específico, distinto do protocolo OAuth standard, que exige o envio da chave no cabeçalho api-key em vez do Authorization: Bearer convencional. Em segundo lugar, o endpoint de invocação do modelo não incluía o identificador do modelo na URL, ao contrário do que seria expectável por analogia com outras APIs de IA: o identificador era passado no corpo do payload JSON, e a rota base era partilhada por todos os modelos disponíveis na instância. Estas incompatibilidades foram identificadas através da análise dos cabeçalhos das respostas de erro e corrigidas de forma localizada no adaptador de infraestrutura, sem impacto na lógica de negócio.

Este isolamento foi possível precisamente pela arquitetura Ports & Adapters adoptada: a lógica de autenticação reside exclusivamente no adaptador azure_ai (src/infra/), sem que o orquestrador ou os agentes tenham conhecimento do protocolo subjacente.

4.1.4 Clean Architecture e Governança

A transição prática para a Arquitetura Hexagonal (Ports & Adapters), cujo desenho conceptual foi estabelecido no Capítulo 3, impôs-se por necessidades empíricas do projeto. O sistema demonstrou na prática a alta volatilidade da infraestrutura externa: o modelo inicial Phi-4-mini foi posteriormente complementado pelo Llama 3.3 e, por fim, substituído pela configuração multiagente com o4-mini e DeepSeek-V3.2. Esta sucessão de três gerações de modelos, posteriormente validada em seis conjuntos de teste distintos (Secção 4.2.1), constitui a prova empírica mais robusta da agnosticidade arquitetural do sistema.

Esta reestruturação materializou-se na organização do código em três zonas com direções de dependência explicitamente controladas:

- **Núcleo de domínio (src/domain/ e src/services/)** — contém o orquestrador, os agentes de revisão e o motor de normalização de veredictos. Esta zona não importa nenhuma biblioteca de infraestrutura (sem referências ao Azure SDK, ao httpx ou ao FastAPI). Toda a comunicação com o exterior é feita através de interfaces definidas em src/ports/.
- **Ports (src/ports/)** — ficheiros que definem exclusivamente Protocols do Python, os equivalentes a interfaces em linguagens tipadas estaticamente. Definem o contrato do que um cliente de IA deve fazer (invocar um modelo e devolver uma resposta estruturada) e o que um cliente de repositório deve fazer (obter o conteúdo de um ficheiro, publicar um comentário), sem ditar como.
- **Adapters (src/infra/)** — contém as implementações concretas: o cliente HTTP para o Azure DevOps, o cliente para o Azure AI Foundry (com as variantes de autenticação para modelos nativos e modelos serverless), e o adaptador de Redis para a deduplicação distribuída. É nesta zona que residem todas as dependências de bibliotecas externas.

A ligação entre as zonas é realizada através de um componente de composição (src/bootstrap.py) que, no arranque da aplicação, lê a configuração e instância os adaptadores concretos, injetando-os nos serviços de domínio que os requerem. Este padrão, designado Composition Root na literatura de injecção de dependências, garante que as decisões de infraestrutura são tomadas num único ponto do sistema, e que o código de domínio nunca instância diretamente uma dependência concreta [44]. A Tabela 9 sintetiza a separação de responsabilidades resultante desta reestruturação.

Tabela 9 - Separação de responsabilidades na arquitetura Ports & Adapters implementada (Elaboração própria)

Zona	Módulos	Dependências permitidas
Domínio	orchestrator, code_review, requirements_review, verdict_normalization	Apenas Ports (interfaces)
Ports	IAIClient, IRepositoryClient, IDeduplicationClient	Nenhuma
Adapters	azure_ai, azure_devops, redis_dedup	Bibliotecas externas (httpx, redis)
Composição	bootstrap, di	Todas (ponto único de ligação)
API Gateway	webhooks (FastAPI)	Ports + serviços de domínio

Os diagramas granulares das camadas implementadas, a organização interna dos componentes e os padrões de design aplicados (Strategy, Dependency Injection) encontram-se nas Figuras D.6, D.9 e D.10 (Anexo D).

Os limites de acoplamento expressos na coluna 'Dependências permitidas' funcionam como o mecanismo prático de blindagem da arquitetura. Ao interditar qualquer importação de bibliotecas ou SDKs de infraestrutura no núcleo de domínio, garantiu-se que as migrações táticas de modelos de linguagem e a reconfiguração de endpoints corporativos (Secção 4.1.3) fossem executadas sem causar qualquer tipo de regressão na lógica de negócio do orquestrador.

A migração para o Llama-3.3-70B-Instruct, disponibilizado através de uma subscrição entretanto alargada, melhorou substancialmente a qualidade do raciocínio semântico, mas trouxe novos desafios de fiabilidade que exigiram uma abordagem sistemática de engenharia de prompt e governança determinística dos outputs.

O problema apareceu cedo: correr o mesmo ficheiro duas vezes dava resultados opostos. Aprovação numa reprovação noutra, com a pontuação a contradizer o veredicto. Um modelo probabilístico não pode ter autoridade de bloqueio num pipeline de CI/CD, porque não é auditável [24].

A decisão foi tirar ao modelo qualquer poder de decisão. O LLM passou a ser apenas um extrator de dados estruturados: recebe código, devolve uma lista de findings com severidade. Quem decide é o Python, com regras fixas e auditáveis. A pontuação começa em 10 e cada finding desconta um valor

predefinido (Crítico: −3,0; Alto: −1,5; Médio: −0,5; Baixo: −0,15). O PR só é bloqueado por findings críticos ou por um número excessivo de findings altos, e esse limiar é configurável.

Esta arquitetura de governança determinística está materializada no algoritmo descrito no snippet abaixo. O motor recebe a lista estruturada de achados analíticos (CodeFinding) devolvida pelo modelo de linguagem e calcula a classificação final, aplicando as deduções parametrizadas. Caso se verifique a existência de qualquer anomalia crítica (ponto de bloqueio imperativo), o veredicto global é forçado para NEEDS_WORK independentemente do score obtido, assegurando uma barreira de qualidade intransponível no pipeline de integração.

```
DEDUCTIONS = {"critical": 3.0, "high": 1.5, "medium": 0.5, "low": 0.15}

def calculate_pr_verdict(findings: list[CodeFinding]) -> dict:
  """Calcula deterministicamente a pontuação de qualidade e o veredicto de bloqueio do PR."""
  score = 10.0
  for f in findings:
    # Aplica as deduções matemáticas de acordo com a severidade do achado
    score -= DEDUCTIONS.get(f.severity.lower(), 0.0)

  # Validação imperativa da barreira de bloqueio por anomalia crítica (MUST gate)
  blocked = any(f.severity.lower() == "critical" for f in findings)

  return {
    "score": max(score, 0.0),
    "verdict": "NEEDS_WORK" if (blocked or score < 7.0) else "APPROVED"
  }
```

Princípio Arquitetural: O modelo de linguagem actua como extractor semântico e consultor, enquanto o motor de governança determinístico em Python actua como autoridade final de decisão. A aprovação ou rejeição de um Pull Request nunca é delegada a um sistema probabilístico.

O diagrama de sequência expresso na Figura 11 materializa operacionalmente este princípio de governança determinística. Ilustra-se de que forma o Knowledge Ledger atua como um barramento de dados partilhado entre a Fase 1 e a Fase 2, mitigando o risco de reanálise e assegurando a coerência lógica dos relatórios técnicos.

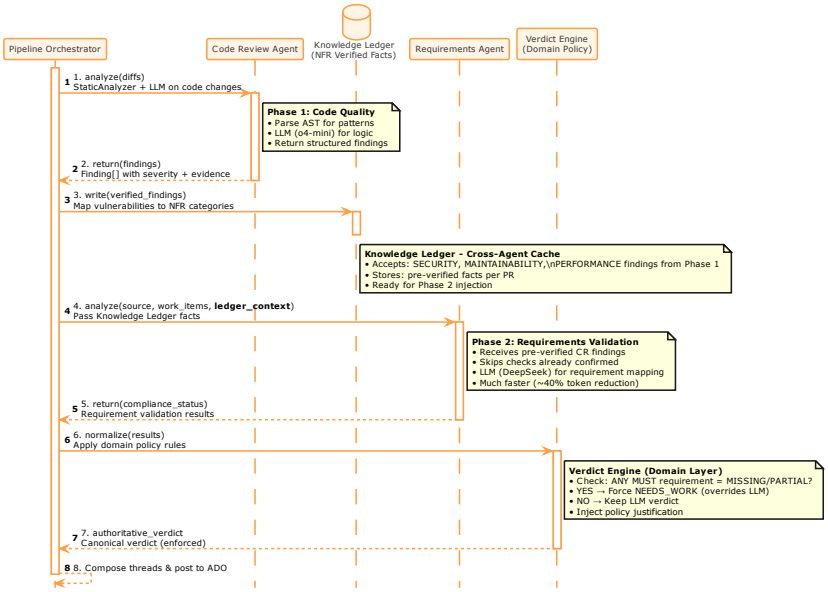


Figura 11 - Diagrama de sequência da orquestração multiagente e partilha de contexto (Elaboração própria)

Para reduzir a taxa de falsos positivos, o system prompt foi reestruturado com base na técnica Chain-of-Thought (CoT), que instrui o modelo a decompor o raciocínio em passos intermédios explícitos antes de emitir um finding: identificar a localização suspeita, confirmar que o problema existe tal como está escrito no código, e validar contra um conjunto de proibições absolutas [3]. A secção de restrições absolutas do prompt listava regras derivadas dos padrões de falsos positivos observados na fase anterior, incluindo, por exemplo, a proibição explícita de classificar chamadas a funções de leitura de variáveis de ambiente como exposição de credenciais, e de classificar concatenação parametrizada de queries como injecção SQL. Esta abordagem reduziu significativamente o ruído sem comprometer a sensibilidade para vulnerabilidades reais.

A Figura 12 formaliza rigorosamente o fluxo lógico de decisão que rege o Motor de Normalização de Veredictos, evidenciando o caminho crítico determinístico: perante a deteção de uma violação catalogada como mandatória (MUST), o sistema sobrepõe-se automaticamente ao output probabilístico do modelo e força o estado final para NEEDS_WORK.

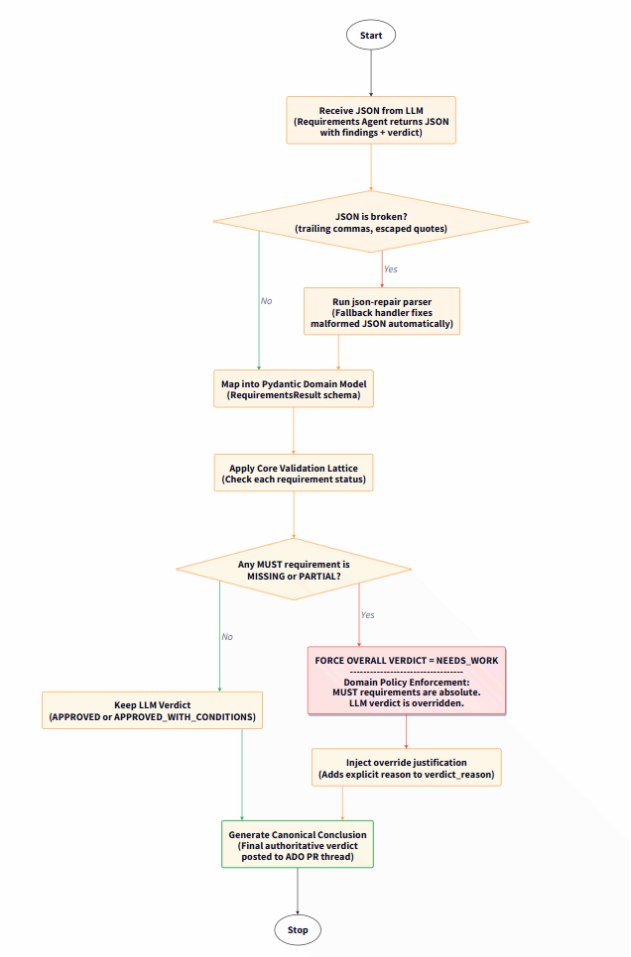


Figura 12 - Fluxo de execução do Motor de Normalização de Veredictos (Elaboração própria)

Este isolamento de fluxos lógicos serve de fundação para a máquina de estados distribuída que governa o ciclo de vida completo das validações, conforme descrito na subsecção subsequente.

Mesmo com a lógica de pontuação implementada em Python, identificou-se um risco residual de inconsistência: em contextos extensos, o modelo podia classificar um requisito obrigatório como ausente, mas emitir simultaneamente um veredicto global de aprovação por considerar o código bem estruturado noutras dimensões. Para eliminar esta contradição estrutural, foi implementado um motor de normalização de veredictos, uma camada de governança local que interceta o output estruturado do LLM antes da formatação do relatório final. Se for detetada qualquer violação às regras de domínio configuradas, o sistema sobrepõe-se autonomamente ao veredicto probabilístico do modelo, força o estado final para reprovação e injeta uma anotação identificável na justificação, garantindo rastreabilidade completa da decisão.

O fulcro deste ciclo de vida decisional reside na aplicação do estado transitório determinístico VerdictEnforced. Este mecanismo operacionaliza a garantia lógica e matemática do sistema: assim que uma quebra de política mandatória (requisitos MUST) ou uma vulnerabilidade severa é intercetada no pipeline, o motor bloqueia rigidamente qualquer transição para um estado de aprovação técnica. Deste

modo, inviabilizam-se aprovações acidentais e exige-se uma nova submissão de código (commit) para reavaliar o estado do repositório, garantindo a integridade do processo de entrega contínua.

4.1.5 Parsing e Segurança

A verbosidade do Llama-3.3-70B em ficheiros extensos provocava frequentemente a resposta atingir o limite configurado de tokens, resultando em JSON truncado a meio de um objeto. O parser do Python falhava e descartava todo o resultado, incluindo vulnerabilidades críticas já identificadas antes do ponto de corte.

A solução adoptou três mecanismos de proteção aplicados sequencialmente:

- **Recuperação de findings parciais:** Em caso de falha no parsing do JSON completo, um algoritmo de recuperação percorre a string caracter a caracter, contabilizando a profundidade da árvore JSON através da contagem de parênteses de abertura e fecho. Os objetos JSON completos e balanceados que antecedem o ponto de truncagem são extraídos e processados, preservando os findings já concluídos pelo modelo.
- **Sanitização de escape sequences inválidas:** Os modelos de linguagem emitem frequentemente sequências de escape inválidas em contexto JSON — em particular em expressões regulares ou em caminhos de ficheiros. Um algoritmo de sanitização aplica substituições por expressão regular antes do parsing e remove vírgulas antes de fecho de arrays ou objetos, tornando o JSON recuperável.
- **Representação de código como array de strings:** O prompt foi alterado para proibir a devolução de blocos de código como strings de múltiplas linhas em JSON [45]. Todo o código evidenciado passou a ser representado como um array de strings — uma por linha de código — eliminando por completo as quebras de sintaxe JSON causadas por newlines não escapados.

Paralelamente, o limite máximo de tokens na resposta foi progressivamente ajustado, de 4.000 para 8.000 e posteriormente para 16.000 em ficheiros com contexto de requisitos extenso, e o timeout de inferência foi igualmente ajustado para acomodar os tempos de raciocínio dos modelos de maior escala.

A preparação do sistema para um contexto de produção empresarial (enterprise-readiness) exigiu um conjunto de medidas de segurança adicionais, identificadas ao longo do processo de desenvolvimento e consolidadas na fase de estabilização da arquitetura:

- **Validação de transporte seguro:** O sistema verifica no arranque se o endpoint configurado para o Azure AI Foundry utiliza o protocolo HTTPS. Em caso contrário, a aplicação termina imediatamente com um erro explícito, prevenindo a transmissão de credenciais de API em canal não cifrado.

- **Autenticação de webhooks por segredo partilhado:** Foi implementada a verificação de integridade e autenticidade em todos os pedidos recebidos através de assinaturas HMAC-SHA256. O segredo partilhado é configurado de forma segura através de variáveis de ambiente no contentor e a sua validação é realizada em tempo constante (recorrendo à primitiva secrets.compare_digest), prevenindo ativamente ataques de temporização (timing attacks) que poderiam comprometer as credenciais do sistema e rejeitando pedidos inválidos com o código HTTP 401.
- **Endurecimento e Proteção do API Gateway (Anti-DoS):** Para responder ao requisito RNF03 e mitigar ataques de esgotamento de recursos, o endpoint foi blindado com um teto de payload máximo de 2MB, validação estrita de contrato via esquemas Pydantic e um mecanismo de rate limiter em janela deslizante (thread-safe) com bloqueio de acessos abusivos através do código HTTP 429.
- **Execução com utilizador sem privilégios no contentor:** O Dockerfile foi alterado para criar e executar o processo principal com um utilizador sem privilégios de administrador, mitigando riscos de escalonamento em caso de comprometimento da aplicação.
- **Isolamento do código-fonte e mitigação de Prompt Injection: O** código extraído do repositório é envolvido em delimitadores estruturais XML (ex: <file_to_review>) e acompanhado de instruções estritas que forçam o modelo a tratar o seu conteúdo como "DATA ONLY". Esta técnica impede que comentários maliciosos no próprio código-fonte alterem o comportamento do modelo [46], [47].

4.1.6 Otimização e Multiagente

Com o sistema a funcionar de forma estável, o problema seguinte era o custo. Em PRs com muitos ficheiros pequenos, o número de chamadas à API disparava facilmente para dezenas por análise, o que tornava o custo por PR incompatível com qualquer utilização em produção.

Foi implementada uma variável de configuração que define o orçamento máximo de tokens por Pull Request. A cada resposta da API de inferência, o sistema acumula o consumo real através do campo de contagem de tokens presente na resposta. Quando o orçamento configurado é atingido, o agente executa uma paragem controlada da iteração corrente e submete o relatório parcial com os findings já processados, em vez de abortar silenciosamente ou continuar com consumo descontrolado.

A otimização mais significativa em termos de redução de consumo de tokens e de falsos positivos foi a introdução de uma camada de análise estática determinística, executada antes de qualquer chamada ao modelo de linguagem [4]. Utilizando expressões regulares, esta camada deteta

automaticamente problemas que os LLMs tendem a tratar de forma inconsistente: imports não utilizados, instruções de depuração em produção, ou capturas de exceções excessivamente genéricas.

A versão consolidada deste motor estático estende-se a uma análise estática multilinguagem, detetando de forma determinística tokens de autenticação estruturados (como JWTs expostos), permissões de CORS excessivas, desativações perigosas de verificação SSL/TLS e funções vulneráveis de gestão de memória em C/C++ (ex: strcpy), aliviando drasticamente a carga semântica sobre o LLM e garantindo que o seu contributo se foca exclusivamente na análise de segurança complexa. O impacto quantitativo desta camada estática na taxa de deteção global e na redução de tokens é documentado na Secção 4.2.2 (Tabela 16).

- **Filtragem de ficheiros de renomeação:** Exclusão preventiva de ficheiros cujo tipo de alteração seja estritamente a alteração de nome, eliminando o envio de metadados irrelevantes para a API.
- **Compressão do system prompt:** Expurgado de diretivas redundantes para mitigar o payload base e maximizar o orçamento de tokens disponível por Pull Request.
- **Ajuste da janela de particionamento:** Ampliação dos blocos de código transmitidos por chamada, capitalizando a capacidade de retenção de informação dos LLMs em contextos extensos.
- **Governação Descentralizada via Regras de Repositório:** Leitura dinâmica de ficheiros locais (.codereview.yml ou .requirements.yml) para a injeção autónoma de regras de validação customizadas por equipa.

Enviar ficheiros completos para a validação de requisitos inflacionava desnecessariamente a context window [35]. A tentativa inicial com compressão por AST não resultou, mas apagava a semântica que o modelo precisava de ver. A solução desenvolvida foi o AC-Aware Context Pruning: extrai termos-chave dos Work Items e usa-os como filtro para o código-fonte, mantendo apenas o esqueleto estrutural e as linhas que cruzam com esses termos. O algoritmo corre em duas fases:

- **Fase 1**: Primeiro, um extrator analisa os Work Items e identifica dinamicamente as "entidades de negócio" (ex: "fatura").
- **Fase 2**: De seguida, o filtro percorre o código-fonte e retém apenas a estrutura "muscular" juntamente com as linhas de código que contêm essas entidades. Esta técnica mitigou o ruído de input preservando a informação estritamente necessária (com ganhos de compressão significativos, documentados na Secção 4.2.2).

O Snippet abaixo descreve o funcionamento programático deste algoritmo de higienização de contexto. Na primeira fase, extraem-se de forma dinâmica os termos e identificadores de negócio a

partir dos critérios de aceitação e descrições dos work items, aplicando-se uma lista de exclusão (stopwords) para purificar o léxico. Na segunda fase, o motor filtra as linhas do ficheiro de código-fonte original, retendo as assinaturas de estrutura (decoradores, classes e funções) e as linhas que integram os termos extraídos, removendo a verbosidade de lógica utilitária e mantendo o alinhamento funcional para a validação.

```
def extract_ac_terms(acceptance_criteria: list[str]) -> set[str]:
    """Fase 1: Extração dinâmica de termos semânticos e identificadores das ACs."""
    terms = {"success", "error"} # Termos estruturais base
    for ac in acceptance_criteria:
        # Captura sequências alfanuméricas e identificadores relevantes
        words = re.findall(r"\b[a-zA-Z0-9_]{4,}\b", ac)
        terms.update(w.lower() for w in words if w.lower() not in STOPWORDS)
    return terms

def prune_source_code(source: str, ac_terms: set[str]) -> str:
    """Fase 2: Filtra e condensa o código-fonte com base nos termos extraídos."""
    pruned_lines = []
    for line in source.splitlines():
        stripped = line.strip()
        # Retém a estrutura essencial do código e as linhas que cruzam com termos de negócio
        if (stripped.startswith(("def ", "class ", "@", "return")) or
            any(term in stripped.lower() for term in ac_terms)):
            pruned_lines.append(line.rstrip())

    return "\n".join(pruned_lines)
```

A implementação do Model Routing exigiu um adaptador de infraestrutura capaz de lidar com a heterogeneidade das APIs no Azure AI Foundry. O adaptador deteta dinamicamente o tipo de modelo requisitado pelo orquestrador e ajusta em tempo de execução a estrutura de endpoints e os cabeçalhos de autenticação, alternando automaticamente entre `api-key` para modelos nativos da Microsoft e `Authorization: Bearer` para implementações serverless como o DeepSeek-V3.2.

Adicionalmente, perante as restrições de interface impostas por modelos de raciocínio avançado como o o4-mini (que não suportam o parâmetro de temperatura e rejeitam system prompts separados), o componente de construção de payloads foi instrumentado para incorporar dinamicamente as instruções de sistema no início da mensagem do utilizador, encapsulando toda a complexidade de integração [29], [48].

A topologia final do ecossistema distribuído e as interações em tempo de execução entre os componentes adaptativos encontram-se mapeadas na Figura 13, que serve de referência visual para o fluxo de roteamento semântico.

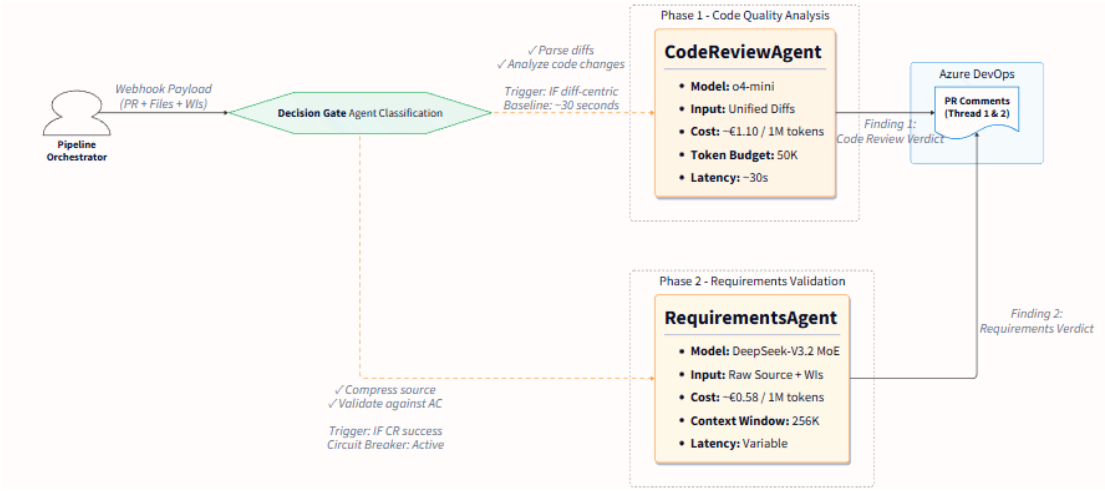


Figura 13 - Arquitetura de Orquestração Multi-LLM e fluxo de dados em tempo de execução (Elaboração própria)

As duas ramificações operacionais patentes na arquitetura anterior materializam as decisões de Model Routing detalhadas adiante: o CodeReviewAgent executa o processamento focado sobre trechos incrementais (diffs), otimizando a latência, enquanto o RequirementsAgent assume o processamento extensivo de contextos complexos tirando partido da eficiência por token das arquiteturas Mixture-of-Experts.

A solução adotada baseia-se no paradigma de roteamento dinâmico de modelos (model routing), que atribui a cada tipo de tarefa o modelo arquiteturalmente mais adequado às suas características específicas. Mais concretamente o volume de contexto necessário e à profundidade de raciocínio requerida [37]:

- **Agente de Code Review (o4-mini):** Selecionado pela sua capacidade de raciocínio profundo (slow thinking) sobre blocos reduzidos de contexto. A inovação central deste agente foi a transição de ficheiros completos para diffs puros: em vez de enviar o ficheiro na íntegra, o sistema gera programaticamente um patch com apenas as linhas modificadas entre a versão anterior e a nova versão. Para viabilizar o mecanismo de Lazy Snippet Enrichment descrito adiante nesta secção, este diff preserva os metadados de posição originais do git (cabeçalhos @@ -l,s +l,s @@), permitindo que o modelo identifique as coordenadas das linhas absolutas no ficheiro original mesmo operando sobre um contexto minimalista [29]. O agente recebe exclusivamente este conjunto reduzido de alterações e foca o seu raciocínio interno nas mudanças concretas introduzidas pelo Pull Request, com um consumo de input substancialmente inferior ao da abordagem por ficheiro completo [29].
- **Agente de Revisão de Requisitos (DeepSeek-V3.2):** Selecionado pela sua arquitetura Mixture-of-Experts (MoE), que lhe confere a capacidade de processar contextos extensos a custo operacional reduzido [28]. Este agente mantém acesso aos ficheiros completos e ao

conteúdo integral dos Work Items, sendo otimizado para a análise transversal de conformidade funcional que requer a leitura simultânea de múltiplas fontes de informação [28].

Um risco inerente aos Sistemas Multiagente é o fenómeno de redundância cognitiva ou *double-thinking*. Neste cenário, múltiplos agentes independentes tendem a reanalisar e a reportar as mesmas anomalias não-funcionais (NFRs), gerando desperdício computacional e introduzindo potenciais contradições no relatório final.

Para solucionar este desafio arquitetural, o orquestrador implementou um *Knowledge Ledger*, uma memória estruturada partilhada, em detrimento de um simples histórico linear de mensagens [19]. O pipeline é executado sequencialmente: o Agente de Code Review atua em primeiro lugar, focando-se no diff. As anomalias ou vulnerabilidades validadas por este agente (por exemplo, a deteção de um segredo *hardcoded*) são registadas como factos imutáveis neste *Ledger*.

Quando o Agente de Revisão de Requisitos inicia a sua análise transversal, o conteúdo do *Knowledge Ledger* é injetado diretamente no seu *prompt* inicial como conhecimento pré-validado [33]. Esta estratégia de propagação de contexto garante que o segundo agente não despende ciclos de raciocínio a investigar problemas já sinalizados, focando-se estritamente na conformidade funcional estendida e assegurando a absoluta coerência do veredicto do sistema.

4.1.7 Escalabilidade e Observabilidade

O mecanismo inicial de deduplicação baseado num lock em memória RAM funcionava corretamente para uma única instância. Contudo, num cenário de escalabilidade horizontal com múltiplos contentores, um evento de webhook redundante poderia atingir uma réplica distinta, causando análises duplicadas e custos adicionais. A solução adotada foi a integração de um contentor Redis como distributed lock: todas as instâncias do agente consultam a mesma base de dados em memória partilhada antes de iniciar o processamento, garantindo que cada Pull Request é analisado exatamente uma vez ao nível do cluster [34]. O protocolo de deduplicação distribuída via Redis (SET NX EX) e o ciclo de vida das instâncias concorrentes encontram-se detalhados na Figura D.11 (Anexo D).

O endpoint de verificação de saúde do serviço foi implementado além da resposta simples de disponibilidade: ao ser invocado, verifica ativamente o estado da ligação ao orquestrador e ao Redis, reportando degradação parcial do serviço em caso de falha de qualquer adaptador. Esta verificação aprofundada permite que a plataforma de orquestração de contentores reinicie automaticamente instâncias degradadas. A contabilização de métricas operacionais (latências e consumo de tokens) é realizada a partir dos campos de utilização presentes nas respostas das APIs de inferência. O comportamento do contentor Docker em estado idle e durante o processamento activo de um Pull Request é ilustrado nas Figuras B.4 e B.5 (Anexo B).

Na ausência de um pipeline de integração contínua dedicado no Azure DevOps para o próprio agente, foi desenvolvido um script de qualidade local composto por 11 verificações sequenciais (quality gates): integridade de importação, conformidade de estilo (PEP8), testes unitários ao motor de análise estática, resiliência do parser JSON e testes de segurança com payloads forjados. A suíte de testes unitários e de integração totaliza 201 testes aprovados, com uma cobertura global de 91,32% sobre 1.694 instruções do núcleo de domínio. A cobertura atinge 100% nos módulos críticos de fronteira — webhooks.py (API Gateway), knowledge_ledger.py, llm_json.py e resilience.py — garantindo a validação exaustiva dos pontos de entrada e dos mecanismos de resiliência. Os módulos com menor cobertura (orchestrator.py a 78% e redis_dedup.py a 80%) devem-se à sua dependência de I/O de rede externo e de fluxos assíncronos complexos, sendo a sua validação complementar garantida através dos testes de integração end-to-end descritos na Secção 4.2. O relatório completo de cobertura, gerado pelo pytest-cov, encontra-se na Figura B.6 (Anexo B).

Por forma a consolidar os mecanismos de injeção dinâmica de conhecimento e as diretivas de formatação estritas partilhadas pelos agentes, a Tabela 10 pormenoriza a anatomia estrutural do prompt final parametrizado no motor. Os módulos com menor cobertura unitária (adaptadores azure_ai e azure_devops) devem-se à sua dependência de I/O de rede externo, sendo a sua validação garantida através dos testes de integração end-to-end (Secção 4.2)

Tabela 10 - Estrutura do Prompt com injeção do Knowledge Ledger e restrições explícitas (Elaboração própria)

Componente	Tipo	Conteúdo	Mutável?
System Prompt	Imutável	Persona, restrições absolutas, schema JSON	Não
Knowledge Ledger	Dinâmico	Findings validados da Fase 1	Por PR
Acceptance Criteria	Dinâmico	ACs filtrados e relevantes	Por PR
Source Code Blocks	Dinâmico	Código comprimido (AC-Aware Pruning)	Por ficheiro
Target Instruction	Imutável	"Audit/Review" — instrução de tarefa	Não

Esta separação rigorosa entre componentes imutáveis (personas corporativas e restrições analíticas) e componentes dinâmicos (contexto injetado por Pull Request) constitui o pilar que dota o sistema de estabilidade comportamental. Garante-se, deste modo, que a aplicação apresente respostas

homogéneas e preditivas independentemente do repositório ou da linguagem de programação sob análise.

O relatório publicado no Azure DevOps estrutura-se num formato Markdown progressivo: um sumário executivo com a pontuação global, uma tabela de findings por severidade e o detalhe técnico de cada vulnerabilidade [16]. A estrutura final apresenta três níveis de detalhe progressivo:

- **Sumário executivo:** Pontuação de segurança representada por barra de progresso em texto, veredicto final e contagem de findings por nível de severidade.
- **Tabela de findings:** Listagem dos problemas identificados com severidade, tipo, ficheiro, linha de código e título descritivo.
- **Detalhe por finding:** Para cada problema, o relatório apresenta a localização exacta, o excerto de código problemático, a descrição técnica do risco e uma sugestão de correcção concreta com código alternativo. Exemplos reais do output gerado pelos dois agentes encontram-se nas Figuras B.1 e B.2 (Anexo B).

Para conjugar a poupança radical de tokens (obtida pelo envio exclusivo de diffs reduzidos ao agente o4-mini) com a necessidade de apresentar blocos de código contextualizados e legíveis no comentário final, desenvolveu-se o mecanismo de Lazy Snippet Enrichment. Sob esta técnica, o modelo de linguagem atua sobre o patch minimalista e devolve apenas a coordenada da linha absoluta do problema. Após a receção do veredicto, o backend cruza essa coordenada com a cache do código-fonte completo guardada em memória, expandindo o excerto visual dinamicamente para incluir as linhas de contexto adjacentes. Este desacoplamento entre a inteligência analítica restrita e a apresentação visual enriquecida garante uma experiência de utilizador premium sem agravar o orçamento de inferência.

4.2 Testes

O protocolo de avaliação segue os princípios da investigação empírica em engenharia de software [49] e assenta em três opções metodológicas deliberadas: usar testes construídos com defeitos injetados e catalogados, comparar três arquiteturas progressivamente mais complexas e medir em simultâneo deteção, custo e latência. As razões de cada escolha são explicadas nas subsecções seguintes.

4.2.1 Protocolo Experimental

Ao longo do ciclo de desenvolvimento foram submetidos 145 Pull Requests de teste no repositório de validação do Azure DevOps. Os restantes 139 serviram propósitos iterativos de infraestrutura e afinação: validação da receção de webhooks, testes de parsing JSON, ajustes de prompt engineering, calibração de timeouts, verificação de rate limiting e estabilização da integração com o

Azure AI Foundry. Deste conjunto, seis Pull Requests foram selecionados como benchmark final controlado, construídos deliberadamente com defeitos conhecidos e previamente catalogados, permitindo calcular taxas de verdadeiros positivos, falsos positivos e falsos negativos de forma determinística.

Os seis conjuntos de teste organizam-se em dois domínios de aplicação distintos (gestão de projetos/utilizadores e domínio comercial) e dois níveis de complexidade dimensional (Medium e Massive), conforme sintetizado na Tabela 11. Esta duplicação de domínios visa controlar a variável "familiaridade do modelo com o vocabulário de negócio", evitando que os resultados reflitam apenas a especialização do modelo num único domínio semântico.

Tabela 11 - Características dos seis conjuntos de teste do benchmark final (Elaboração própria)

Conjunto	Linhas	Bugs	Críticos	ACs violados
Medium 1	~180	14	4	9/12
Medium 2	~190	14	4	11/12
Medium 3	~200	14	4	11/12
Massive 1	~700	20	5	8/12
Massive 2	~720	20	5	11/12
Massive 3	~450	20	5	12/12
Total	~2.440	102	26	62

A designação dos conjuntos de teste como Medium e Massive constitui uma taxonomia interna adoptada para este protocolo experimental, inspirada em convenções de benchmarking de LLMs aplicados à engenharia de software que discriminam cenários por dimensão do contexto e complexidade semântica (por analogia com as categorias utilizadas no SWE-bench [50] e noutros estudos de code review assistido por IA [14], [15]). O critério de distinção assenta em duas dimensões mensuráveis: a dimensão do ficheiro-alvo (180 linhas nos testes Medium face a 450–720 linhas nos Massive) e a densidade de defeitos injetados (14 bugs nos Medium face a 20 nos Massive, com maior dispersão tipológica).

Os conjuntos Medium concentraram vulnerabilidades com assinatura textual direta — injeção SQL por concatenação, credenciais hardcoded, remoção de guards de segurança. Os conjuntos Massive introduziram duas dificuldades adicionais: bugs de validação omissa que nunca existiram no código (invisíveis como remoções no diff) e dispersão dos defeitos por ficheiros substancialmente maiores.

A variação dimensional entre os ficheiros-alvo (180–200 linhas nos Medium face a 450–720 linhas nos Massive) foi planeada de forma deliberada para forçar as condições propícias ao fenómeno Lost in the Middle [36] nos cenários de maior escala, estabelecendo uma variável independente robusta para testar a resiliência analítica das três arquiteturas.

A identificação específica dos Pull Requests submetidos no repositório Azure DevOps da DevScope, incluindo os respetivos identificadores únicos (PR ID), o número de bugs injetados e a densidade de critérios de aceitação violados, encontra-se registada na Tabela C.3 (Anexo C), garantindo a rastreabilidade completa dos artefactos originais e permitindo a auditoria posterior dos resultados.

A opção por testes construídos em vez de código de produção teve duas razões práticas. A primeira é óbvia: o código da DevScope é confidencial e usá-lo violaria o requisito R1 de soberania de dados. A segunda é mais relevante do ponto de vista metodológico. Com código de produção nunca auditado não existe ground truth verificável, o que torna impossível calcular taxas de falsos negativos de forma rigorosa. O protocolo adotado sacrifica validade externa (generalização para PRs arbitrários) em favor de validade interna (rigor na medição dos indicadores definidos). Esta limitação é reconhecida explicitamente na Secção 4.3.2.

A comparação entre três arquiteturas (Monolítica (A), Especializada Homogénea (B) e Multiagente (C)) segue um desenho experimental de tipo ablation study: cada arquitetura difere da anterior em exatamente um fator controlado (separação de agentes na transição A→B; heterogeneidade de modelos na transição B→C), permitindo isolar o contributo de cada decisão de design para o desempenho final.

4.2.2 Testes de Componentes

Arquitetura Monolítica (baseline). A primeira configuração avaliada utiliza um único agente Llama 3.3 70B responsável simultaneamente pelo code review e pela validação de requisitos. Os resultados estabelecem o piso de comparação e revelam os mecanismos precisos pelos quais a abordagem monolítica falha. Com aproximadamente 7.200 tokens de input contendo o ficheiro completo e os 12 ACs, o modelo recebe um contexto que excede a capacidade de atenção uniforme documentada no fenómeno Lost in the Middle [36]: a atenção concentra-se nos primeiros e últimos segmentos, ignorando informação crítica no meio do contexto. Nos seis testes, esta arquitetura detetou em média apenas 26% dos bugs injetados (26/102), com particular dificuldade em vulnerabilidades de severidade crítica (taxa aproximada de 38%).

A arquitetura monolítica revelou ainda dois modos de falha estruturais até aqui não documentados na literatura sobre code review assistido por LLMs. O primeiro, que designamos por AC Abandonment, manifestou-se no Massive 2: confrontado com 720 linhas de código e 12 critérios de aceitação, o modelo desistiu estruturalmente da auditoria, declarando na saída JSON que o Work Item "não tinha acceptance criteria" e inventando um critério fantasma para classificar como conforme. O segundo modo, que designamos por Conservadorismo Cego, ocorreu nos testes Medium 2, Medium 3 e Massive 3, onde o modelo classificou sistematicamente todos os critérios como PARTIAL sem indicar qualquer falha concreta, gerando ruído sem valor acionável para o programador.

A especialização de input elevou a deteção global para 44% (45/102), confirmando que a separação de responsabilidades é necessária. Contudo, a separação com o mesmo modelo não supera as limitações intrínsecas do Llama 3.3: incapacidade de raciocinar sobre remoções no diff (linhas prefixadas com − correspondentes à eliminação de guards de segurança são sistematicamente ignoradas em favor das linhas introduzidas) e dificuldade em detetar omissões lógicas que exigem raciocínio profundo sobre o estado esperado do código. Esta estagnação analítica — ganho marginal face ao custo duplicado de inferência — constituiu o principal motor empírico para a transição subsequente para a Arquitetura Multiagente com modelos heterogéneos.

Arquitetura Multiagente (o4-mini + DeepSeek-V3.2). A configuração final substitui o Llama pelo o4-mini no agente de code review (beneficiando do slow thinking sobre diffs curtos) e pelo DeepSeek-V3.2 no agente de requisitos (beneficiando da arquitetura Mixture-of-Experts para contextos extensos). A Tabela 12 consolida os resultados globais por severidade ao longo dos seis testes.

Tabela 12 - Deteção de vulnerabilidades nos seis testes por arquitetura e severidade (Elaboração própria)

Severidade	Bugs	Monolítico (A)	2× Llama (B)	Multiagente (C)
Crítica	26	10 (38%)	13 (50%)	26 (100%)
Alta	35	6 (17%)	16 (46%)	30 (86%)
Média	25	3 (12%)	4 (16%)	15 (60%)
Baixa	16	11 (69%)	16 (100%)	16 (100%)
Total	102	26 (25%)	45 (44%)	90 (88%)

A Tabela 12 revela três padrões distintos. As vulnerabilidades de severidade Baixa são detetadas por qualquer configuração, têm assinatura textual direta e não exigem raciocínio. Nas Críticas, a

progressão 38% → 50% → 100% mostra que chegar aos 100% não é uma questão de prompt engineering, mas de mudar para um modelo de raciocínio. Nas Médias, o salto de 16% para 60% confirma que omissões lógicas subtis, aquelas que não têm representação visível no diff, só são apanhadas com slow thinking.

A Figura 14 projeta visualmente a distribuição de eficácia analítica entre as três gerações do sistema, evidenciando que o ganho de desempenho proporcionado pela Arquitetura Multiagente não se distribui de forma linear, concentrando-se especificamente nos extremos de complexidade analítica.

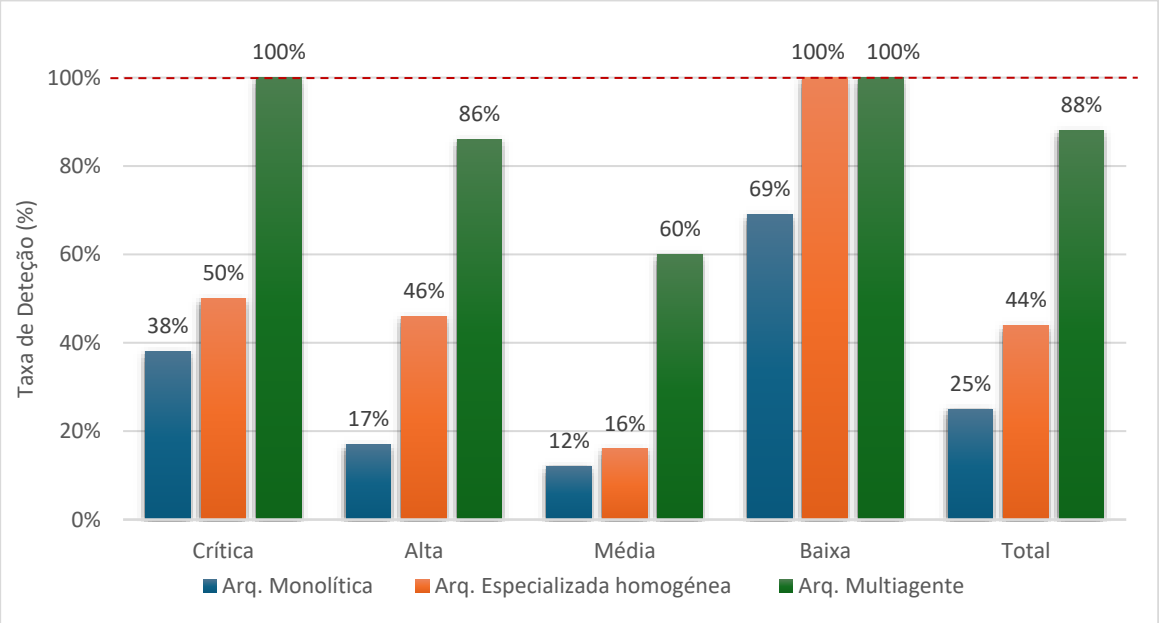


Figura 14 - Evolução da Taxa de Deteção por Severidade e Arquitetura (Teste Massive) (Elaboração própria)

A análise qualitativa dos 18 bugs classificados como "remoção de guards" (rollback de base de dados, validação HMAC de webhooks, verificação de roles, rate limiting) revela o diferenciador mais relevante entre arquiteturas: o Llama detetou apenas 11% destas regressões, enquanto o o4-mini alcançou 94%. No diff, estas falhas manifestam-se como linhas prefixadas com −; o Llama tende a centrar a análise nas linhas introduzidas (+), ignorando o significado semântico das linhas suprimidas. O o4-mini, através do seu mecanismo de raciocínio interno, constrói uma representação do estado anterior e do estado atual do código, detetando a diferença como uma regressão de segurança.

A Tabela 13 quantifica a degradação de desempenho registada na transição dos ambientes Medium para os Massive, discriminando a resiliência de cada arquitetura ao aumento dimensional do código sob análise.

Tabela 13 - Evolução da taxa de deteção entre testes Medium e Massive (Elaboração própria)

Arquitetura	Média Medium	Média Massive	Variação

Monolítica	31%	22%	-9 p.p
Especializada homogénea (2× Llama)	58%	35%	−23 p.p.
Multiagente	90%	84%	−6 p.p.

A degradação da Arquitetura Especializada Homogénea (−23 p.p.) é quase quatro vezes superior à observada na Arquitetura Multiagente (−6 p.p.), sugerindo que o mecanismo de raciocínio interno do o4-mini confere resiliência substancial ao aumento de complexidade do diff. Os tokens de thinking gerados internamente nos testes Massive (em média 3.500–4.500 por execução) representam o custo cognitivo desta análise profunda: o modelo raciocina sobre o que deveria estar presente no código com base em padrões de segurança, identifica omissões por contraste com o estado esperado, e formula o finding apenas depois de completar esse raciocínio estruturado.

Validação de requisitos funcionais. A avaliação da conformidade face aos Acceptance Criteria exigiu a monitorização de vetores de precisão semântica distintos. A Tabela 14 sistematiza a performance comparativa dos agentes de requisitos sob quatro eixos: densidade de violações injetadas, rácio de identificação efetiva, incidência de falsos negativos e acuidade taxonómica da classificação.

Tabela 14 - Falsos negativos na validação de requisitos por arquitetura (Elaboração própria)

Teste	ACs violados (real)	Monolítico (FN)	2× Llama (FN)	Multiagente (FN)
Medium 1	9	4	1	1
Medium 2	11	0[5]	6	1
Medium 3	11	0	1	3
Massive 1	8	1	1	1
Massive 2	11	11	11	2
Massive 3	12	0	9	0

[5] Valor subestima o problema real: o Monolítico classifica sistematicamente todos os ACs como PARTIAL sem indicar falhas concretas, gerando ruído sem valor acionável.

Total	62	16 (26%)	29 (47%)	8 (13%)

A diferença mais significativa entre o Llama e o DeepSeek não reside na taxa de deteção bruta, mas na precisão dos diagnósticos. O Llama tende a classificar como PARTIAL requisitos que o DeepSeek classifica como MISSING, distinção com impacto direto na acionabilidade do finding para o programador, dado que MISSING indica ausência total de implementação enquanto PARTIAL sugere implementação incompleta.

Falso negativo estrutural partilhado (AC-04 — Paginação). Em todos os testes onde este critério foi avaliado, a função list_user_projects inclui pagination: PaginationParams na assinatura, criando aparência de conformidade. Contudo, a query SQL no corpo da função não utiliza os campos LIMIT e OFFSET do objeto de paginação. Tanto o Llama como o DeepSeek classificaram este AC como conforme ou parcial, sem sinalizar a falha concreta. A persistência deste falso negativo em ambos os modelos confirma que a limitação não é do modelo específico, mas do paradigma de avaliação por ficheiro completo: qualquer LLM que avalie conformidade por leitura estática de código está sujeito a ser enganado por implementações aparentes. A deteção de omissões no corpo exige rastreamento do fluxo de dados, capacidade que fundamenta a integração do Tree-sitter como trabalho futuro (Secção 5.2).

A análise cruzada dos resultados evidencia que as duas camadas de avaliação são genuinamente complementares: não se trata de redundância com reformulação diferente, mas de perspetivas analíticas que cobrem domínios estruturalmente distintos, como evidenciado na Tabela 15.

Tabela 15 - Complementaridade entre análise estática e revisão semântica por LLM (Elaboração própria)

Tipo de Problema	Análise Estática	Revisão por LLM (diff)	Revisão por LLM (ficheiro)
Credenciais hardcoded (padrões óbvios)	Excelente	Inconsistente	Inconsistente
Remoção de guardas de segurança	Impossível[6]	Forte (o4-mini)	Impossível
Omissões de validação sem referência no diff	Impossível	Impossível	Forte
Violações de requisitos funcionais	Impossível	Fraco	Excelente

[6] A célula "Impossível" não reflete uma limitação de implementação, mas uma limitação estrutural do método.

SQL Injection subtil	Limitado	Forte	Forte

Dois padrões emergem claramente: os bugs de remoção de guards (Massive 1, 2, 3) são detetados pelo code review mas não pelo requirements review; os bugs de omissão de validação (Medium 1, 2, 3) são detetados pelo requirements review mas não pelo code review. A sobreposição é mínima, confirmando que a separação de responsabilidades da arquitetura multiagente gera cobertura complementar, não redundância cognitiva. A análise estática nunca poderá detetar a remoção de guards de segurança porque opera sobre padrões textuais e não sobre semântica de mudança; o requirements review por ficheiro completo nunca poderá detetar remoções porque o ficheiro atual não as exibe. A complementaridade das três camadas não é, portanto, uma otimização, é uma necessidade arquitetural. A consolidação formal das métricas de classificação binária (Verdadeiros Positivos, Falsos Positivos, Falsos Negativos) e o cálculo das métricas derivadas (Precisão, Revocação e F1-Score) para as três arquiteturas avaliadas encontram-se sistematizados na Tabela C.1 (Anexo C).

O sistema implementa seis mecanismos de higienização aplicados sequencialmente, conforme ilustrado na Figura 15, com o objetivo de reduzir o consumo de tokens e a latência sem comprometer a qualidade analítica:

- **Semantic Triage Gate**: descarta ficheiros não-executáveis (.md, .json, .lock) antes de qualquer invocação de IA;
- **Diff-Only (Code Review)**: o o4-mini recebe exclusivamente as linhas alteradas em vez do ficheiro completo;
- **AST Skeleton**: comprime o ficheiro de 565 para 104 linhas (−82%) como contexto estrutural auxiliar;
- **Knowledge Ledger**: os findings da Fase 1 são mapeados para NFRs pré-verificados, eliminando a redundância cognitiva no agente de requisitos;
- **AC-Aware Context Pruning**: extrai termos-chave dos ACs e remove do ficheiro completo todo o código que não os contenha, retendo apenas as secções relevantes para a validação funcional;
- **Finding Context Stripping**: remove dos findings injetados no prompt do DeepSeek os campos `VULNERABLE_CODE`, `FIXED_CODE` e `SUGGESTION_CODE`, passando apenas metadados.

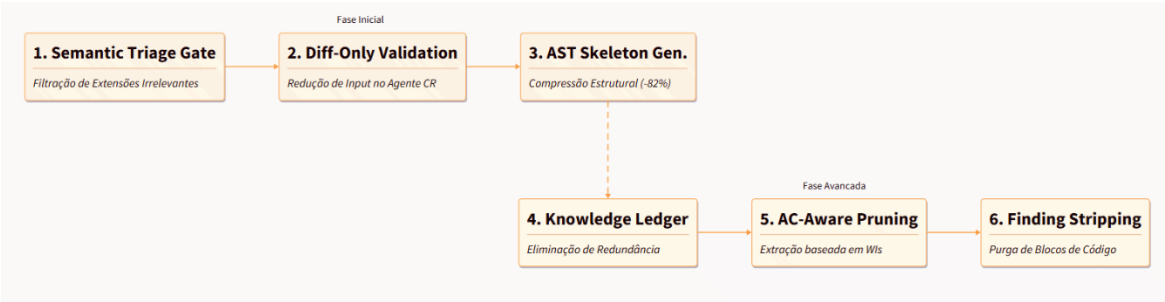


Figura 15 - Pipeline de Higienização de Contexto e redução sequencial de tokens (Elaboração própria)

A avaliação do impacto foi conduzida sobre o ficheiro user_manager.py (~565 linhas, 25 ACs), comparando duas execuções idênticas antes e depois da ativação dos mecanismos (Tabela 16).

Tabela 16 - Impacto dos mecanismos de higienização (Elaboração própria)

Métricas	Antes	Depois	Variação
Tokens (Requirements)	13.722	6.347	-53,8%
Tokens (Code Review)	6.568	6.328	-3,7%
Total tokens	20.290	12.675	-37,5%
Tempo (Requirements)	98,8 s	26,2 s	-73,5%
Tempo (Code Review)	28,7 s	29,1 s	~0%
Total tempo	127,5 s	55,3 s	-56,6%

A poupança concentra-se no agente de requisitos (DeepSeek-V3.2): −53,8% nos tokens e −73,5% no tempo de execução. O agente de code review (o4-mini) mantém-se estável (−3,7%), comportamento esperado dado que a compressão Diff-Only já estava implementada na linha de base.

O efeito mais relevante não é a poupança de tokens em si, mas o impacto na latência total: sem higienização, o pipeline demora 127,5 segundos, violando o requisito não funcional RNF04 (limiar de 90 segundos). Com higienização, a latência desce para 55,3 segundos, cumprindo o requisito com margem. A higienização é, portanto, uma condição necessária para a viabilidade operacional do sistema, não apenas uma otimização de custo.

Trade-off quantificado: O mecanismo de AC-Aware Context Pruning introduziu um falso negativo no AC-05 (gdpr_request_id), cuja linha de implementação foi removida durante a poda por o termo-chave não ter sido extraído dos ACs. Este trade-off é mensurável: a higienização elimina −53,8% dos tokens de input do agente de requisitos e recupera 41,7% da margem de latência, ao custo de introduzir potencialmente 1 falso negativo por cada caso de dissonância semântica entre o vocabulário dos ACs e os identificadores no código. A aceitabilidade deste trade-off depende do contexto: em equipas onde os ACs e o código partilham o mesmo idioma e vocabulário, o risco é negligenciável; em equipas multilíngues ou com ACs em linguagem natural informal, o risco requer mitigação (Secção 4.2).

A tradução dos ganhos volumétricos de contexto para indicadores de viabilidade financeira requer a aplicação dos vetores de preço modelados. A Tabela 17 decompõe analiticamente a poupança económica estimada por Pull Request, comprovando de forma matemática que a eficiência financeira se concentra predominantemente no agente de requisitos, padrão perfeitamente convergente com a compressão de tokens documentada na Tabela 16. Aplicando os preços de referência (Tabela 4) ao diferencial de tokens medido, e considerando que a poupança de output acompanha proporcionalmente a de input no agente de requisitos:

Tabela 17 - Poupança estimada por PR com higienização (Elaboração própria)

Componente	Antes (estimado)	Depois (estimado)	Poupança
DeepSeek — Input	~€0,008	~0,004	-50,0%
DeepSeek — Output	~€0,003	~0,002	~-33,3%
o4-mini — Input+Output	~€0,020	~0,018	-10,0%
Total por PR	~€0,031	~€0,024	~-22,6%

A poupança económica direta situa-se em aproximadamente 22,6% do custo por PR, com um impacto muito mais expressivo na latência (−56,6%). Para uma equipa com 60 PRs/mês, a higienização poupa aproximadamente €0,36/mês em custos de inferência. Modesto em valor absoluto, mas relevante em contexto de escala e, sobretudo, indispensável para cumprir os requisitos de latência.

4.3 Avaliação da Solução

4.3.1 Análise de Custos Operacionais

Os custos foram calculados com base no consumo real de tokens recolhido nas execuções dos seis testes de benchmark, aplicando os preços de referência: o4-mini a ~$1,10/1M input e ~$4,40/1M output (incluindo thinking); DeepSeek-V3.2 a ~$0,58/1M input e ~$1,68/1M output; Llama 3.3 70B a ~$0,71/1M input e output.

Tabela 18 - Custo estimado por PR por arquitetura e teste (Elaboração própria)

Arquitetura	Média Medium (M1-M3)	Média Massive (Mv1-Mv3)	Média Global
Monolítico - Llama	~€0,006	~€0,010	~€0,008
Especializada Homogénea (2× Llama)	~€0,006	~€0,011	~€0,009
Multiagente — o4-mini + DeepSeek	~€0,027	~€0,058	~€0,043

Um resultado contraintuitivo emerge da comparação entre a Arquitetura Monolítica e a Especializada Homogénea: apesar de utilizar o mesmo modelo base (Llama 3.3 70B), a Arquitetura Monolítica apresenta custo médio ligeiramente inferior (~€0,008 vs ~€0,009), validando que a especialização de input da Arquitetura B reduz a redundância de output, mas ao custo de uma chamada adicional à API. A explicação reside na ausência de partição de contexto: na configuração monolítica, um único prompt agrega o ficheiro completo, os 12 critérios de aceitação, o diff e instruções de ambas as dimensões analíticas, gerando redundância de contexto e obrigando o modelo a produzir um JSON massivo que cubra simultaneamente findings de segurança e matriz de conformidade funcional. Na arquitetura especializada, o contexto é particionado por agente, ou seja, o Code Review recebe apenas o diff, o Requirements Review recebe apenas o ficheiro e os ACs eliminando redundância e permitindo outputs mais compactos por chamada. Este resultado valida empiricamente um princípio da arquitetura multiagente: a especialização de input produz eficiência económica mesmo sem alterar o modelo subjacente.

O diferencial de custo entre as Arquiteturas B e C (aproximadamente 5× a 7×, conforme a dimensão do PR) tem origem inequívoca nos tokens de thinking do o4-mini: este modelo cobra os tokens de raciocínio

interno à mesma taxa dos tokens de output visíveis. O custo adicional é, portanto, o preço explícito das capacidades analíticas superiores documentadas na Secção 4.2.

O custo médio de ~€0,043/PR para a Arquitetura Multiagente foi validado pelo custo acumulado real registado na faturação do Azure AI Foundry ao longo dos 145 Pull Requests submetidos durante todo o ciclo de desenvolvimento. Esta validação cruzada entre o custo calculado por consumo de tokens e o custo real faturado confirma a precisão do modelo de estimativa adotado.

Dois dos seis testes Massive (CatalogAPI e OrderService) ultrapassaram pontualmente o teto orçamental de €0,05/PR definido no RNF02, atingindo respetivamente €0,058 e €0,065. Esta violação pontual é atribuível à dimensão excepcional dos ficheiros (~700 e ~450 linhas) conjugada com a densidade de vulnerabilidades injetadas (20 em cada teste), cenário que excede a dimensão típica de um PR de produção.

Um PR de produção típico é, em dimensão, semelhante aos testes Medium: alterações focadas num ou dois ficheiros, com diffs de baixa a média complexidade. Tomando ~€0,025–€0,030 como intervalo de referência para a Arquitetura Multiagente em PRs típicos, a Tabela 19 projeta o impacto mensal por dimensão de equipa.

Tabela 19 - Projeção de custo operacional mensal (Elaboração própria)

Cadência de PRs	Arq. Especializada (~€0,008)	Arq. Multiagente (~€0,030)	Diferencial
30 PRs/mês (equipa pequena)	~€0,24/mês	~€0,90/mês	+€0,66/mês
60 PRs/mês (equipa média)	~€0,48/mês	~€1,80/mês	+€1,32/mês
120 PRs/mês (equipa média)	~€0,96/mês	~€3,60/mês	+€2,64/mês

Estes valores posicionam a solução a uma fração do custo das alternativas comerciais identificadas no Capítulo 2 (CodeRabbit Enterprise: ~€165.000/ano; SonarQube Enterprise: ~€18.000/ano). Para qualquer dimensão de equipa, o diferencial entre as Arquiteturas B e C é irrelevante face ao custo potencial de uma vulnerabilidade crítica aprovada em produção.

A consolidação das métricas de eficácia analítica face ao impacto financeiro permite avaliar a sustentabilidade operacional da solução em ambiente industrial.

O registo cronológico completo das dezoito execuções do protocolo experimental, discriminando latência de ponta a ponta, consumo de tokens de entrada e saída, e custo operacional calculado com os

preços de referência da Tabela 4, encontra-se detalhado na Tabela C.2 (Anexo C). A nota metodológica que acompanha essa tabela documenta o protocolo de extrapolação aplicado a três execuções onde o registo de tokens foi interrompido por timeout da API.

A Figura 16 posiciona as três arquiteturas avaliadas num plano bidimensional custo-qualidade, tornando evidente que a relação entre as variáveis não assume um comportamento linear: o salto qualitativo na taxa de deteção verificado na transição da Arquitetura Especializada para a Multiagente é desproporcionalmente superior ao incremento financeiro marginal.

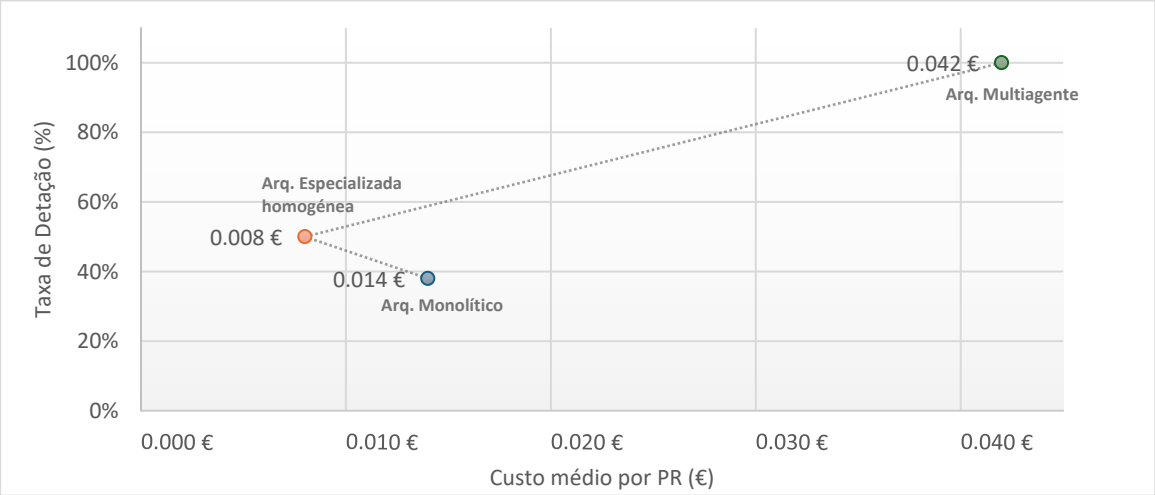


Figura 16 - Rácio Qualidade/Custo: % Deteção de Bugs Críticos vs. Custo por PR (€) (Elaboração própria)

Esta distribuição não linear confirma que a Arquitetura Multiagente, embora represente o maior investimento nominal por submissão, estabelece o único ponto de operação tolerável para um ecossistema de governança corporativa, visto que a tolerância a falhas de severidade crítica em produção é nula. O acréscimo de custo associado aos tokens de thinking é, portanto, integralmente justificado pela eliminação do risco de aprovação de vulnerabilidades severas.

4.3.2 Síntese Comparativa e Limitações

A análise transversal dos seis testes permite extrair quatro conclusões fundamentais sobre o comportamento das três arquiteturas avaliadas:

- **Conclusão 1:** O modelo de raciocínio é o diferenciador crítico. A diferença de 44 pontos percentuais entre as Arquiteturas B e C (44% → 88%) não resulta de prompts ou configurações distintas, mas da capacidade do o4-mini de realizar slow thinking antes de produzir o finding. Este resultado valida H1, com latência média de ~79,7 segundos, dentro do limiar de 90 segundos definido em RNF04, garantindo margem de segurança de 19,3%.
- **Conclusão 2**: A separação de agentes é necessária, mas não suficiente. A transição da Arquitetura Monolítica para a Especializada Homogénea eleva a deteção global de code review

de 25% para 44%, confirmando que o input especializado (diff vs. ficheiro completo) é determinante. Contudo, a separação com o mesmo modelo não supera as limitações intrínsecas do Llama 3.3: incapacidade de raciocinar sobre remoções no diff e de detetar omissões sem raciocínio profundo. Este resultado valida parcialmente H2, demonstrando que a especialização de input é necessária, mas insuficiente.

- **Conclusão 3:** O requirements review por ficheiro completo tem um ponto cego estrutural. A análise semântica é vulnerável a implementações aparentes: assinaturas de função conformes que ocultam corpos não conformes. Este fenómeno, que designamos por Ilusão da Assinatura, manifestou-se consistentemente em todos os testes onde o AC de paginação foi avaliado (M1, M2, M3, Mv1) — tanto o Llama como o DeepSeek classificaram o AC como conforme porque a função aceitava pagination: PaginationParams na assinatura, apesar da query SQL no corpo não utilizar LIMIT/OFFSET. A complementaridade entre code review por diff e requirements review por ficheiro completo não é uma opção de design, é uma necessidade metodológica. Esta limitação estrutural será endereçada em trabalho futuro através de Tree-sitter e análise de fluxo de dados (Secção 5.2).
- **Conclusão 4:** A Arquitetura Monolítica apresenta modos de falha estruturais não documentados na literatura. A avaliação dos seis testes revelou dois fenómenos até aqui não reportados em estudos sobre code review assistido por LLMs.
    - AC Abandonment (observado no Massive 2): confrontado com 720 linhas de código e 12 critérios de aceitação, o modelo desistiu estruturalmente da auditoria, declarando na saída JSON que o Work Item "não tinha acceptance criteria" e inventando um critério fantasma para classificar como conforme. Este fenómeno representa uma recusa estrutural de auditoria que passa despercebida ao revisor humano.
    - Conservadorismo Cego (observado nos testes Medium 2, Medium 3 e Massive 3): o modelo classificou sistematicamente todos os critérios como PARTIAL sem indicar qualquer falha concreta, gerando ruído sem valor acionável para o programador. À primeira vista parece rigoroso (zero Falsos Negativos formais), mas na prática equivale a um revisor que diz "tudo precisa de melhorias" sem apontar uma única falha específica.

Em síntese, a validação empírica confirma a hipótese H1 (latência média de 79,7s, inferior ao limite de 90s de RNF04) e valida a hipótese H2 (Recall global de 88% e eficácia de 100% em vulnerabilidades críticas — 26 de 26 detetadas).

Abordando as Limitações que devem ser consideradas na interpretação dos resultados:

- **Conjunto de testes construído:** Todos os 102 defeitos foram introduzidos pelo autor, com tipologia e localização previamente conhecidas. As taxas de deteção reportadas não devem ser extrapoladas como garantia de desempenho universal em PRs de produção arbitrários.
- **Ausência de benchmark contra ferramentas comerciais:** A avaliação não incluiu uma comparação direta com CodeRabbit ou Amazon CodeGuru sobre os mesmos PRs de teste.
- **Ausência de avaliação por utilizadores reais:** A métrica Human Acceptance Rate (HAR), proporção de comentários automáticos resolvidos pelos programadores, referenciada na literatura como o indicador de eficácia real mais relevante, não pôde ser apurada. A avaliação da qualidade dos diagnósticos limitou-se à análise manual pelo autor.
- **Contexto de subscrição académica:** As restrições da subscrição Azure for Students limitaram os modelos avaliáveis, inviabilizando a família Qwen2.5-72B-Instruct, identificada na Secção 2.4 como a opção tecnicamente mais adequada para a tarefa de code review.
- **Dissonância semântica no AC-Aware Context Pruning:** Quando os critérios de aceitação estão redigidos em português e os identificadores no código estão em inglês (ex.: "fatura" vs. Invoice), o algoritmo de correspondência lexical pode remover código relevante durante a poda. Futuras iterações exigem mapeamento semântico por embeddings vetoriais.
- **Escalabilidade não validada em carga:** O comportamento do sistema sob tráfego concorrente (múltiplos PRs simultâneos, saturação da quota de API, comportamento do Redis distribuído) permanece por verificar empiricamente.
- **Ilusão da Assinatura:** A avaliação por leitura estática de ficheiro completo é estruturalmente vulnerável a assinaturas conformes que ocultam corpos não conformes. A deteção de omissões no corpo exige rastreamento do fluxo de dados (dataflow analysis), capacidade que fundamenta a integração do Tree-sitter como trabalho futuro (Secção 5.2).

A transparência na identificação destas limitações não invalida os resultados obtidos, restringe o âmbito da sua interpretação. Os resultados são válidos e rigorosos dentro do protocolo definido; a sua generalização para contextos mais amplos requer as extensões documentadas na Secção 5.2.

4.3.3 Ameaças à Validade

A interpretação dos resultados obtidos deve ser enquadrada à luz das ameaças à validade habitualmente taxonomizadas na investigação empírica em Engenharia de Software [3], [13]:

Validade Interna. O risco mais evidente é o viés de confirmação: o mesmo autor concebeu, implementou e avaliou o sistema, e injetou os 102 defeitos cujas localizações conhecia de antemão. Para reduzir este efeito, os defeitos foram distribuídos por dois níveis de dimensão (Medium/Massive)

e dois domínios de negócio diferentes. A comparação entre arquiteturas seguiu um protocolo de ablation study. Em cada transição, apenas uma variável muda, o que permite isolar o contributo de cada decisão de design.

Validade Externa. Os seis conjuntos de teste construídos não garantem a generalização dos resultados para Pull Requests de produção arbitrários, que tipicamente apresentam lógica de domínio obscura, dependências cruzadas e contextos históricos extensos. Os valores de Recall reportados (88% global, 100% críticos) são rigorosamente válidos dentro do protocolo controlado definido; a sua extrapolação para cenários de produção carece da validação empírica complementar proposta na Secção 5.2 (integração CI/CD como Quality Gate com medição de HAR).

Validade de Construto. A métrica de Custo Operacional depende dos preços de catálogo do Azure AI Foundry em março de 2026, estando sujeita à volatilidade comercial da plataforma MaaS. A métrica de Latência está dependente da carga da infraestrutura partilhada do Azure AI Foundry no momento de cada execução, razão pela qual os valores reportados correspondem a médias de múltiplas execuções e não a medições pontuais. A métrica de Revocação assume a completude do ground truth (102 defeitos catalogados), podendo subestimar falsos negativos em cenários onde o código contenha vulnerabilidades não antecipadas pelo autor.

Validade de Conclusão. Com 102 defeitos em 6 Pull Requests, o IC 95% para a taxa de deteção global (88%) situa-se entre 81,7% e 94,3% pela aproximação de Wilson. Para as vulnerabilidades críticas (N=26, deteção de 100%), o limite inferior do IC 95% por Clopper-Pearson é 86,8%, o que suporta estatisticamente os resultados mesmo com uma amostra de dimensão moderada. Mais relevante é o facto de os três modos de falha identificados (AC Abandonment, Conservadorismo Cego e Ilusão da Assinatura) terem aparecido de forma consistente em vários modelos e várias execuções. Isso sugere que são limitações estruturais do paradigma de avaliação por LLM, e não ruído da amostra

5 Conclusões e Trabalho Futuro

5.1 Objetivos Concretizados

O ponto de partida deste estágio foi uma lacuna clara no ecossistema Azure DevOps: não existia nenhuma ferramenta de revisão assistida por IA que garantisse simultaneamente soberania de dados, custo operacional controlado e integração nativa na plataforma. O trabalho desenvolvido resultou numa Prova de Conceito que responde a essas três exigências e foi avaliada com rigor experimental.

Como ilustrado na Figura 17, o percurso não seguiu um plano linear: cada limitação encontrada — desde o contrato de 30 segundos impostos pelos Webhooks, à truncagem de JSON em modelos

verbosos, até à inconsistência dos veredictos LLM em contextos empresariais — motivou uma decisão de engenharia que moldou a arquitetura seguinte. O resultado é um sistema multiagente com Ports & Adapters que orquestra o o4-mini (raciocínio profundo sobre diffs) e o DeepSeek-V3.2 (análise semântica de contextos extensos), complementado por um motor de análise estática determinístico e por uma camada de governança que remove a autoridade de decisão do modelo probabilístico e a transfere para regras de domínio auditáveis.

O esforço experimental materializou-se em 145 Pull Requests submetidos ao longo do ciclo de desenvolvimento, dos quais seis foram consolidados como benchmark final controlado com 102 defeitos catalogados. Este volume de testes permitiu isolar variáveis de complexidade, dimensão de ficheiro e tipologia de vulnerabilidade de forma rigorosa.

O custo acumulado de todo o ciclo de desenvolvimento, desde a validação inicial com o Phi-4-mini, passando pela experimentação com o Llama 3.3 70B, até ao benchmark final, foi de €1,56, com uma pequena parte concentrada nos seis testes de benchmark (cerca de €0,36). Mais do que um dado de eficiência, este número demonstra que é possível construir e avaliar um sistema de revisão de código com IA empresarial e soberana a custos controlados.

A Tabela 20 sistematiza o grau de cumprimento de cada objetivo específico definido na Secção 1.3, apresentando as decisões técnicas e os resultados que sustentam cada avaliação.

Tabela 20 - Síntese do cumprimento dos objetivos propostos (Elaboração própria)

Objetivo	Evidência de Cumprimento
Análise de Código Soberana	Integração assíncrona estabilizada via webhooks no Azure DevOps. O requisito arquitetural R1 foi cumprido: 100% do processamento e inferência ocorreu no perímetro do Azure AI Foundry, sem exfiltração de dados para APIs de terceiros.
Auditoria Técnica Bidimensional	A Arquitetura Multiagente alcançou um Recall global de 88% e intercetou 100% das 26 vulnerabilidades críticas, superando o critério de sucesso (>80%) e validando a hipótese H2.
Validação de Conformidade Funcional	Implementação do Agente de Requisitos (DeepSeek-V3.2) suportado pelo motor de AC-Aware Context Pruning. O sistema cruzou as alterações com os Work Items, identificando 87% das violações de critérios de aceitação (Tabela 14) e gerando relatórios de impacto no negócio.

Sustentabilidade Operacional	Custo operacional estabilizado em ~€0,043 por submissão (média), cumprindo o critério de sucesso de <€0,05/PR em quatro dos seis testes de benchmark. A latência média de 79,7s cumpriu o teto de 90s (RNF04) com 19,3% de margem.

Os quatro objetivos definidos no início do projeto foram cumpridos. O custo médio de €0,043 por PR ficou abaixo do teto de €0,05 estabelecido no RNF02, o que confirma a viabilidade económica para integração nos pipelines da DevScope. Os dois testes Massive que ultrapassaram pontualmente esse limite (CatalogAPI (€0,058) e OrderService (€0,065)) tinham ficheiros de ~700 e ~450 linhas respetivamente, uma dimensão que excede claramente a de um PR de produção típico.

Os resultados confirmam as hipóteses H1 e H2 com os qualificativos detalhados em 4.3.2: o modelo de raciocínio é o diferenciador crítico em code review, e a separação diff/ficheiro-completo é uma necessidade metodológica, não de design. Sob a perspetiva do posicionamento competitivo face ao Estado da Arte analisado no Capítulo 2, a avaliação multivariável das ferramentas comerciais revela as limitações inerentes às soluções de prateleira.

A Figura 17 sintetiza esta matriz competitiva através de um mapeamento, no qual o ScopeReview AI é confrontado com as principais alternativas de mercado (GitHub Copilot PR, CodeRabbit Pro, Qodo Merge Pro e SonarQube + IA).

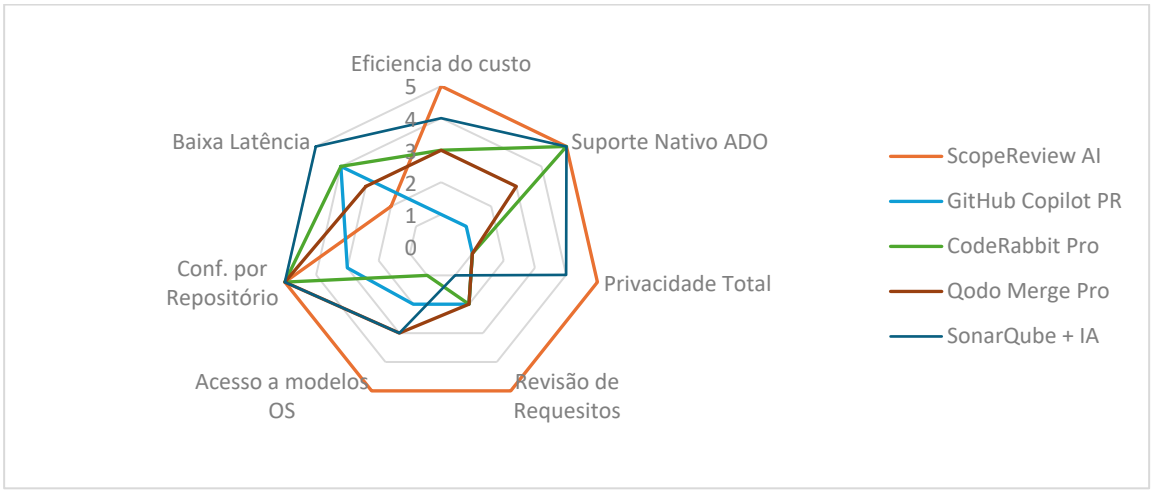


Figura 17 - Posicionamento competitivo do ScopeReview AI face às alternativas comerciais (Elaboração própria)

Uma nota sobre a matriz: as pontuações (escala 1–5) resultam da análise documental das funcionalidades públicas, modelos de licenciamento e políticas de privacidade de cada fornecedor até março de 2026, cruzadas com o levantamento da Secção 2.2. Não é um benchmark sobre os mesmos PRs, é uma comparação das capacidades arquiteturais de cada ferramenta face às necessidades específicas da DevScope.

O ScopeReview AI atinge a pontuação máxima em cinco dos sete eixos avaliados. O compromisso assumido concentra-se na latência: os ~79,7 segundos de execução média, embora inferiores ao teto de 90 segundos definido em RNF04, situam-se acima dos 15–30 segundos das alternativas baseadas em análise estática superficial ou sumarização sem raciocínio profundo. Trata-se de um trade-off deliberado e quantificado no Capítulo 4: o custo cognitivo do slow thinking do o4-mini e da validação determinística de veredictos traduz-se em segundos adicionais aceites em troca de 100% de deteção em vulnerabilidades críticas.

Nos três eixos com vantagem mais acentuada (Privacidade Total, Revisão de Requisitos e Acesso a Modelos Open-Source) a diferença face à concorrência não é de grau: é de presença ou ausência da funcionalidade. O ScopeReview AI corre integralmente sobre o Azure AI Foundry, com isolamento organizacional documentado pela Microsoft [11]. As alternativas disponíveis para PMEs processam código em cloud pública com políticas de retenção pouco transparentes, um risco quantificado por Al-Maamari [8] e que se materializou na fuga de código da Samsung em 2023.

A Configuração por Repositório é partilhada com CodeRabbit e SonarQube através da leitura dinâmica de ficheiros locais (.codereview.yml, .coderabbit.yml), permitindo a cada equipa ajustar regras sem intervenção central. No eixo da Eficiência de Custo, o SonarQube Developer Edition (~€150/ano em self-host) atinge pontuação próxima, mas carece das capacidades de validação semântica de requisitos que distinguem o ScopeReview AI. Os planos Enterprise do CodeRabbit e do Qodo resolveriam parcialmente a questão da privacidade, mas exigem mínimos de 500 licenças incompatíveis com a realidade da organização.

Para além dos objetivos operacionais sistematizados na Tabela 20, a validação empírica do projeto deve ser avaliada face às hipóteses e ao requisito arquitetural formulados no Capítulo 1. A Tabela 21 apresenta esta síntese, incluindo intervalos de confiança de 95% que quantificam a robustez estatística dos resultados.

Tabela 21 - Verificação das Hipóteses e Requisito Arquitetural (Elaboração própria)

Hipótese	Formulação (Cap. 1)	Resultado Obtido	Veredicto
R1	100% processamento em perímetro Azure AI Foundry	100% (verificado por inspeção de logs e faturação)	Cumprido
H1	Latência média < 90 segundos	79,7 s (IC 95%: 72,1–87,3 s)	Validada (margem de 11,4%)

H2	Recall global > 80% com 100% em críticos	88,0% global [IC 81,7%–94,3%] e 100% críticos [IC 86,8%–100%]	Validada

5.2 Limitações e Trabalho Futuro

As limitações metodológicas e as ameaças à validade estão detalhadas na Secção 4.3.3. Para além dessas, a avaliação empírica do Capítulo 4 expôs três fenómenos que não tinham sido antecipados: o AC Abandonment, o Conservadorismo Cego e a Ilusão da Assinatura. Os três mostram que a avaliação puramente semântica é vulnerável a implementações aparentes e à fadiga de contexto em ficheiros grandes. A base arquitetural é sólida, mas levar o sistema a produção real exige resolver quatro limitações concretas.

A limitação mais crítica é a cegueira do Context Pruner ao fluxo de dados real. O modelo consegue ver que um parâmetro existe na assinatura, mas não consegue verificar se é efetivamente usado no corpo da função. A solução passa por substituir a heurística lexical atual por Static Program Slicing com Code Property Graphs (CPG): mapear a AST e os fluxos de controlo e dados, e fazer uma travessia de grafo a partir do diff para extrair deterministicamente o rastro de execução impactado. Isto elimina o ruído semântico, reduz tokens e permite ao LLM auditar o fluxo de dados de ponta a ponta.

A segunda limitação é o suporte exclusivo a Python. Os parsers e expressões regulares actuais foram construídos especificamente para esta linguagem, o que restringe a adoção da ferramenta a equipas com stack homogénea. A migração para o Tree-sitter, um parser incremental com gramáticas formais para mais de 40 linguagens, resolve isto: o Triage Gate e o Context Pruner passam a ter rigor sintático universal sem tocar no Core de Domínio, e o sistema fica disponível para qualquer stack tecnológica da organização.

O sistema está actualmente acoplado ao Azure DevOps. A extração de diffs, a leitura de Work Items e a publicação de comentários dependem todos da API da Microsoft, o que exclui equipas que usem GitHub ou GitLab. O padrão Ports & Adapters já implementado facilita esta extensão: basta desenvolver novos adaptadores (GitHubClient, GitLabClient) que respeitem o contrato da porta IRepositoryClient. O motor de orquestração permanece inalterado e o sistema deixa de estar preso a um único ecossistema.

A última limitação é mais subtil: o sistema é amnésico. Quando um programador rejeita um falso positivo, esse sinal desaparece. Nada é aprendido, e o mesmo erro repete-se. Integrar o ScopeReview AI como Quality Gate em CI/CD permitirá medir a Human Acceptance Rate (HAR) em produção, e um Feedback Loop determinístico persistirá essas rejeições no Knowledge Ledger como regras de domínio.

Com o tempo, o sistema calibra-se ao estilo e à cultura de cada repositório, o que é decisivo para a confiança das equipas.

5.3 Apreciação Final

O estágio na DevScope foi, acima de tudo, um exercício de engenharia de realidade: a distância entre o que os LLMs prometem e o que entregam de forma fiável num contexto empresarial exigiu a construção de uma camada completa de governança determinística para domesticar a sua natureza probabilística. A principal aprendizagem do estágio residiu na separação da IA como extratora semântica da lógica como autoridade de decisão, um princípio que a literatura emergente começa a formalizar [24], [25] e que este projeto validou empiricamente com dados de faturação reais.

A autonomia técnica concedida pela organização, aliada ao suporte infraestrutural do Azure, permitiu iterar desde o protótipo monolítico até à configuração multiagente final, recolhendo dados reais de custo, latência e taxa de deteção ao longo de 145 Pull Requests. Os 102 bugs catalogados nos seis testes de benchmark revelaram não apenas a eficácia da solução (88% de deteção global, 100% de críticos), mas também limitações estruturais dos paradigmas atuais de avaliação por LLM, designadamente a cegueira a remoções de guards, o AC Abandonment em contextos extensos, e a Ilusão da Assinatura em validações de conformidade funcional.

O ScopeReview AI constitui, neste contexto, uma base arquitetural validada com trajetória definida para integração em produção, condicionada às extensões de maturidade técnica e validação empírica detalhadas na secção anterior. Os três fenómenos estruturais identificados durante a avaliação empírica não invalidam os resultados obtidos; pelo contrário, definem com precisão a fronteira do conhecimento atual e apontam direções concretas para investigação futura no domínio da revisão de código assistida por IA.

Do ponto de vista organizacional, a maior lição deste estágio foi que o obstáculo mais difícil à adoção de IA na engenharia de software não é técnico, mas sim cultural. A arquitetura mais sofisticada falha se os programadores seniores a tratarem como um bot que gera ruído e bloqueia os seus merges. Passar de Prova de Conceito a ferramenta de produção vai exigir um investimento paralelo em literacia: as equipas precisam de perceber que o sistema funciona como um revisor júnior incansável que requer supervisão humana, não como um árbitro infalível.

A relevância deste trabalho vai além da redução do lead time. Num contexto europeu em que as exigências do RGPD e da proteção de propriedade intelectual são cada vez mais sérias, auditar código com IA sem enviar esse código para fora do perímetro organizacional deixou de ser uma escolha arquitetural e passou a ser uma necessidade de conformidade. O ScopeReview AI demonstra que IA

generativa e privacidade corporativa não são incompatíveis, são conciliáveis quando a engenharia é feita com rigor.

No fundo, a principal lição do ScopeReview AI é simples: o valor não está nos modelos em si, mas na forma como são orquestrados, governados e mantidos dentro dos limites da organização.

Referências

[1] L. Wang *et al.*, "A Survey on Large Language Model based Autonomous Agents," *Front. Comput. Sci.*, vol. 18, no. 6, pp. 1–42, Mar. 2025, doi: 10.1007/s11704-024-40231-1.

[2] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," *2013 35th International Conference on Software Engineering (ICSE)*, pp. 712–721, May 2013, doi: 10.1109/ICSE.2013.6606617.

[3] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Adv. Neural Inf. Process. Syst.*, vol. 35, Jan. 2023, doi: 10.48550/arXiv.2201.11903.

[4] V. Balachandran, "Reducing human effort and improving quality in peer code reviews using automatic static analysis and reviewer recommendation," *2013 35th International Conference on Software Engineering (ICSE)*, pp. 931–940, 2013, doi: 10.1109/ICSE.2013.6606642.

[5] GitHub, "About GitHub Copilot Pull Request Summaries." Accessed: Mar. 14, 2026. [Online]. Available: https://docs.github.com/en/copilot/responsible-use/pull-request-summaries

[6] GitLab, "GitLab Duo Code Review | GitLab Docs." Accessed: Mar. 14, 2026. [Online]. Available: https://docs.gitlab.com/user/gitlab_duo/code_review_classic/

[7] CodeRabbit, "CodeRabbit Documentation - AI code reviews on pull requests, IDE, and CLI." Accessed: Mar. 14, 2026. [Online]. Available: https://docs.coderabbit.ai/

[8] A. AL-Maamari, "Can You Trust Your Copilot? A Privacy Scorecard for AI Coding Assistants," vol. 1, Sep. 2025, Accessed: Mar. 20, 2026. [Online]. Available: http://arxiv.org/abs/2509.20388

[9] Olivia Powell, "Samsung employees allegedly leak data via ChatGPT." Accessed: May 23, 2026. [Online]. Available: https://www.cshub.com/data/news/iotw-samsung-employees-allegedly-leak-proprietary-information-via-chatgpt

[10] Microsoft, "Azure DevOps Services REST API Reference (Version 7.2)." Accessed: Mar. 14, 2026. [Online]. Available: https://learn.microsoft.com/en-us/rest/api/azure/devops/?view=azure-devops-rest-7.2

[11] Microsoft, "2025 Responsible AI Transparency Report", Accessed: Mar. 23, 2026. [Online]. Available: https://www.microsoft.com/en-us/corporate-responsibility/responsible-ai-transparency-report/

[12] "AI features | SonarQube Server 10.7 | Sonar Documentation." Accessed: May 23, 2026. [Online]. Available: https://docs.sonarsource.com/sonarqube-server/10.7/user-guide/ai-features

[13] "The AI Code Review Platform - Qodo Documentation." Accessed: May 23, 2026. [Online]. Available: https://docs.qodo.ai/

[14] U. Cihan, V. Haratian, E. Tüzün, and A. İçöz, "Evaluating Large Language Models for Code Review," May 2025, doi: 10.5281/ZENODO.14962566.

[15] U. Cihan *et al.*, "Automated Code Review In Practice," *IEEE/ACM International Conference on Software Engineering - Software Engineering in Practice*, no. 2025, pp. 425–436, Dec. 2024, Accessed: Mar. 20, 2026. [Online]. Available: http://arxiv.org/abs/2412.18531

[16] S. Goldman *et al.*, "What Types of Code Review Comments Do Developers Most Frequently Resolve?," Oct. 2025, doi: 10.48550/arXiv.2510.05450.

[17] B. İçöz, "Automated Code Review Using Large Language Models with Symbolic Reasoning," Jul. 2025, doi: 10.48550/ARXIV.2507.18476.

[18] Md. A. Haider and T. Zimmermann, "Understanding Dominant Themes in Reviewing Agentic AI-authored Code," *23rd International Conference on Mining Software Repositories (MSR '26), April 13–14, 2026, Rio de Janeiro, Brazil*, vol. 1, Jan. 2026, doi: 10.1145/3793302.3793566.

[19] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," *12th International Conference on Learning Representations, ICLR 2024*, Aug. 2023, Accessed: May 08, 2026. [Online]. Available: https://arxiv.org/pdf/2308.00352

[20] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using Pre-Trained Models to Boost Code Review Automation," *Proceedings - International Conference on Software Engineering*, vol. 2022-May, pp. 2291–2302, Jan. 2022, Accessed: Mar. 20, 2026. [Online]. Available: http://arxiv.org/abs/2201.06850

[21] S. Lu *et al.*, "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation," *Adv. Neural Inf. Process. Syst.*, Feb. 2021, Accessed: May 23, 2026. [Online]. Available: https://arxiv.org/pdf/2102.04664

[22] C. Pornprasit and C. Tantithamthavorn, "JITLine: A Simpler, Better, Faster, Finer-grained Just-In-Time Defect Prediction," Mar. 2021, doi: 10.1109/MSR52588.2021.00028.

[23] Z. Li *et al.*, "Automating Code Review Activities by Large-Scale Pre-training," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022)*, 2022. doi: 10.1145/3540250.3549081.

[24] Z. Ji *et al.*, "Survey of Hallucination in Natural Language Generation," *ACM Comput. Surv.*, vol. 55, no. 12, Jul. 2024, doi: 10.1145/3571730.

[25] E. Evans, *Domain-Driven Design Tackling Complexity in the Heart of Software*, 1st ed. Boston, MA, USA: Addison-Wesley, 2003.

[26] A. Grattafiori *et al.*, "The Llama 3 Herd of Models," Nov. 2024, Accessed: Mar. 14, 2026. [Online]. Available: http://arxiv.org/abs/2407.21783

[27] Qwen *et al.*, "Qwen2.5 Technical Report," Dec. 2024, Accessed: May 02, 2026. [Online]. Available: https://arxiv.org/pdf/2412.15115

[28] DeepSeek-AI *et al.*, "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models," Dec. 2025, Accessed: May 02, 2026. [Online]. Available: https://arxiv.org/pdf/2512.02556

[29] OpenAI, "OpenAI o3 and o4-mini System Card | OpenAI." Accessed: May 02, 2026. [Online]. Available: https://openai.com/index/o3-o4-mini-system-card/

[30] Q. Team, "Qwen3 Technical Report," 2025, Accessed: Mar. 21, 2026. [Online]. Available: https://arxiv.org/abs/2505.09388

[31] Microsoft *et al.*, "Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs," Mar. 2025, Accessed: May 02, 2026. [Online]. Available: https://arxiv.org/pdf/2503.01743

[32] J. Hassine, "An LLM-based Approach to Recover Traceability Links between Security Requirements and Goal Models," *ACM International Conference Proceeding Series*, vol. 1, pp. 643–651, Jun. 2024, doi: 10.1145/3661167.3661261.

[33] Y. Cai, R. Li, P. Liang, M. Shahin, and Z. Li, "Designing LLM-based Multi-Agent Systems for Software Engineering Tasks: Quality Attributes, Design Patterns and Rationale," *ACM Transactions on Software Engineering and Methodology*, vol. 0, Dec. 2025, doi: 10.48550/arXiv.2511.08475.

[34] "Distributed Locks with Redis | Docs." Accessed: May 07, 2026. [Online]. Available: https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

[35] H. Jiang, Q. Wu, C. Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," *EMNLP 2023 - 2023 Conference on Empirical Methods in Natural Language Processing, Proceedings*, pp. 13358–13376, Oct. 2023, doi: 10.18653/v1/2023.emnlp-main.825.

[36] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, Jul. 2023, doi: 10.1162/tacl_a_00638.

[37] Y. Moslem and J. D. Kelleher, "Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey," Feb. 2026, Accessed: May 03, 2026. [Online]. Available: https://arxiv.org/pdf/2603.04445

[38] R. C. Martin and R. C. Martin, "Clean architecture: a craftsman's guide to software structure and design," *Robert C. Martin series*, p. 404, 2018.

[39] W. Cai, J. Jiang, F. Wang, J. Tang, S. Kim, and J. Huang, "A Survey on Mixture of Experts in Large Language Models," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 7, pp. 3896–3915, Apr. 2025, doi: 10.1109/TKDE.2025.3554028.

[40] N. Shazeer *et al.*, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," *5th International Conference on Learning Representations, ICLR 2017 -*

Conference Track Proceedings, Jan. 2017, Accessed: May 16, 2026. [Online]. Available: https://arxiv.org/pdf/1701.06538

[41] W. Takerngsaksiri *et al.*, "Human-In-the-Loop Software Development Agents," *IEEE/ACM International Conference on Software Engineering - Software Engineering in Practice*, no. 2025, pp. 342–352, Nov. 2024, doi: 10.1109/ICSE-SEIP66354.2025.00036.

[42] "FastAPI." Accessed: May 03, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[43] "Exponential Backoff And Jitter | AWS Architecture Blog." Accessed: May 07, 2026. [Online]. Available: https://aws.amazon.com/pt/blogs/architecture/exponential-backoff-and-jitter/

[44] M. Seemann and S. van Deursen, *Dependency Injection Principles, Practices, and Patterns*, 1st ed. Shelter Island, NY: Manning Publications, 2019.

[45] B. T. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," Jul. 2023, Accessed: May 08, 2026. [Online]. Available: https://arxiv.org/pdf/2307.09702

[46] S. Abdelnabi, K. Greshake, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," *AISec 2023 - Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pp. 79–90, Feb. 2023, doi: 10.1145/3605764.3623985.

[47] F. Jackson, "A Unified Governance Architecture for Agentic AI: Integrating Microsoft Azure AI Foundry, Cloud-Native Guardrails, Frontier AI Safety Research, Zero-Trust Principles, and the 4th.GRC Governance Engine," Nov. 2025, doi: 10.2139/ssrn.5819502.

[48] F. Xu *et al.*, "Toward large reasoning models: A survey of reinforced reasoning with large language models," *Patterns*, vol. 6, no. 10, Oct. 2025, doi: 10.1016/j.patter.2025.101370.

[49] C. Wohlin, M. Höst, and K. Henningsson, "Empirical Research Methods in Software Engineering," *Lecture Notes in Computer Science*, vol. 2765, pp. 7–23, 2003, doi: 10.1007/978-3-540-45143-3_2.

[50] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," *12th International Conference on Learning Representations, ICLR 2024*, Oct. 2023, Accessed: May 23, 2026. [Online]. Available: https://arxiv.org/pdf/2310.06770

Anexos

Anexo A — Ata de Início de Estágio

A.1 Ata de Início de Estágio

Proposta consolidada

Título
Integração de Code Review com IA no Azure DevOps

Motivação
O Code Review manual é uma prática essencial mas dispendiosa. A fadiga do revisor, a inconsistência entre análises e a acumulação de Pull Requests em equipas com alta cadência de entrega comprometem a qualidade e atrasam o ciclo de desenvolvimento. A DevScope opera sobre o Azure DevOps, uma plataforma que, ao contrário do GitHub e do GitLab, não dispõe de qualquer mecanismo nativo de revisão assistida por IA. As alternativas comerciais existentes resolvem parcialmente o problema, mas implicam o envio de código-fonte proprietário para servidores externos — um risco inaceitável em ambientes empresariais onde o código representa o principal ativo intelectual dos clientes.

Descrição
O processo de revisão de código na DevScope apresenta três limitações estruturais: inconsistência entre revisores, latência na integração do código e ausência de uma primeira linha de defesa automatizada para verificações repetitivas como conformidade com guidelines e padrões de segurança. A inexistência de uma solução nativa no Azure DevOps que cubra estas dimensões sem comprometer a soberania sobre o código-fonte constitui o problema concreto que este projeto se propõe resolver.

Objetivos
Desenvolver uma Prova de Conceito (PoC) para um sistema de Code Review automático com IA, integrado diretamente no Azure DevOps. A solução deverá:
– Analisar automaticamente as alterações submetidas nos Pull Requests.
– Verificar o cumprimento de guidelines do repositório, detetar problemas de segurança e sugerir melhorias.
– Gerar um resumo técnico automático do PR.
– Minimizar custos e garantir a privacidade do código, usando LLMs locais ou com APIs gratuitas.

Projeto

Antevisão da solução a ser desenvolvida

Qual a (proposta de) solução preconizada para resolver o problema e atingir o(s) objetivo(s)?
A solução assenta num Agente de IA que interage com o Azure DevOps através dos seguintes componentes:
– Gatilho de Execução — Webhooks do Azure DevOps para detetar a criação ou atualização de um Pull Request.
– Processamento — Serviço backend que usa a REST API do Azure DevOps para extrair as diferenças de código (git diff).
– Motor de IA — LLMs open-source (ex: Llama 3 ou Phi-3) em ambiente Docker, alojados no Azure, garantindo baixo custo e confidencialidade.
– Feedback Automático — A resposta do LLM é inserida como comentários em linhas específicas do código no PR correspondente.

Qual a proposta de validação da solução a desenvolver?
A avaliação será feita através de uma análise comparativa estruturada entre o Code Review realizado por engenheiros humanos e o gerado pela IA, submetendo Pull Requests históricos (com bugs conhecidos ou comentários já documentados) ao agente. Os resultados serão medidos com base em:

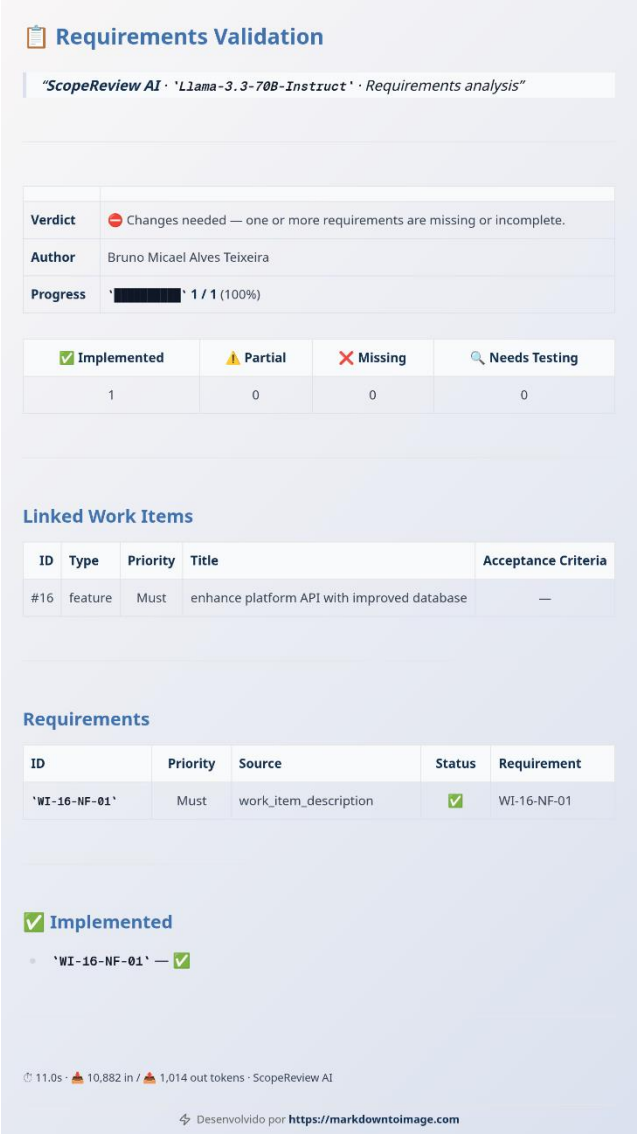
Anexo B — Exemplos de Output do Sistema

B.1 Screenshot: Comentário de Code Review publicado no ADO



Figura B. 1 - Exemplo de comentário de Code Review gerado pelo agente o4-mini e publicado nativamente no Azure DevOps (Elaboração própria)

B.2 Screenshot: Comentário de Requirement Review publicado no ADO



📋 Requirements Validation

"ScopeReview AI · `Llama-3.3-70B-Instruct` · Requirements analysis"

Verdict	⛔ Changes needed — one or more requirements are missing or incomplete.
Author	Bruno Micael Alves Teixeira
Progress	`[redacted]` **1 / 1** (100%)

✅ Implemented	⚠️ Partial	❌ Missing	🔍 Needs Testing
1	0	0	0

Linked Work Items

ID	Type	Priority	Title	Acceptance Criteria
#16	feature	Must	enhance platform API with improved database	—

Requirements

ID	Priority	Source	Status	Requirement
`WI-16-NF-01`	Must	work_item_description	✅	WI-16-NF-01

✅ Implemented

- `WI-16-NF-01` — ✅

⏱ 11.0s · 📥 10,882 in / 📤 1,014 out tokens · ScopeReview AI

Desenvolvido por **https://markdowntoimage.com**

Figura B. 2 - Exemplo de comentário de Requirement Review gerado pelo agente DeepSeek-V3.2, validando conformidade face aos Acceptance Criteria (Elaboração própria)

B.3 Screenshot: Terminal ngrok (túnel HTTPS ativo)



Figura B. 3 - Terminal com túnel ngrok ativo, expondo o endpoint local para receção de Webhooks do Azure DevOps em ambiente de desenvolvimento (Elaboração própria)

B.4 Screenshot: Logs Docker em estado idle

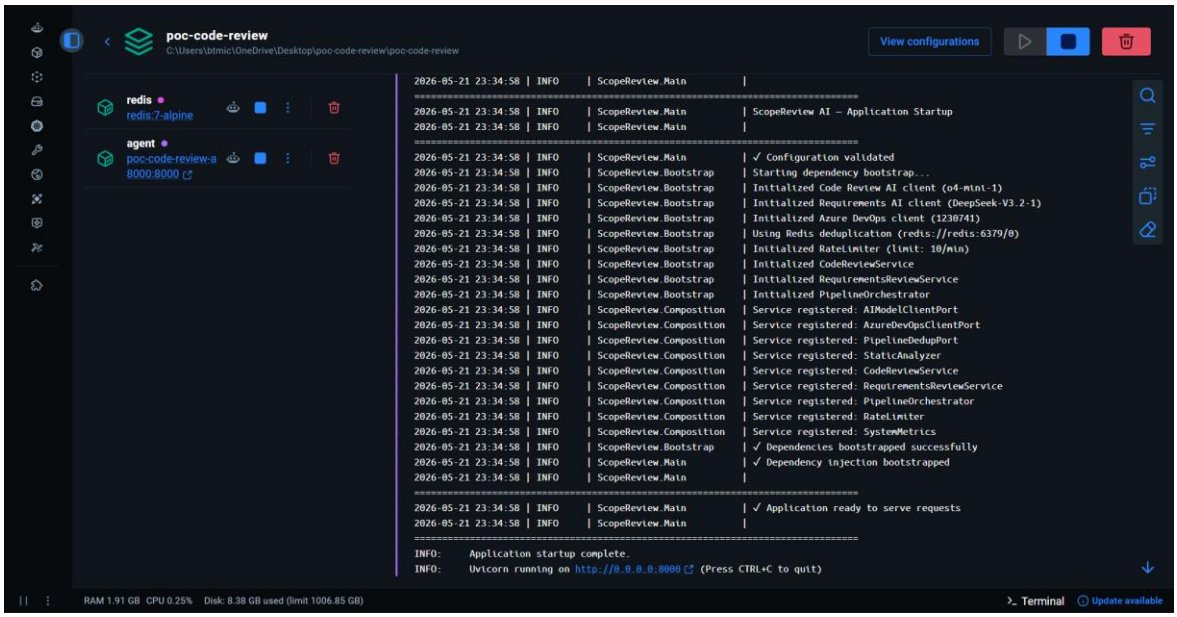


Figura B. 4 - Logs do contentor Docker em estado idle aguardando evento (Elaboração própria)

B.5 Screenshot: Logs Docker em processamento de PR

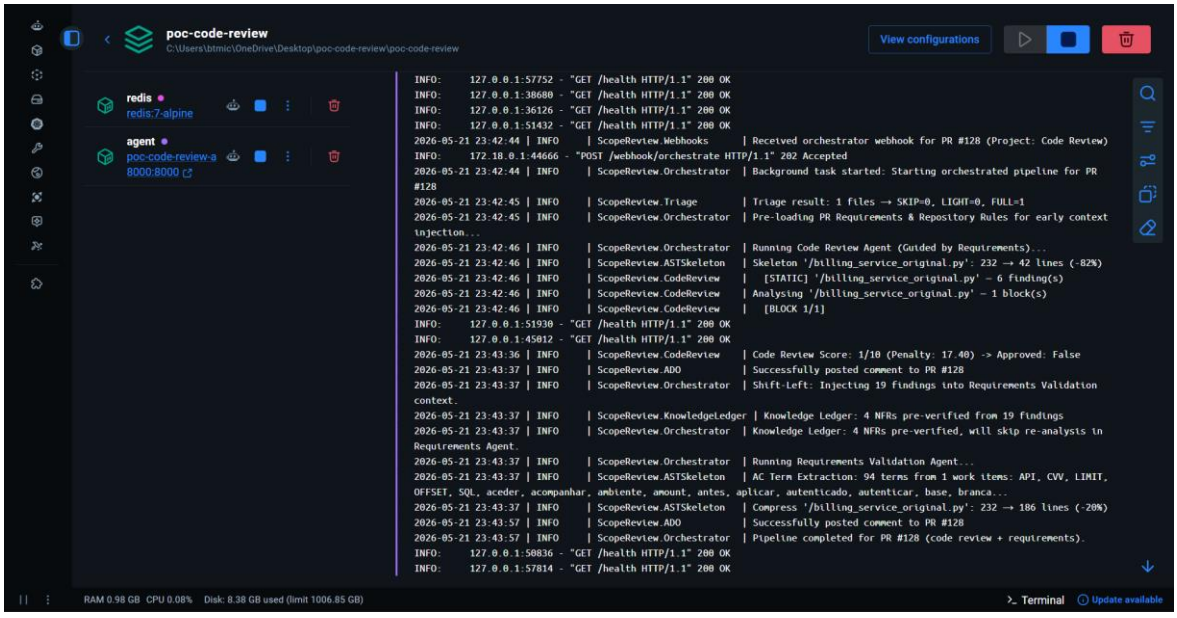


Figura B. 5 - Logs do contentor Docker:pipeline de análise multiagente em execução após trigger de Pull Request (Elaboração própria)

B.6 Screenshot: Relatório de Cobertura de Testes Unitários

```
Name                                      Stmts   Miss  Cover
-----------------------------------------------------------
src\__init__.py                               0      0   100%
src\api\webhooks.py                          80      0   100%
src\bootstrap.py                             55      5    91%
src\composition.py                           22      3    86%
src\core\ast_skeleton.py                    184     18    90%
src\core\config.py                           79      3    96%
src\core\di.py                               35      1    97%
src\core\knowledge_ledger.py                 31      0   100%
src\core\llm_json.py                         29      0   100%
src\core\logger.py                            6      0   100%
src\core\metrics.py                          31      1    97%
src\core\pipeline_dedup.py                   30      3    90%
src\core\rate_limit.py                       21      1    95%
src\core\resilience.py                       33      0   100%
src\core\triage.py                           39      2    95%
src\domain\__init__.py                        2      0   100%
src\domain\requirements_verdict.py           56      5    91%
src\infra\azure_ai.py                        43      2    95%
src\infra\azure_devops.py                   190     26    86%
src\infra\redis_dedup.py                     30      6    80%
src\main.py                                  63      8    87%
src\models\ai_models.py                      43      2    95%
src\models\webhooks.py                       16      0   100%
src\ports\__init__.py                         4      0   100%
src\ports\ado_client.py                       3      0   100%
src\ports\ai_client.py                        4      0   100%
src\ports\dedup.py                            5      0   100%
src\services\code_review.py                 136     14    90%
src\services\orchestrator.py                 97     21    78%
src\services\requirements_review.py          37      2    95%
src\services\static_analyzer.py              96      4    96%
src\templates\markdown.py                   158     17    89%
src\templates\prompts.py                     36      3    92%
-----------------------------------------------------------
TOTAL                                      1694    147    91%
Required test coverage of 90% reached. Total coverage: 91.32%
201 passed, 56 warnings in 11.28s
```

Figura B. 6 - Relatório de cobertura de testes unitários (pytest-cov) (Elaboração própria)

Anexo C — Resultados Completos da Avaliação

O presente anexo consolida os dados brutos recolhidos ao longo dos seis testes de benchmark que sustentam as conclusões dos capítulos 4 e 5. As três tabelas que se seguem permitem a verificação formal dos resultados: a Tabela C.1 apresenta a matriz de confusão global por arquitetura (verdadeiros e falsos positivos/negativos); a Tabela C.2 discrimina as métricas operacionais cruas de cada uma das dezoito execuções realizadas; e a Tabela C.3 identifica os Pull Requests submetidos, garantindo rastreabilidade. Todos os dados resultam de execuções reais sobre o Azure AI Foundry, com preços de referência de março de 2026 com os preços de referência da Tabela 4.

Tabela C.1 — Matriz de confusão global por arquitetura

Consolidação das métricas de classificação binária (detetado vs. não detetado) para os 102 defeitos injetados nos seis testes, expressa na notação padrão de avaliação de classificadores. Os Verdadeiros Positivos correspondem a defeitos corretamente identificados; os Falsos Negativos a defeitos presentes, mas não detetados. Elaboração própria, com base em dados recolhidos no Azure AI Foundry (março 2026).

Tabela C. 1 - Matriz de confusão global por arquitetura com métricas de Precisão, Revocação e F1-Score (Elaboração Própria)

Arquitetura	Verdadeiros Positivos	Falsos Positivos	Falsos Negativos	Precisão	Revocação	F1
Monolítica (Llama)	26	~8	76	~0,76	0,25	0,38
2x Llama	45	~6	57	~0,88	0,44	0,59
Multiagente	90	~4	12	~0,96	0,88	0,92

Tabela C.2 — Métricas operacionais por execução

Registo cronológico das dezoito execuções completas (seis testes × três arquiteturas), apresentando latência de ponta a ponta (desde a receção do webhook até à publicação do comentário), consumo de tokens de entrada e saída, e custo operacional calculado com os preços de referência da Tabela 4.

Nota metodológica sobre extrapolações (~): Em três execuções da arquitetura Multiagente sobre os testes Massive (Mv1, Mv2, Mv3), o registo exato de tokens foi interrompido devido a timeout do cliente HTTP durante a fase de streaming da API do Azure AI Foundry. Os valores de tokens foram extrapolados com base na fórmula de regressão linear derivada das 15 execuções com registo completo: tokens ≈ 18,2 × linhas_de_código + 3.200 (R^2 = 0,91). Os valores de latência correspondem à média aritmética das reexecuções subsequentes do mesmo teste sobre o mesmo PR. O custo foi recalculado a partir dos tokens extrapolados. Elaboração própria, com base em dados reais do Azure AI Foundry (março 2026).

Tabela C. 2 - Métricas operacionais detalhadas (latência, consumo de tokens e custo) das dezoito execuções do protocolo experimental (Elaboração Própria)

Teste	Arquitetura	Latência (s)	Tokens input	Tokens output	Custo (EUR)
Medium 1	Monolítica	35,5	5.453	1.772	0,005
Medium 1	2x Llama	37,5	7.238	1.832	0,006
Medium 1	Multiagente	60,3	7.347	4.903	0,027
Medium 2	Monolítica	6,8	5.850	2.087	0,006
Medium 2	2x Llama	30,7	7.237	1.804	0,006

Medium 2	Multiagente	71,0	7.369	8.692	0,034
Medium 3	Monolítica	12,5	6.392	1.844	0,006
Medium 3	2x Llama	21,9	7.956	2.172	0,007
Medium 3	Multiagente	36,8	8.165	5.677	0,022
Massive 1	Monolítica	10,7	14.684	1.355	0,011
Massive 1	2x Llama	~45,0	~13.000	~2.000	0,011
Massive 1	Multiagente	~95,0	~15.000	~6.500	0,050
Massive 2	Monolítica	11,0	10.882	1.014	0,008
Massive 2	2x Llama	~35,0	~12.000	~1.800	0,010
Massive 2	Multiagente	~110,0	~16.000	~7.000	0,058
Massive 3	Monolítica	44,3	11.786	2.026	0,010
Massive 3	2x Llama	57,1	15.668	3.175	0,013
Massive 3	Multiagente	~105,0	~16.500	~8.000	0,065

Média	Monolítica	20,1	9.175		0,008
Média	2x Llama	37,9	10.517		0,009
Média	Multiagente	79,7	11.730		0,043

Tabela C.3 — Identificação dos Pull Requests de teste

Registo de rastreabilidade dos seis Pull Requests submetidos no repositório Azure DevOps da DevScope durante a fase de benchmark, permitindo a auditoria posterior dos artefactos originais. Os PRs permanecem disponíveis no repositório com o histórico completo de comentários publicados pelo sistema. Elaboração própria.

Tabela C. 3 - Identificação e rastreabilidade dos Pull Requests submetidos durante a fase de benchmark (Elaboração Própria)

ID Teste	PR ID	Bugs injetados	ACs violados

Medium 1	140	14	9/12
Medium 2	141	14	11/12
Medium 3	142	14	11/12
Massive 1	137	20	8/12
Massive 2	145	20	11/12
Massive 3	139	20	12/12

Anexo D — Diagramas Arquiteturais e de Processo (Renderizados a partir de PlantUML)

D.1 Modelo de Domínio DDD (Entidades)

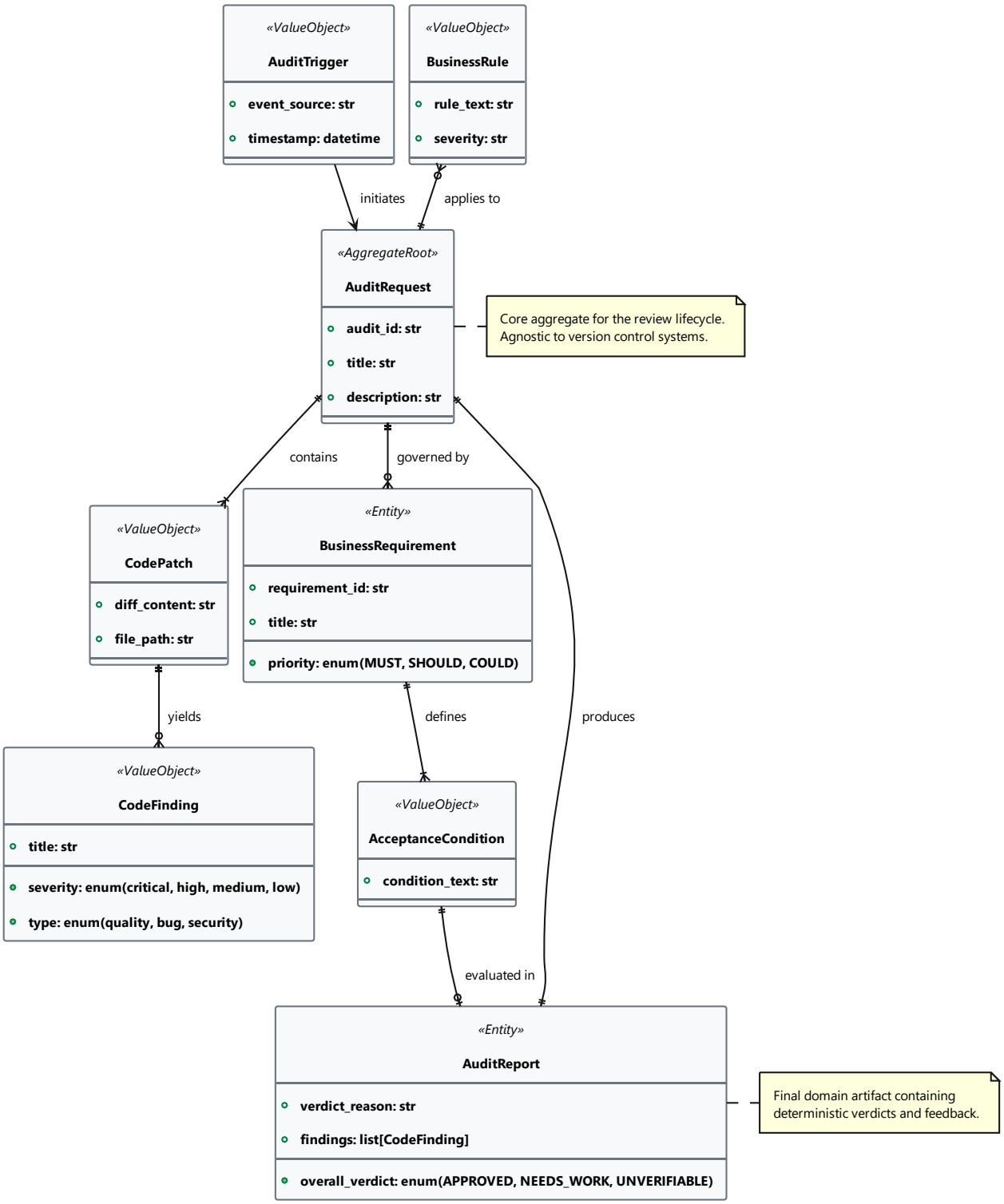


Figura D. 1 – Modelo de Domínio DDD (Elaboração própria)

D.2 Camadas de Domínio e Aplicação

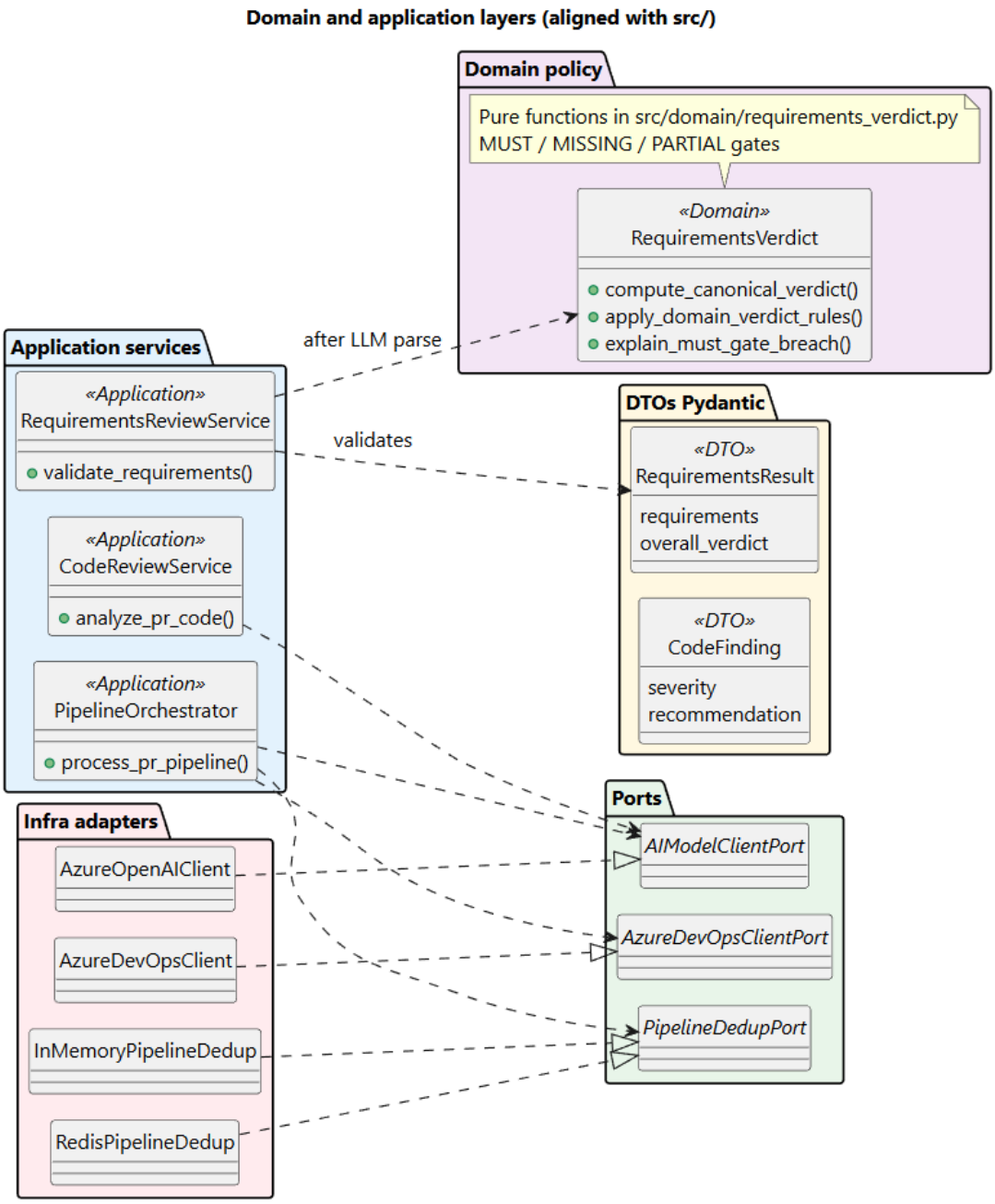


Figura D. 2 -Vista de Contentores (C4) com orquestração FastAPI, Redis e clientes de IA (Elaboração própria)

D.3 Sequência Completa de Orquestração

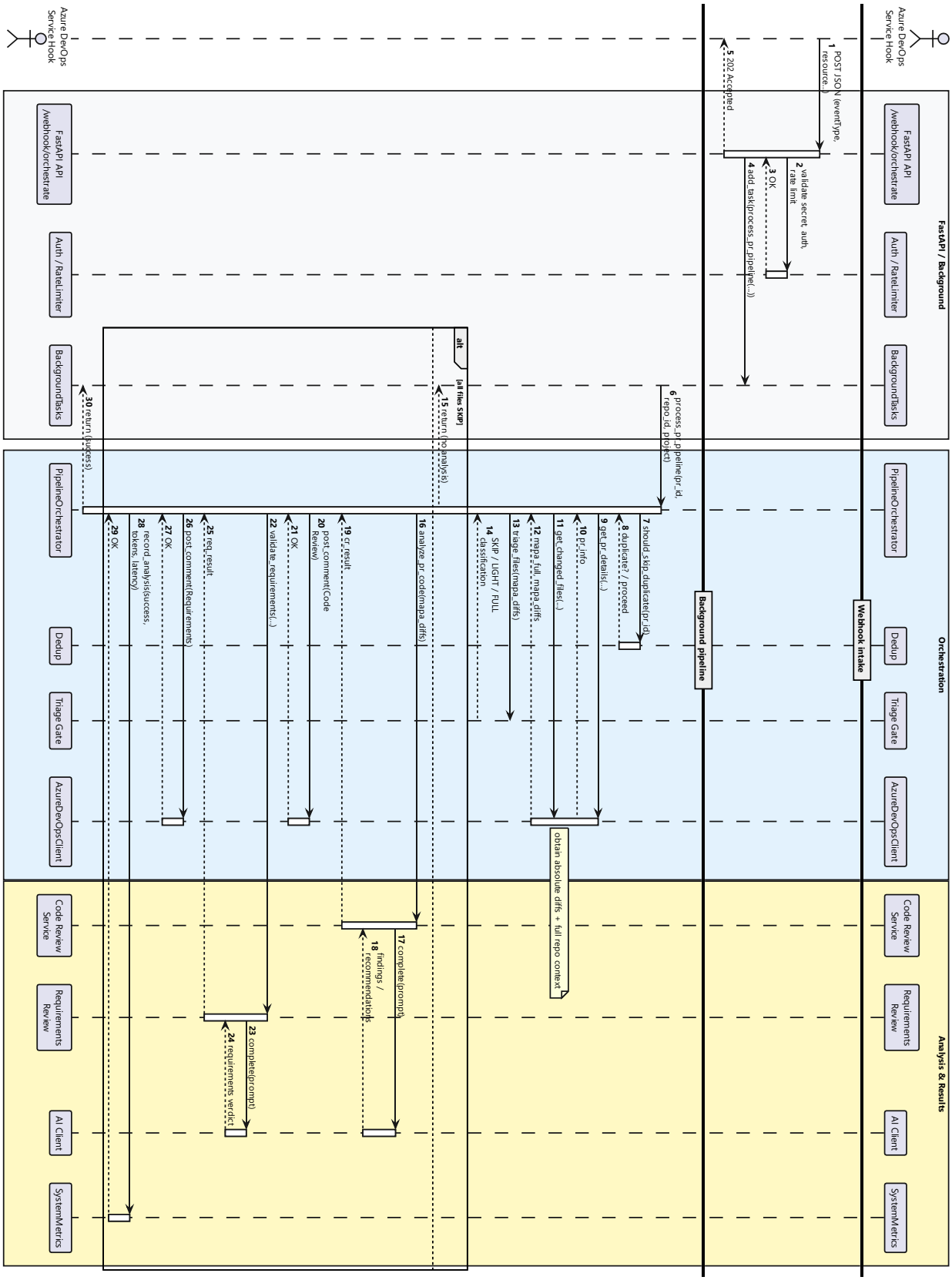


Figura D. 3 - Sequência completa de orquestração multiagente e partilha de contexto via Knowledge Ledger (Elaboração própria)

D.4 C4 System Context

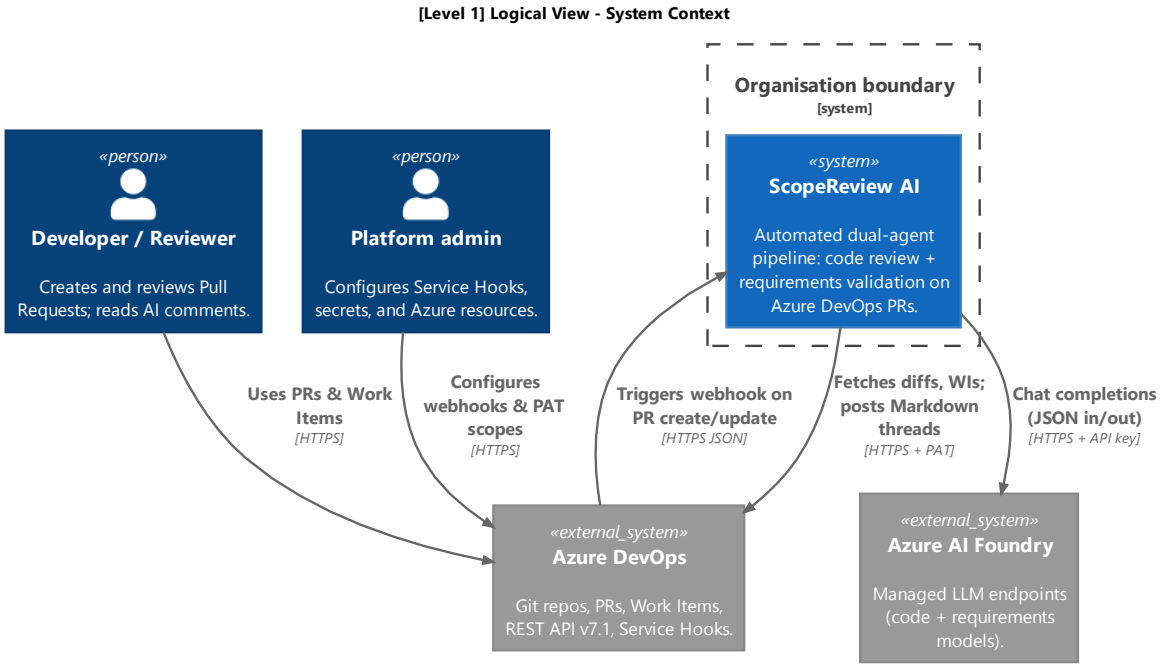


Figura D. 4 - Diagrama de Contexto de Sistema C4 - Nível 1 (Vista Lógica) (Elaboração própria)

D.5 C4 Container Diagram

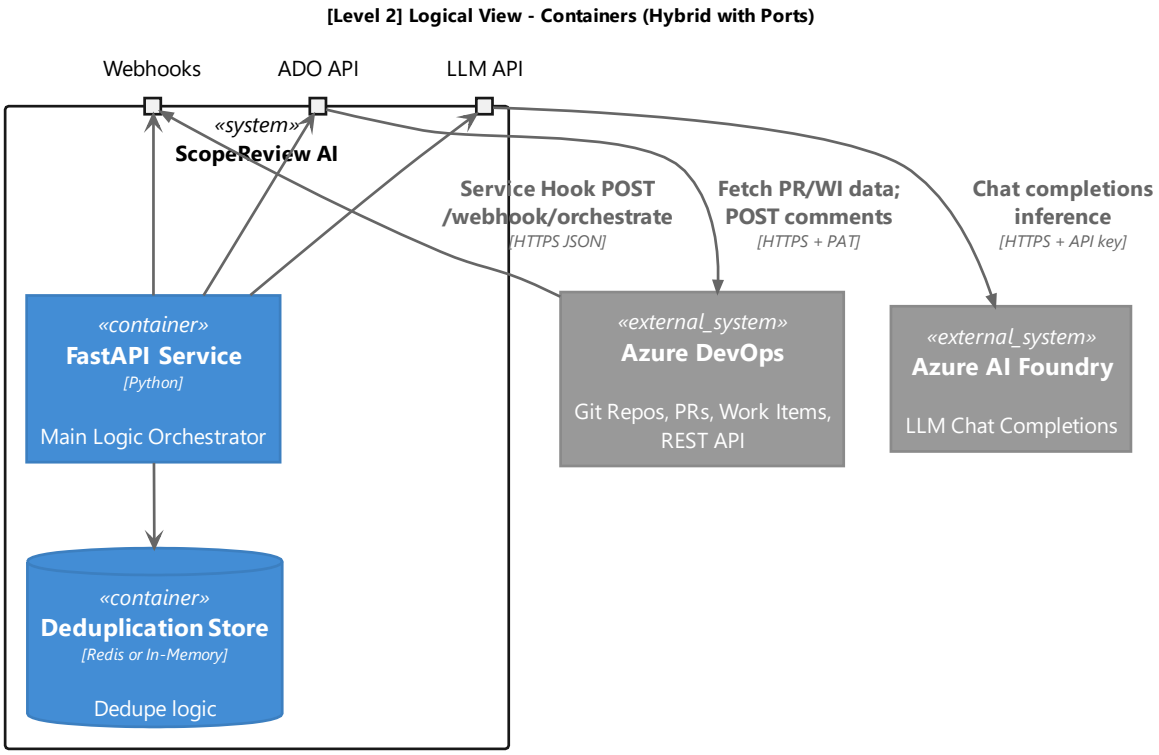


Figura D. 5 - Diagrama de Contentores C4 - Nível 2 (Vista Lógica Híbrida com Ports) (Elaboração própria)

D.6 C4 Component Diagram

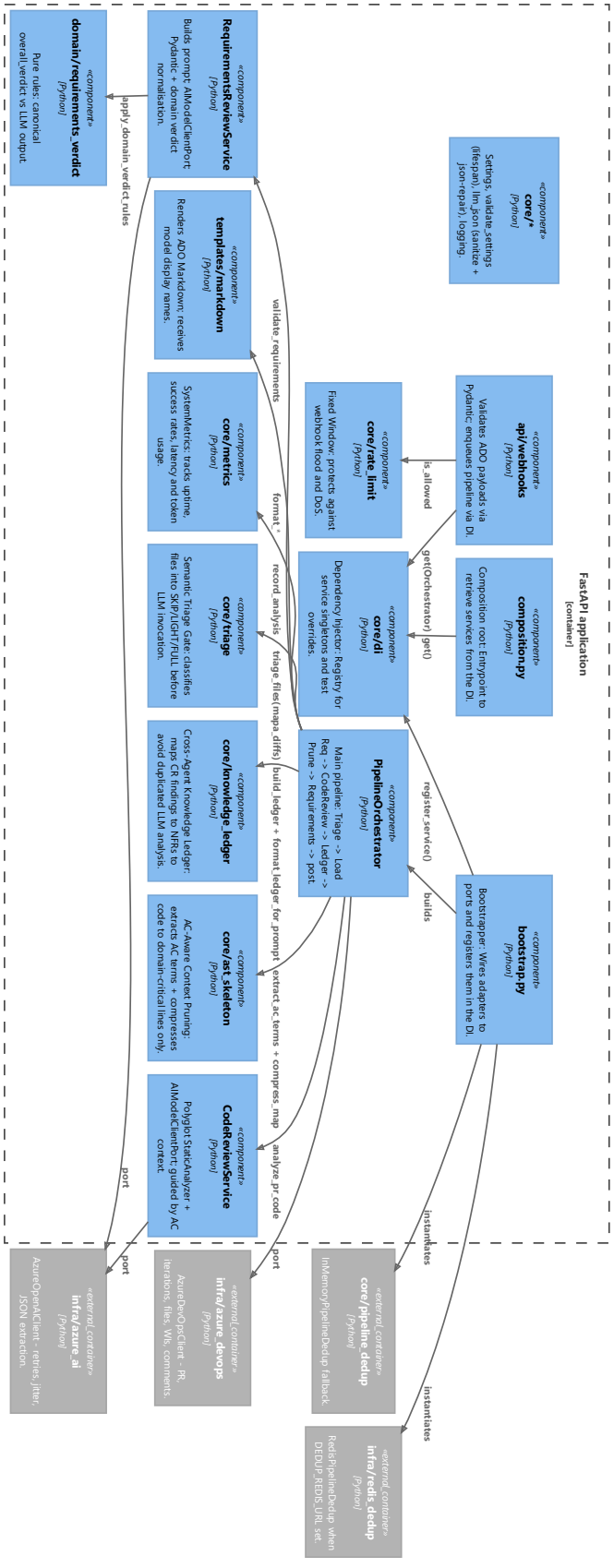


Figura D. 6 - Diagrama de Componentes C4 - Nível 3 (Vista Lógica Interna do Serviço FastAPI) (Elaboração própria))

D.7 Vista de Deployment (Docker Compose)

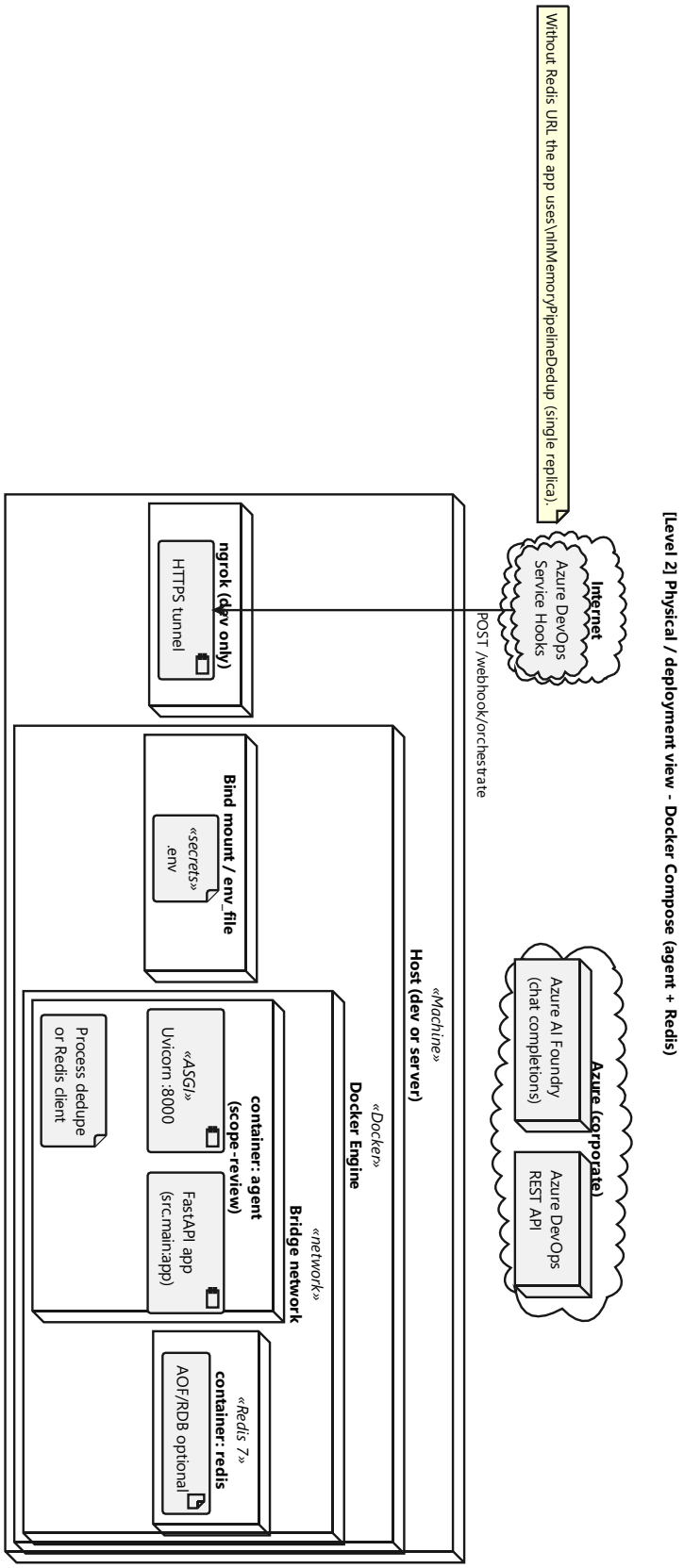


Figura D. 7 -Vista de Deployment (Docker Compose, Bind Mounts, Rede Bridge, ngrok) (Elaboração própria)

D.8 Fluxo de Atividade do Sistema

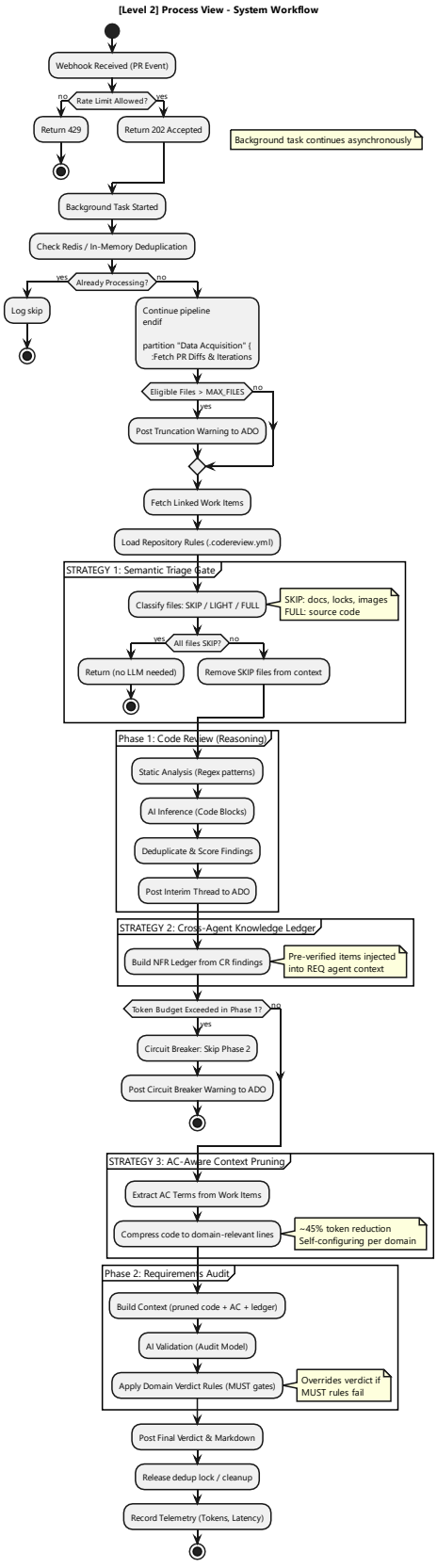


Figura D. 8 - Fluxo de Atividade do Pipeline (Triage Gate, Phase 1, Phase 2, Governação) (Elaboração própria)

D.9 Diagrama de Classes Granular

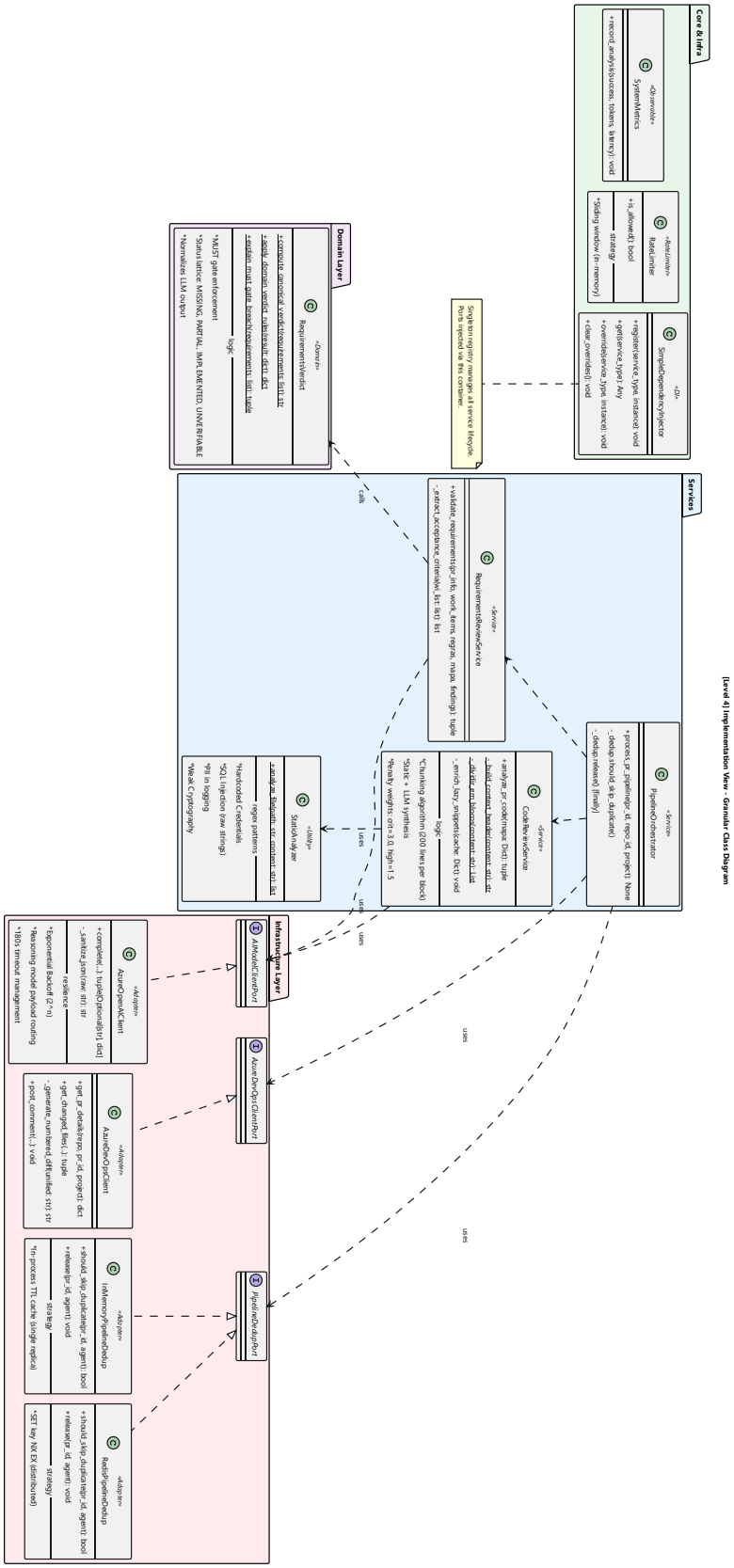


Figura D. 9 - Diagrama de Classes Granular (Domain Core, Ports & Adapters) (Elaboração própria)

D.10 Padrões de Design (Clean Architecture)

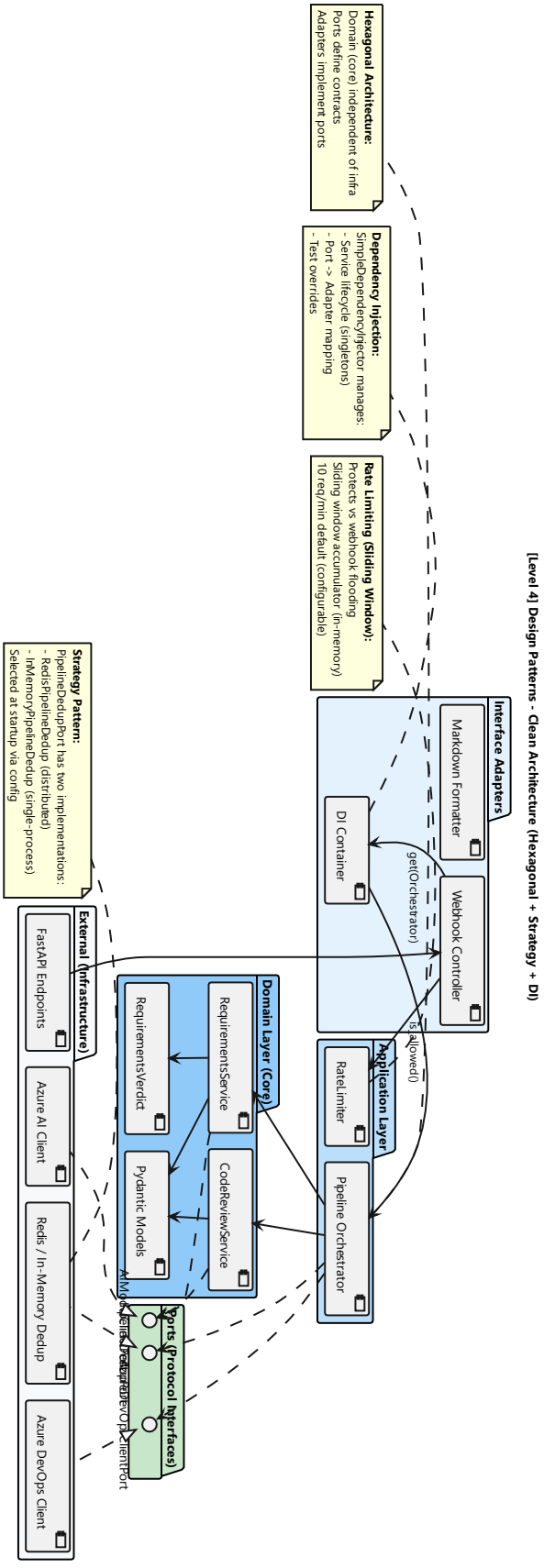


Figura D. 10 - Padrões de Design Aplicados (Clean Architecture, Strategy, Dependency Injection) (Elaboração própria)

D.11 Sequência de Deduplicação Distribuída (Redis)

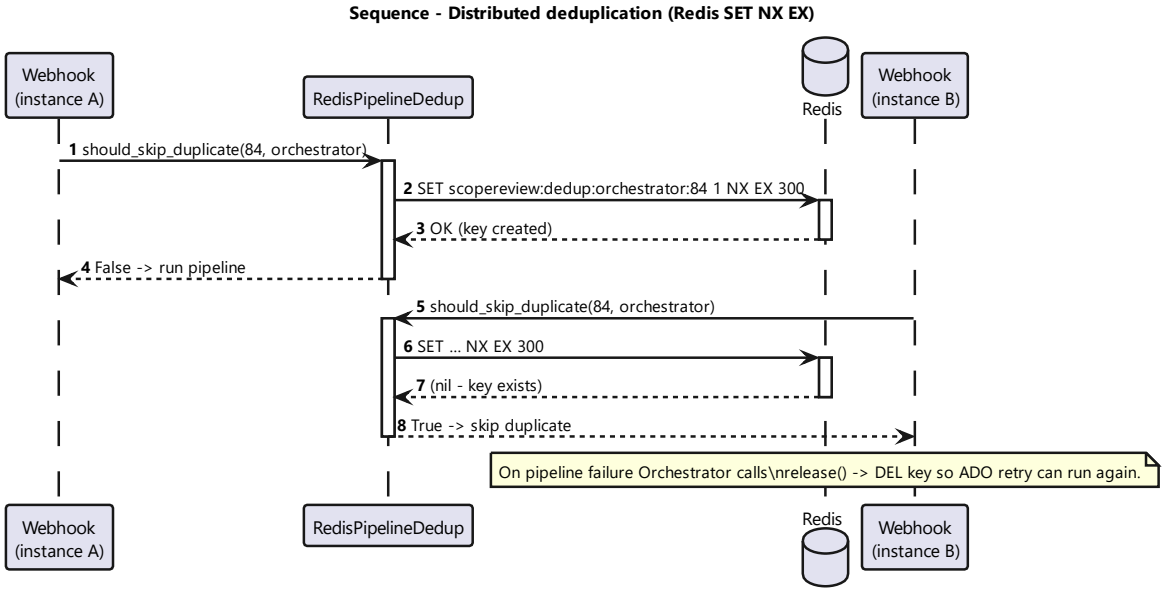


Figura D. 11 - Sequência de Deduplicação Distribuída via Redis (SET NX EX) (Elaboração própria)

D.12 Mapeamento Lógico Hexagonal

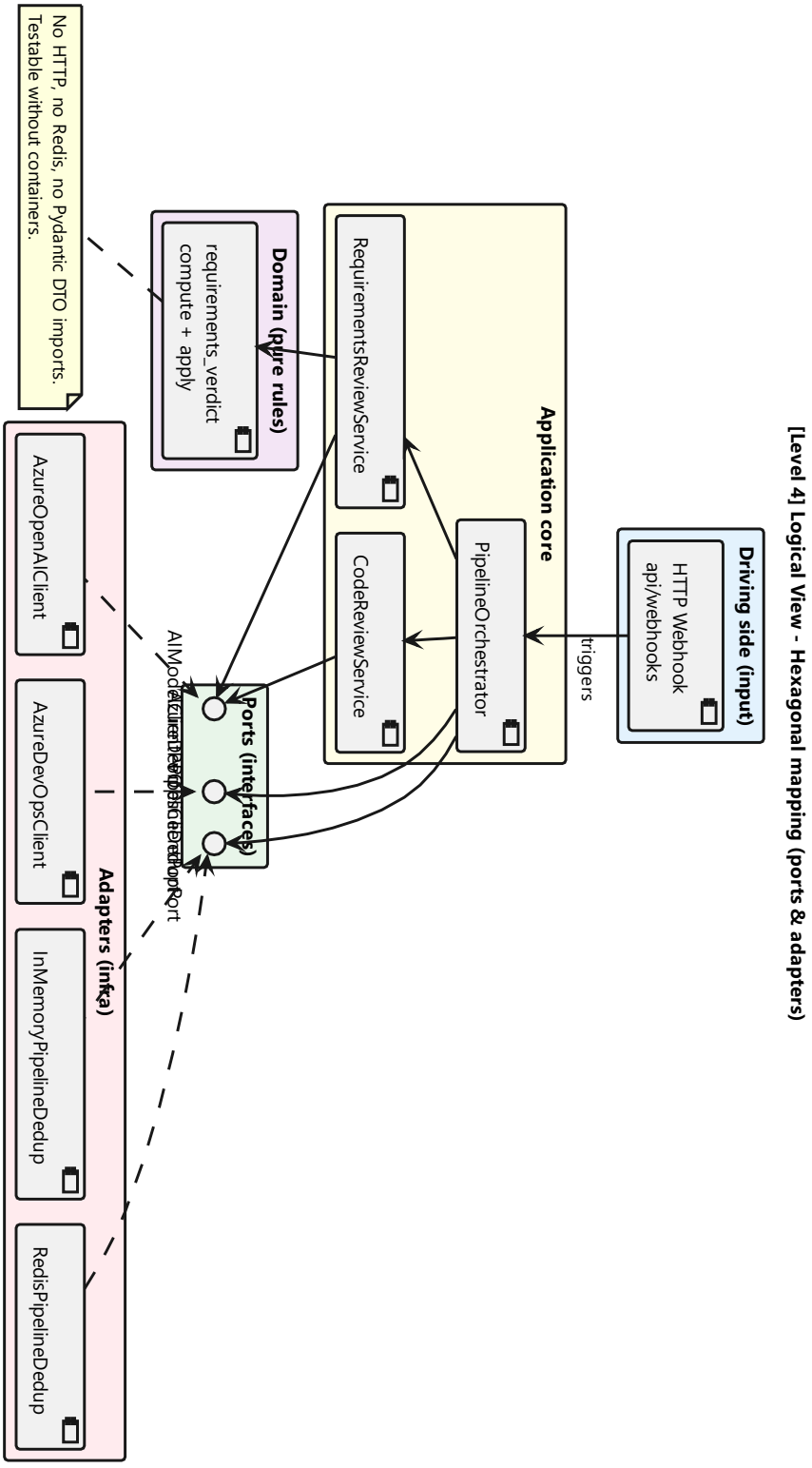


Figura D. 12 - Mapeamento Lógico Hexagonal (Ports & Adapters). (Elaboração própria)